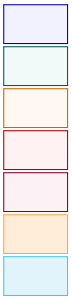
Mathematical Foundations, Architecture Diagrams, and State Analysis of All Modern LLM Training Methods

Chief Strategist J

June 13, 2026

Colour Legend



Document structure: This reference covers **85 LLM training methods** organised into 14 parts. Each method includes: (1) the core mathematical objective, (2) a TikZ architecture diagram, and (3) a before/after state table.

Contents

Contents

Part I
Foundation Training Methods

Pretraining

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \ldots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{1}$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \tag{2}$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{3}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \tag{4}$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{5}$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t. The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{6}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{7}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{8}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{9}$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \tag{10}$$

Architecture Flow Diagram

State Transition Table

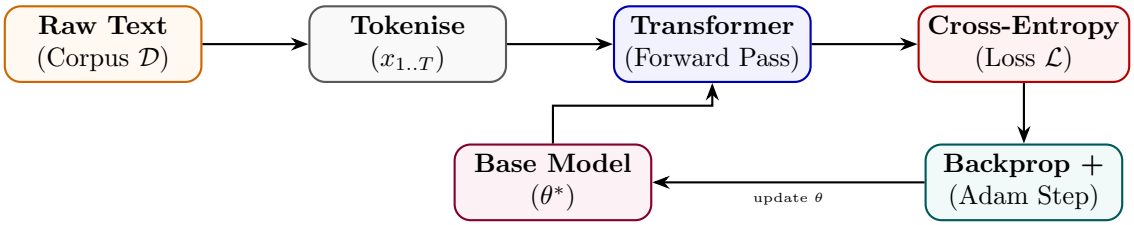


Figure 1: Pretraining pipeline: corpus sampling, forward pass, loss computation, and weight update loop.

Property	Before	After
Parameter source	Random init $\theta_0 \sim \mathcal{N}(0, \sigma^2)$	Converged weights θ^*
Training corpus	Not consumed	Trillions of tokens processed
Perplexity	Very high ($> 10^3$)	Low (< 20 on held-out data)
World knowledge	None	Broad language & factual knowledge
requires_grad	True (all params)	True (all params)

Table 1: State properties before and after pretraining.

Continued Pretraining (CPT)

Mathematical Foundation

Continued Pretraining (CPT) initializes parameters with a pretrained checkpoint θ_{base} and continues autoregressive training. To mitigate catastrophic forgetting, training batches are sampled from a mixture of the new domain-specific corpus $\mathcal{D}_{\text{new}}$ and a replay buffer of general pretraining data $\mathcal{D}_{\text{replay}}$:

$$x \sim \lambda \mathcal{D}_{\text{new}} + (1 - \lambda)\mathcal{D}_{\text{replay}} \tag{11}$$

where $\lambda \in [0, 1]$ is the mixing ratio. The CPT objective minimizes the cross-entropy loss over this mixed distribution:

$$\mathcal{L}_{\text{CPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{mix}}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{12}$$

The parameter updates follow the AdamW optimizer starting from $\theta_0 = \theta_{\text{base}}$, typically using a warm-restart cosine learning rate schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T_{\max}}\pi\right)\right) \tag{13}$$

where the peak learning rate $\eta_{\max}$ is set to a lower value than during the initial pretraining phase to preserve general capabilities.

Architecture Flow Diagram

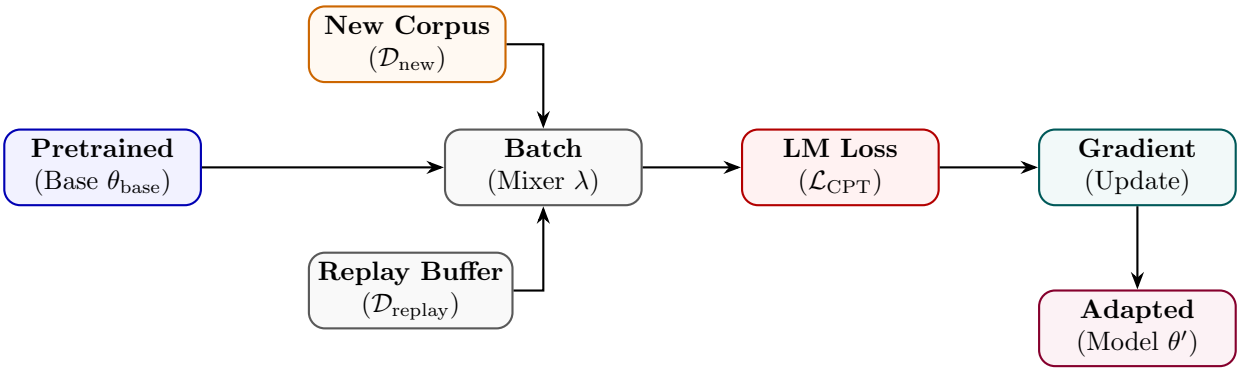


Figure 2: CPT mixes new corpus and replay buffer batches to update a pretrained checkpoint.

State Transition Table

Property	Before	After
Starting checkpoint	Pretrained θ_{base}	$\theta' = \theta_{\text{base}} + \Delta\theta$
New domain knowledge	Absent	Incorporated
Original knowledge	Retained	Partially retained via replay
Learning rate schedule	Finished	Warm-restart applied
Perplexity on new domain	High	Significantly reduced

Table 2: State properties before and after continued pretraining.

Domain Adaptive Pretraining (DAPT)

Mathematical Foundation

Domain Adaptive Pretraining (DAPT) adapts a general-domain pretrained model θ_{base} to a target domain (e.g., medical, legal, or code) by training on a large, unlabelled domain-specific corpus $\mathcal{D}_{\text{domain}}$. The objective function is the causal language modeling loss over the domain dataset:

$$\mathcal{L}_{\text{DAPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{domain}}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{14}$$

where parameters are initialized at $\theta_0 = \theta_{\text{base}}$. DAPT uses a lower learning rate than initial pretraining to prevent catastrophic disruption of base semantic features while allowing the model to adapt vocabulary representations and learn domain-specific concepts.

Architecture Flow Diagram

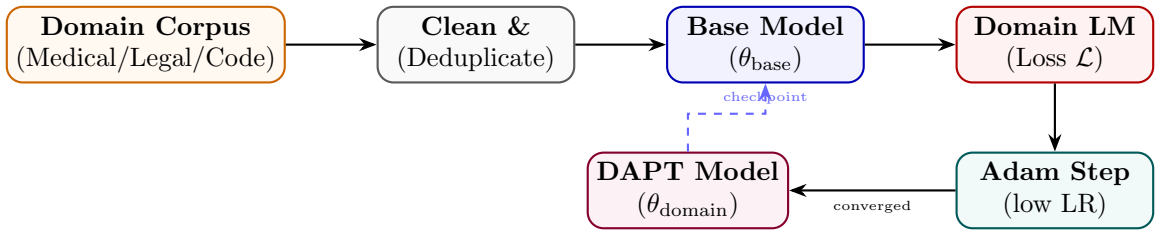


Figure 3: DAPT pipeline: domain corpus is cleaned, fed into the base model, and trained with standard LM loss.

State Transition Table

Property	Before	After
Domain vocabulary familiarity	Low	High
Domain perplexity	High ($\rho \gg 1$)	Low (near 1)
General language ability	Strong	Marginally reduced
Downstream task (same domain)	Weak	Significantly stronger
Training data type	General web text	Domain-specific unlabelled

Table 3: State properties before and after domain adaptive pretraining.

Task Adaptive Pretraining (TAPT)

Mathematical Foundation

Task Adaptive Pretraining (TAPT) optimizes a pretrained or domain-adapted checkpoint θ_{base} on a task-specific unlabelled dataset $\mathcal{D}_{\text{task}}$:

$$\mathcal{L}_{\text{TAPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{task}}} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{15}$$

Since the task dataset size $|\mathcal{D}_{\text{task}}|$ is typically much smaller than a general or domain-specific corpus, TAPT uses very few epochs, a small learning rate, and regularizers (such as dropout) to prevent overfitting before final supervised fine-tuning.

Architecture Flow Diagram

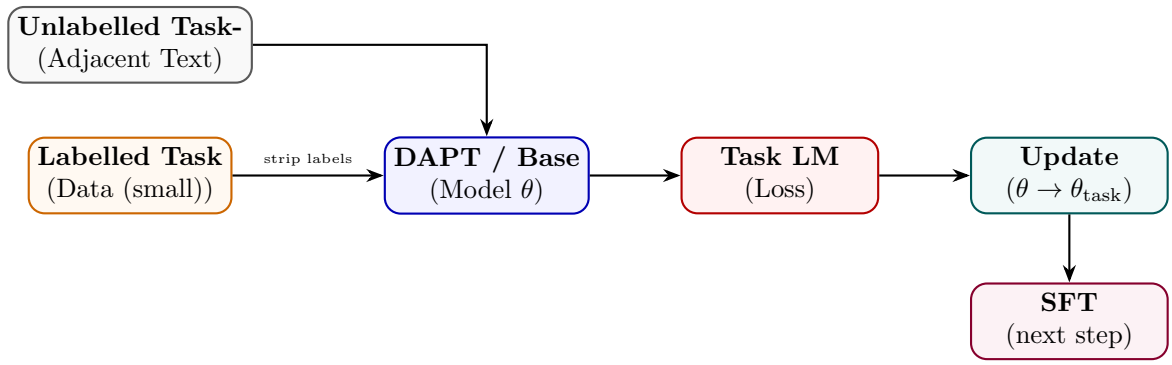


Figure 4: TAPT uses task-adjacent unlabelled text to bridge base model and supervised fine-tuning.

State Transition Table

Property	Before	After
Task vocabulary alignment	Partial	Strong
Corpus size	Millions of docs	Thousands of docs
Training epochs	1–3	10–100 (small corpus)
Downstream label efficiency	Requires more labels	Requires fewer labels
Transfer to other tasks	Good	Reduced (narrow focus)

Table 4: State properties before and after task adaptive pretraining.

Part II
Supervised Fine-Tuning Methods

Supervised Fine-Tuning (SFT)

Mathematical Foundation

SFT trains the model on structured prompt-response pairs $(x, y) \sim \mathcal{D}_{\text{SFT}}$ using teacher forcing. Let $s = [x; y]$ be the concatenated token sequence of length $M + U$, where $x = (s_1, \ldots, s_M)$ is the prompt and $y = (s_{M+1}, \ldots, s_{M+U})$ is the response. The key math lies in applying a target label mask $L \in \{-100, \mathcal{V}\}^{M+U}$ defined as:

$$L_t = \begin{cases} -100 & \text{if } t \leq M \\ s_t & \text{if } t > M \end{cases} \tag{16}$$

The cross-entropy loss function ignores targets set to -100, directing gradient updates exclusively to the response generation tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{U} \sum_{t=M+1}^{M+U} \log P_\theta(s_t \mid s_{<t}) \tag{17}$$

This loss ignores the prompt distribution $P(x)$, preventing the model from degrading its output generation capability to match arbitrary prompts. The gradients only backpropagate through response tokens:

$$\nabla_\theta \mathcal{L}_{\text{SFT}} = -\frac{1}{U} \sum_{t=M+1}^{M+U} \nabla_\theta \log P_\theta(s_t \mid s_{<t}) \tag{18}$$

Architecture Flow Diagram

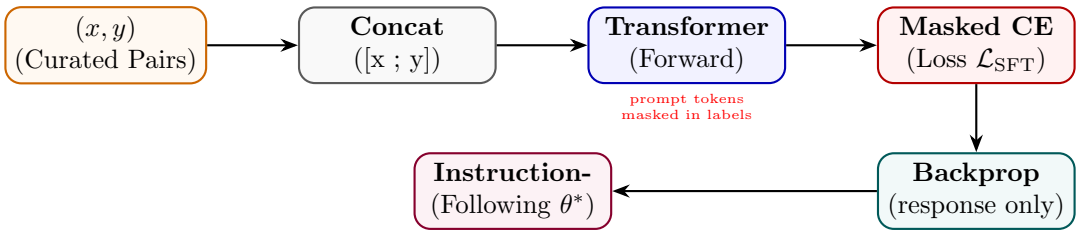


Figure 5: SFT pipeline with prompt-masked loss: gradients flow only through response tokens.

State Transition Table

Property	Before	After
Behaviour	Completion only	Instruction following
Loss region	N/A	Response tokens only
Label masking	None	Prompt masked to -100
Data format	Raw text	(instruction, response) pairs
Output quality	Random completion	Task-aligned responses

Table 5: State properties before and after supervised fine-tuning.

Instruction Tuning

Mathematical Foundation

Instruction Tuning exposes the model to multi-turn conversational patterns. A prompt template $T(I, x)$ converts an instruction I and context x into structured sequences. Turn boundaries are defined using chat-formatting systems (like ChatML). A single sequence layout is:

$$S = \langle \text{user} \rangle I \parallel x \langle \text{assistant} \rangle y \langle \text{eos} \rangle \tag{19}$$

Let $\mathcal{D}_{\text{inst}}$ consist of instructions spanning hundreds of tasks. The model parameters θ are optimized on the joint multi-task dataset:

$$\mathcal{L}_{\text{IT}}(\theta) = -\mathbb{E}_{(I,x,y)\sim\mathcal{D}_{\text{inst}}} \sum_{t=1}^{|y|} \log P_\theta\big(y_t \mid T(I, x), y_{<t}\big) \tag{20}$$

We mask all prompt-side tokens, including conversational format tags (like $\langle \text{user} \rangle$ and instruction text), ensuring parameters are trained only on synthesizing assistant responses:

$$\mathcal{L}_{\text{IT}}(\theta) = -\frac{1}{|y|} \sum_{i=1}^{|T(I,x)|+|y|} M_i \log P_\theta(S_i \mid S_{<i}), \quad M_i = \mathbb{I}(i > |T(I, x)|) \tag{21}$$

Architecture Flow Diagram

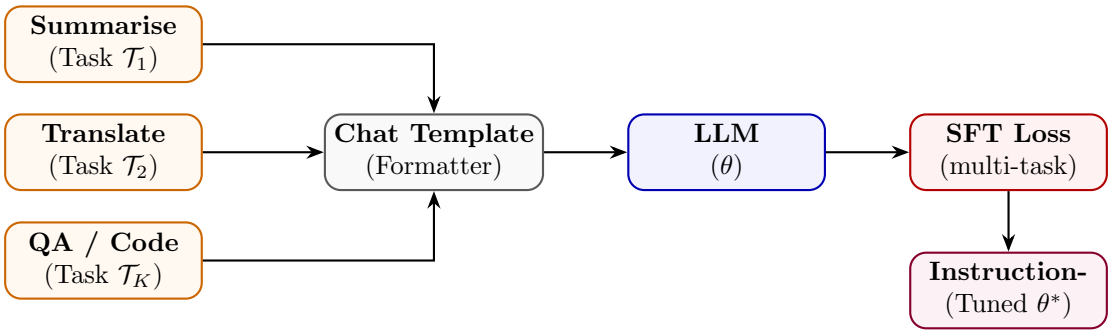


Figure 6: Instruction tuning over diverse task types funnelled through a unified chat template.

State Transition Table

Property	Before	After
Zero-shot generalisation	Poor	Strong across task types
Task diversity in training	N/A	K heterogeneous tasks
Input format	Free-form text	Structured chat template
Prompt sensitivity	High	Reduced
Held-out task performance	Weak	Strong (FLAN, InstructGPT)

Table 6: State properties before and after instruction tuning.

Multi-Task Training

Mathematical Foundation

Multi-Task Training aggregates losses from K distinct tasks. A simple summation can result in tasks with large gradient scales dominating the parameter updates. To resolve this, we use the GradNorm algorithm. Let W be the shared parameter weights (e.g., the last transformer block weights). We define the gradient norm for task k at training step t as:

$$G_k(t) = \|\nabla_W(w_k(t)\mathcal{L}_k(t))\|_2 \tag{22}$$

Let $\bar{G}(t) = \frac{1}{K}\sum_k G_k(t)$ be the average gradient norm. The target gradient norm for task k balances task difficulties:

$$\hat{G}_k(t) = \bar{G}(t) \times \left(\frac{\tilde{\mathcal{L}}_k(t)}{\frac{1}{K}\sum_j \tilde{\mathcal{L}}_j(t)}\right)^{\alpha} \tag{23}$$

where $\tilde{\mathcal{L}}_k(t) = \mathcal{L}_k(t)/\mathcal{L}_k(0)$ is the task loss ratio indicating task training progress, and α is a hyperparameter managing structural gradient balancing. The task weights $w_k(t)$ are updated by minimizing the L1 distance between $G_k(t)$ and $\hat{G}_k(t)$:

$$\mathcal{L}_{\text{grad}}(w_1, \ldots, w_K; t) = \sum_{k=1}^{K} \left| G_k(t) - \hat{G}_k(t) \right| \tag{24}$$

The final joint parameter objective is:

$$\mathcal{L}_{\text{MT}}(\theta) = \sum_{k=1}^{K} w_k \mathcal{L}_k(\theta) \tag{25}$$

Architecture Flow Diagram

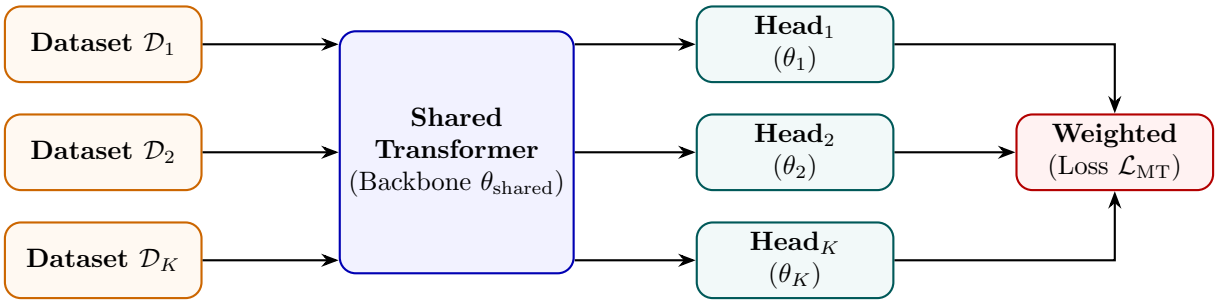


Figure 7: Multi-task training: each dataset feeds task-specific heads on a shared backbone.

State Transition Table

Property	Before	After
Backbone specialisation	Single task	K tasks jointly
Task heads	1	K heads
Generalisation	Narrow	Broad
Data efficiency	Per-task only	Cross-task transfer
Gradient conflicts	N/A	Managed via λ_k

Table 7: State properties before and after multi-task training.

Chain-of-Thought SFT

Mathematical Foundation

CoT SFT trains models to output intermediate reasoning steps before generating the final answer. Let x be the prompt, $c = (c_1, \ldots, c_M)$ be the reasoning chain tokens, and $y = (y_1, \ldots, y_U)$ be the final answer tokens. The joint probability factorization is:

$$P(c, y \mid x) = \prod_{i=1}^{M} P_\theta(c_i \mid x, c_{<i}) \prod_{j=1}^{U} P_\theta(y_j \mid x, c, y_{<j}) \tag{26}$$

The model is trained by minimizing the joint loss:

$$\mathcal{L}_{\text{CoT}}(\theta) = -\beta \sum_{i=1}^{M} \log P_\theta(c_i \mid x, c_{<i}) - (1-\beta) \sum_{j=1}^{U} \log P_\theta(y_j \mid x, c, y_{<j}) \tag{27}$$

During autoregressive generation, we sample tokens using Temperature-scaled softmax and Nucleus (top-p) filtering. The logits z_t are modified as:

$$\tilde{z}_{t,i} = \frac{z_{t,i}}{T} \tag{28}$$

where $T > 0$ is the temperature. The top-p vocabulary subset $\mathcal{V}^{(p)}$ at step t is the smallest set satisfying:

$$\sum_{v \in \mathcal{V}^{(p)}} P(v \mid x, s_{<t}) \geq p \tag{29}$$

Tokens outside $\mathcal{V}^{(p)}$ are assigned probability 0 before final selection.

Architecture Flow Diagram

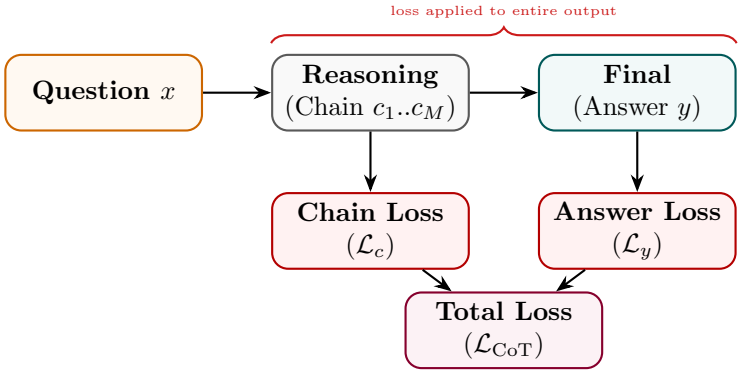


Figure 8: CoT SFT: loss is computed over both the reasoning chain and the final answer.

State Transition Table

Property	Before	After
Output format	Direct answer	Reasoning chain + answer
Training signal per example	$\lvert y \rvert$ tokens	$\lvert c \rvert + \lvert y \rvert$ tokens
Multi-step reasoning	Weak	Strong
Data annotation cost	Low	High (chain annotation)
Accuracy on complex tasks	Low	Substantially improved

Table 8: State properties before and after Chain-of-Thought SFT.

Step-by-Step SFT

Mathematical Foundation

Step-by-Step SFT formats the reasoning chain into discrete steps separated by a boundary token $T_{\text{step}} = \texttt{[STEP]}$. Let the response contain S steps: $s = (s_1, T_{\text{step}}, s_2, T_{\text{step}}, \ldots, s_S, T_{\text{step}})$. The step-level loss is:

$$\mathcal{L}_{\text{SBS}}(\theta) = -\sum_{k=1}^{S}\sum_{t=1}^{|s_k|} \log P_\theta(s_{k,t} \mid x, s_{<k}, s_{k,<t}) - \sum_{k=1}^{S} \log P_\theta(T_{\text{step}} \mid x, s_{\leq k}) \tag{30}$$

We define the average step perplexity PPL_k as:

$$\text{PPL}_k = \exp\left(-\frac{1}{|s_k|}\sum_{t=1}^{|s_k|} \log P_\theta(s_{k,t} \mid x, s_{<k}, s_{k,<t})\right) \tag{31}$$

Steps with high perplexity ($\text{PPL}_k > \tau$, where τ is a threshold) indicate low model confidence and can be flagged for step-level revision during generation. The model is trained to minimize the combined cross-entropy loss over all steps.

Architecture Flow Diagram

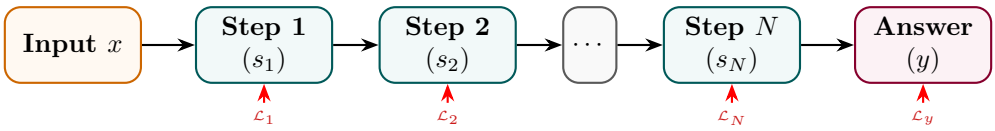


Figure 9: Step-by-Step SFT: each numbered step and the final answer contribute to the total loss.

State Transition Table

Property	Before	After
Solution structure	Unstructured	Numbered procedural steps
Step delimiter tokens	Not in vocabulary	Learned as first-class tokens
Interpretability	Low	Step-level traceability
Annotation format	Raw answer	$(x, [s_1, ..., s_N], y)$
Task decomposition ability	Implicit	Explicit

Table 9: State properties before and after Step-by-Step SFT.

Process Supervision

Mathematical Foundation

Process Supervision trains a Process Reward Model (PRM) using step-level binary correctness labels. Let x be the problem description, and $s_1, \ldots, s_S$ be the generated steps. Human or model-based annotations assign correctness labels $r_k \in \{0, 1\}$ for each step s_k. The PRM model outputs correctness probabilities by applying a logistic sigmoid function to the step boundary token representation:

$$\hat{r}_k = \sigma\left(\mathrm{PRM}_\phi(h_{t_k})\right) = \frac{1}{1 + \exp\left(-\mathrm{PRM}_\phi(h_{t_k})\right)} \tag{32}$$

where h_{t_k} is the transformer hidden output at index t_k representing the step boundary token. The PRM parameters ϕ are optimized using binary cross entropy:

$$\mathcal{L}_{\mathrm{PRM}}(\phi) = -\sum_{k=1}^{S} \left[r_k \log \hat{r}_k + (1 - r_k) \log(1 - \hat{r}_k)\right] \tag{33}$$

During inference, candidate sequences are generated. The total sequence correctness score is calculated as the product of step correctness probabilities:

$$\mathrm{Score}(s_{\leq S}) = \prod_{k=1}^{S} \hat{r}_k \tag{34}$$

This score is used to rank candidate rollouts in Best-of-N sampling or guide selection during Monte Carlo Tree Search (MCTS).

Architecture Flow Diagram

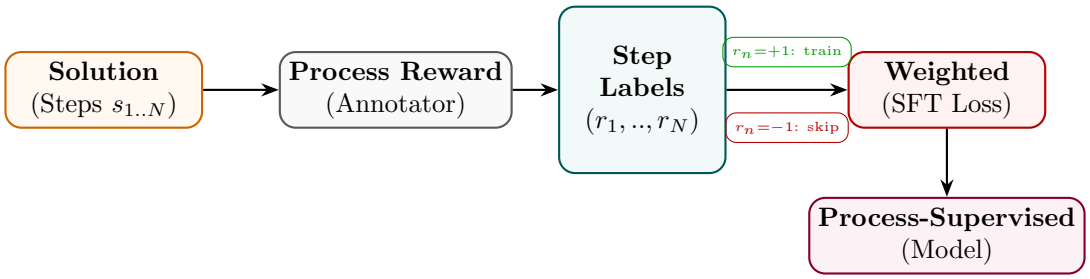


Figure 10: Process supervision: a step annotator labels each reasoning step; only correct steps contribute to the SFT loss.

State Transition Table

Property	Before	After
Supervision granularity	Final answer only	Per-step labels
Error signal	Outcome-level	Step-level
Error localisation	None	Identifies which step failed
Annotation cost	Low	High (step-level labelling)
Reasoning quality	Outcome-driven	Step-correctness driven

Table 10: State properties before and after process supervision.

Part III
Parameter-Efficient Fine-Tuning (PEFT)

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA represents the weight update matrix $\Delta W \in \mathbb{R}^{k\times d}$ for a linear projection layer as the product of two low-rank matrices $B \in \mathbb{R}^{k\times r}$ and $A \in \mathbb{R}^{r\times d}$, where the rank satisfies $r \ll \min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{35}$$

where α is a constant scaling parameter. The base weight matrix $W_0 \in \mathbb{R}^{k\times d}$ is frozen; only B and A are updated. Under a loss $\mathcal{L}$, the gradient propagation rules using the chain rule are derived as follows: Let $z = Ax \in \mathbb{R}^r$ be the intermediate bottleneck activation. Then $h = W_0 x + \frac{\alpha}{r} Bz$. The gradient with respect to output activation is $\nabla_h \mathcal{L} \in \mathbb{R}^{k\times 1}$ (represented as a column vector). The gradient with respect to the intermediate bottleneck representation z:

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \in \mathbb{R}^{r\times 1} \tag{36}$$

The gradient with respect to the matrix B:

$$\nabla_B \mathcal{L} = \frac{\alpha}{r} \nabla_h \mathcal{L} z^\top = \frac{\alpha}{r} \nabla_h \mathcal{L} x^\top A^\top \in \mathbb{R}^{k\times r} \tag{37}$$

The gradient with respect to the matrix A:

$$\nabla_A \mathcal{L} = \nabla_z \mathcal{L} x^\top = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} x^\top \in \mathbb{R}^{r\times d} \tag{38}$$

To eliminate latency at deployment, the weights are merged directly into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA \tag{39}$$

Architecture Flow Diagram

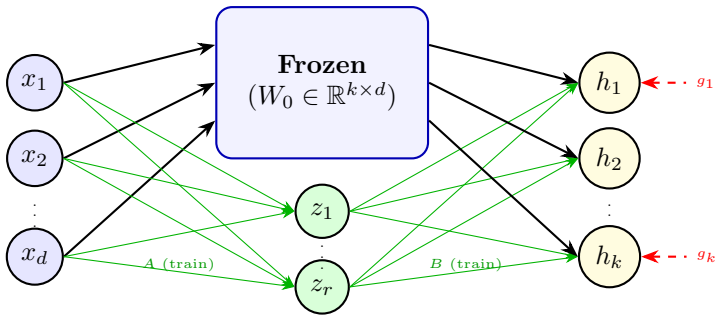


Figure 11: LoRA: input flows through frozen W_0 and trainable low-rank path $A \to B$; outputs are summed.

State Transition Table

Property	Before	After
Base weights W_0	requires_grad = True	Frozen (False)
Trainable params	$k \times d$	$r(k + d) \ll kd$
Matrix B	Does not exist	$\mathbb{R}^{k\times r}$, init $= 0$
Matrix A	Does not exist	$\mathbb{R}^{r\times d}$, init $\sim \mathcal{N}$
Inference overhead	None	Extra BA multiply (or merged)

Table 11: State properties before and after LoRA injection.

Memory Complexity Analysis

In this section, we analyze the memory footprint of the optimizer states. We compare full fine-tuning of a base weight matrix $W_0 \in \mathbb{R}^{k \times d}$ against low-rank training of A and B. Let $N_{\text{base}} = k \cdot d$ be the number of base parameters, and $N_{\text{LoRA}} = r(k + d)$ be the number of LoRA parameters.

AdamW tracks the following state elements for each trainable parameter:

1. **Parameter value copy** (master weights in mixed-precision): 4 bytes.
2. **First moment vector m (momentum)**: 4 bytes.
3. **Second moment vector v (variance)**: 4 bytes.

This results in $M_{\text{opt}} = 12$ bytes of optimizer state memory overhead per trainable parameter.
The memory saving ratio Φ is defined as:

$$\Phi = \frac{M_{\text{LoRA}}}{M_{\text{Full}}} = \frac{r(k+d)}{k \cdot d} = r\left(\frac{1}{k} + \frac{1}{d}\right) \tag{40}$$

Assuming a standard linear layer in a transformer block where $k = d = 4096$, and adapter rank $r = 8$:

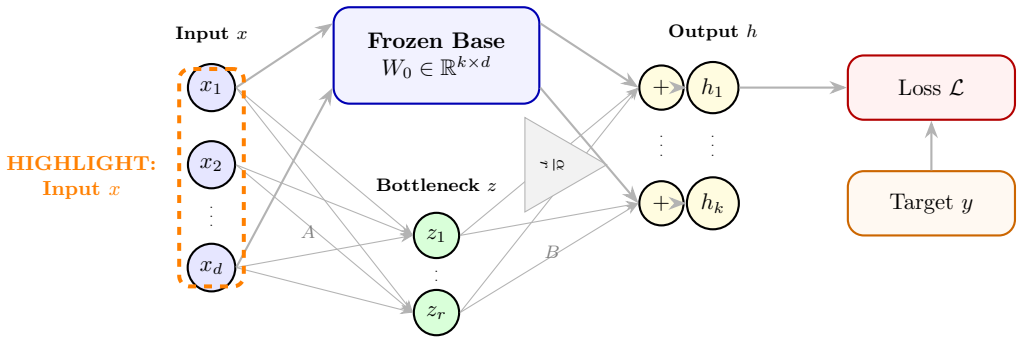
$$\Phi = 8\left(\frac{1}{4096} + \frac{1}{4096}\right) = \frac{16}{4096} = \frac{1}{256} \approx 0.39\% \tag{41}$$

This indicates a **256-fold reduction** (a decrease of 99.61%) in the memory required to store optimizer states for this weight matrix.

Detailed Operational Analysis of LoRA Variables

To understand the execution of LoRA at a granular level, we detail the mathematical nature of each variable, how its values are derived in reality, its child-level control dial mental model, and its physical mechanism at the individual neuron level. We provide a full neuron-level highlight diagram for each variable.

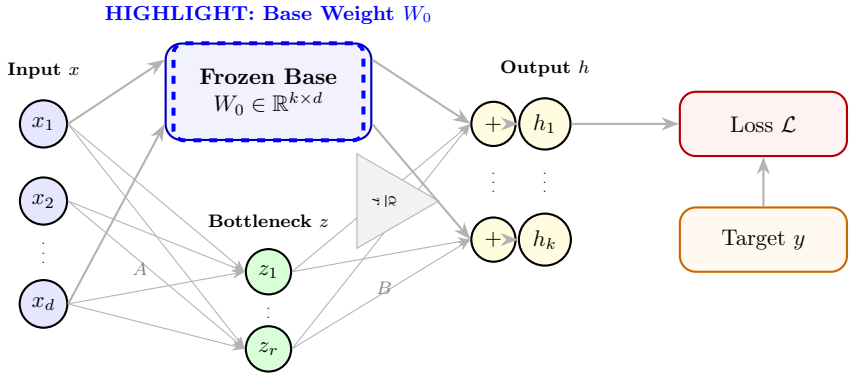
1. **Input Activation Vector (x)**:
   - **Formula**: x (output of previous layer)
   - **Neuron Highlight Diagram**:



   - **Mental Model**: Think of this vector as a list of positions for a row of thousands of physical dials. When the model processes a word, it adjusts these dials to specific numbers (e.g., dial 1 is at 4.2, dial 2 is at -1.5) to capture the meaning of the word. The model does not think in words; it thinks in these thousands of dial settings.
   - **Neuron-Level Physical Explanation**: A single neuron in the current layer receives x as a set of electrical signals from the neurons of the previous layer. Each element x_i represents the activation level (firing rate) of a previous neuron after its activation function (such as ReLU or GELU). The dendrites of the current neuron receive these signals.
   - **Mathematical Representation**: $x \in \mathbb{R}^{d \times 1}$
   - **Code Representation**: `x` (a PyTorch tensor of shape `(batch_size, seq_len, d)`)
   - **Relationships**: Fed into the base projection $W_0 x$ and compressed to $z = Ax$.

2. **Frozen Base Weight Matrix (W_0)**:

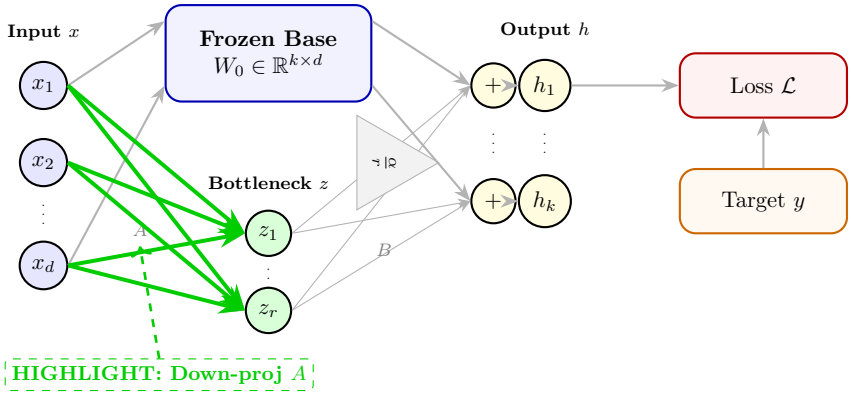
- **Formula**: W_0
- **Neuron Highlight Diagram**:



- **Mental Model**: This is a giant, permanent circuit board inside the control room. The wires are soldered in place and cannot be moved (frozen). They take the positions of the incoming dials (x), mix them together according to the permanent rules, and output a new set of general dial positions.
- **Neuron-Level Physical Explanation**: A single neuron's base connections are represented by a row of W_0. The neuron performs a dot product: it multiplies each incoming signal x_i by the strength of its corresponding synaptic connection $W_{0,ji}$ and sums them up: $y_j = \sum_i W_{0,ji} x_i$. During LoRA, these synaptic weights are frozen and cannot change, meaning their chemical/electrical conductance is locked.
- **Mathematical Representation**: $W_0 \in \mathbb{R}^{k \times d}$
- **Code Representation**: `W_0` (a PyTorch tensor with `W_0.requires_grad = False` to disable gradient computation)
- **Relationships**: Multiplied by the input to compute the baseline representation: $h_{\text{base}} = W_0 x$.

3. **Down-projection Adapter (A)**:

- **Formula**: A
- **Neuron Highlight Diagram**:

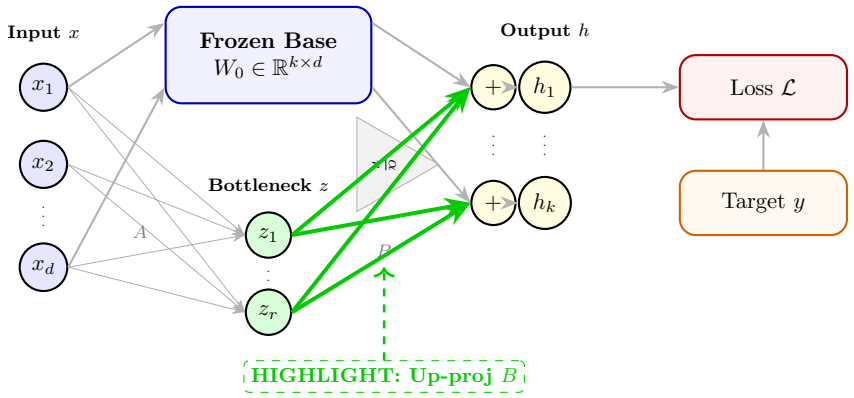


- **Mental Model**: This is a smaller circuit board that acts as a compression funnel. It looks at all the thousands of input dials and summarizes them into just a few dials (e.g., summarizing 4096 dials into just 8 dials). It only listens to the dial changes that are relevant to the new task we are teaching the model.
- **Neuron-Level Physical Explanation**: Inside the adapter path, a single neuron in the bottleneck layer corresponds to a row of matrix A. This neuron listens to all input signals x_i, multiplies them by its trainable weights A_{mi}, and sums them up: $z_m = \sum_i A_{mi} x_i$. This bottleneck neuron acts as a feature detector that extracts a specific combination of input signals.

- **Mathematical Representation**: $A \in \mathbb{R}^{r \times d}$
- **Code Representation**: `A` (initialized as `nn.Linear(d, r, bias=False)` with weight values drawn from a Gaussian distribution: `nn.init.kaiming_uniform_(A.weight, a=math.sqrt(5))`)
- **Relationships**: Compresses x into the bottleneck space via $z = Ax$.

4. **Up-projection Adapter (B)**:

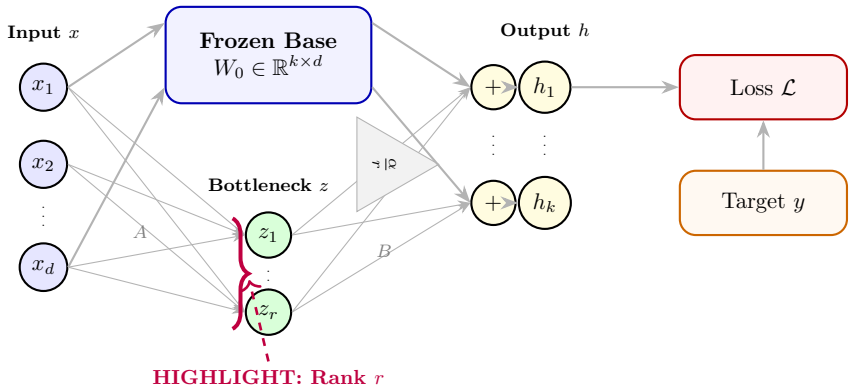
- **Formula**: B
- **Neuron Highlight Diagram**:



- **Mental Model**: This is another small circuit board that acts as an expansion nozzle. It takes the tiny summary from the funnel (our 8 dials) and expands it back into thousands of dials, showing how to nudge the final output dials to fit the new task.
- **Neuron-Level Physical Explanation**: A single output neuron receives contributions from the bottleneck neurons. The connection strength from bottleneck neuron m to output neuron j is the trainable weight B_{jm}. The output neuron sums the weighted signals from all bottleneck neurons: $u_j = \sum_m B_{jm} z_m$, scaling up the low-dimensional task features.
- **Mathematical Representation**: $B \in \mathbb{R}^{k \times r}$
- **Code Representation**: `B` (initialized as `nn.Linear(r, k, bias=False)` with weight values set to all zeros: `nn.init.zeros_(B.weight)`)
- **Relationships**: Expands the bottleneck activations via Bz, scaled by $\frac{\alpha}{r}$.

5. **Bottleneck Rank (r)**:

- **Formula**: r
- **Neuron Highlight Diagram**:

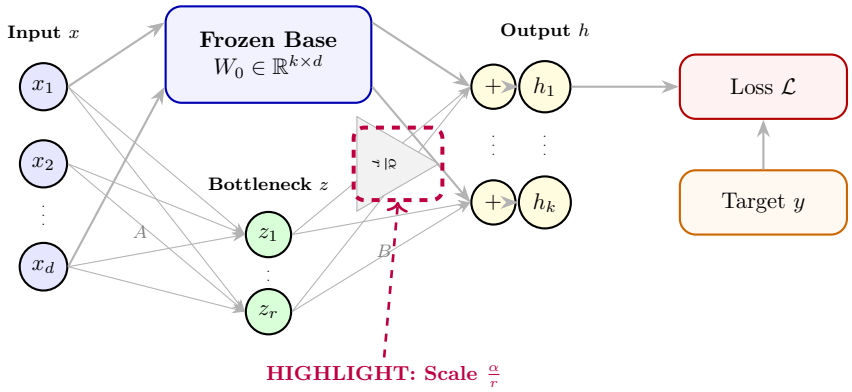


- **Mental Model**: This is the exact number of summary dials inside the bottleneck. If $r = 8$, it means we are forcing the model to summarize all its knowledge into just 8 dials. A smaller number makes the funnel tighter, forcing the model to only keep the most important summaries.

- **Neuron-Level Physical Explanation**: This is the exact number of neurons present in the bottleneck layer of the adapter. Selecting $r = 8$ means there are exactly 8 neurons in this intermediate layer. Each of these 8 neurons acts as a specialized task detector.
- **Mathematical Representation**: $r \in \mathbb{Z}^+$
- **Code Representation**: `r` (passed as an integer configuration parameter, e.g., `lora_config = LoraConfig(r=8)`)
- **Relationships**: Determines the inner dimension of matrices A and B, and acts as a scaling denominator in $\frac{\alpha}{r}$.

6. **Scaling Constant (α)**:

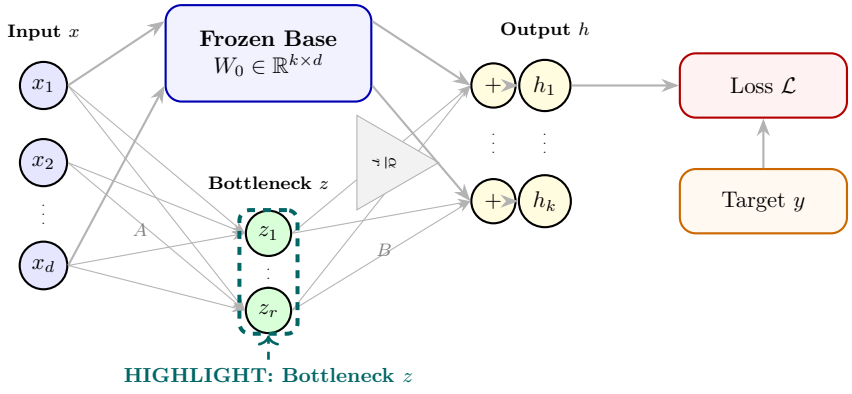
- **Formula**: α
- **Neuron Highlight Diagram**:



- **Mental Model**: This is a volume control knob. It decides how loudly the changes from the new adapters (A and B) should shout compared to the original, permanent circuit board (W_0). If we turn this knob up, the model listens more to its new training and less to its old pretrained knowledge.
- **Neuron-Level Physical Explanation**: At the neuron level, this acts as a synaptic gain control. The signal coming from the bottleneck via the adapter pathway is amplified or attenuated by a constant scale factor of $\frac{\alpha}{r}$ before it is added to the base neuron's output. This behaves like a biological neuromodulator that globally scales the synaptic sensitivity of the adapter path.
- **Mathematical Representation**: $\alpha \in \mathbb{R}^+$
- **Code Representation**: `lora_alpha` (passed as a configuration parameter, e.g., `lora_config = LoraConfig(lora_a`
- **Relationships**: Multiplies the adapter pathway output, yielding $\frac{\alpha}{r} Bz$.

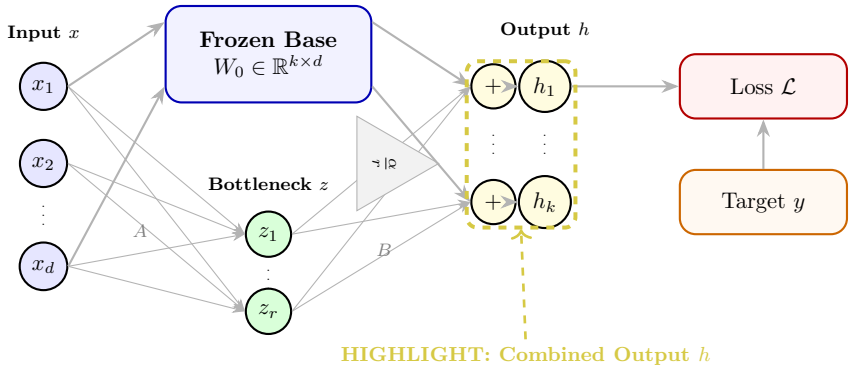
7. **Bottleneck Activation Vector (z)**:

- **Formula**: $z = Ax$
- **Neuron Highlight Diagram**:



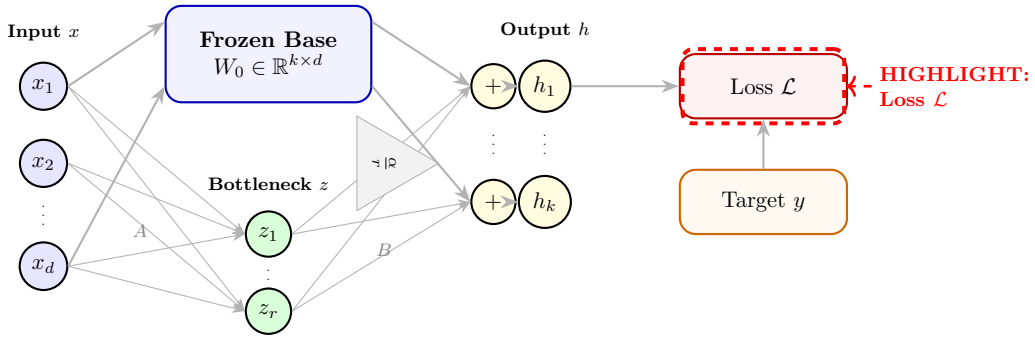
- **Mental Model**: These are the actual values of our few summary dials inside the bottleneck. It represents the compressed task-specific message as it flows from the compression funnel to the expansion nozzle.
- **Neuron-Level Physical Explanation**: Each element z_m is the electrical activation level (firing rate) of a single neuron in the bottleneck layer. It is calculated by that neuron summing its inputs: $z_m = \sum_i A_{mi} x_i$.
- **Mathematical Representation**: $z \in \mathbb{R}^{r \times 1}$
- **Code Representation**: `z = F.linear(x, A.weight)` (or `z = x @ A.weight.t()`)
- **Relationships**: Created by A, consumed by B.

8. **Combined Output Vector (h)**:
   - **Formula**: $h = W_0 x + \frac{\alpha}{r} B z$
   - **Neuron Highlight Diagram**:



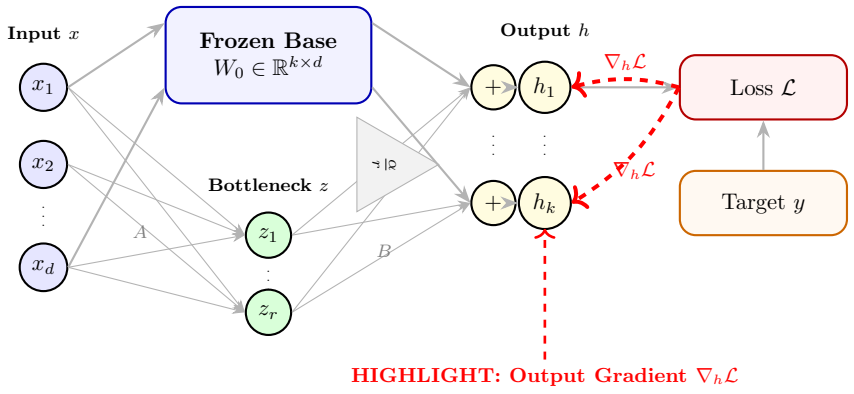
   - **Mental Model**: This is the final row of output dials. It is created by taking the general dial settings computed by the permanent circuit board (W_0) and nudging them slightly using the task-specific dial settings computed by our adapters.
   - **Neuron-Level Physical Explanation**: A single output neuron j sums the signals from the frozen base connection path and the scaled trainable adapter path: $h_j = \sum_i W_{0,ji} x_i + \frac{\alpha}{r} \sum_m B_{jm} z_m$. The neuron acts as a physical summing junction, integrating the general-purpose signal and the specialized task signal.
   - **Mathematical Representation**: $h \in \mathbb{R}^{k \times 1}$
   - **Code Representation**: `h = base_layer(x) + (alpha / r) * B(z)`
   - **Relationships**: Fed as input to subsequent layers of the network.

9. **Scalar Loss Value ($\mathcal{L}$)**:
   - **Formula**: $\mathcal{L}$ (e.g. Cross-Entropy loss)
   - **Neuron Highlight Diagram**:



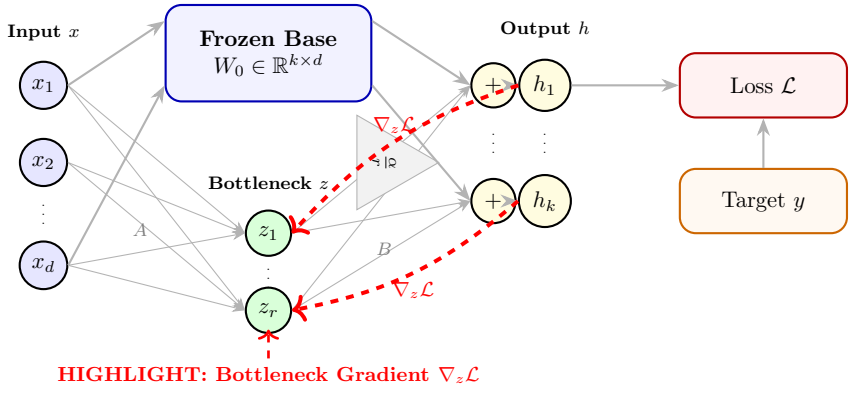
- **Mental Model**: This is a single scorecard number. It tells us how far off the final output dials are from what they should be. If the score is high, it means the model is making bad guesses. The goal of training is to adjust the new dials until the scorecard reads zero.
- **Neuron-Level Physical Explanation**: The loss $\mathcal{L}$ measures how far the firing rates of the output neurons in the final layer differ from the desired target firing rates. The target represents the correct next word, and the error signal is generated by comparing the target with the actual outputs.
- **Mathematical Representation**: $\mathcal{L} \in \mathbb{R}$
- **Code Representation**: `loss = F.cross_entropy(logits, targets)`
- **Relationships**: Traversed in reverse during backpropagation; all gradients are derivatives of $\mathcal{L}$.

10. **Output Layer Gradient ($\nabla_h \mathcal{L}$)**:
    - **Formula**: $\nabla_h \mathcal{L} = \frac{\partial \mathcal{L}}{\partial h}$
    - **Neuron Highlight Diagram**:



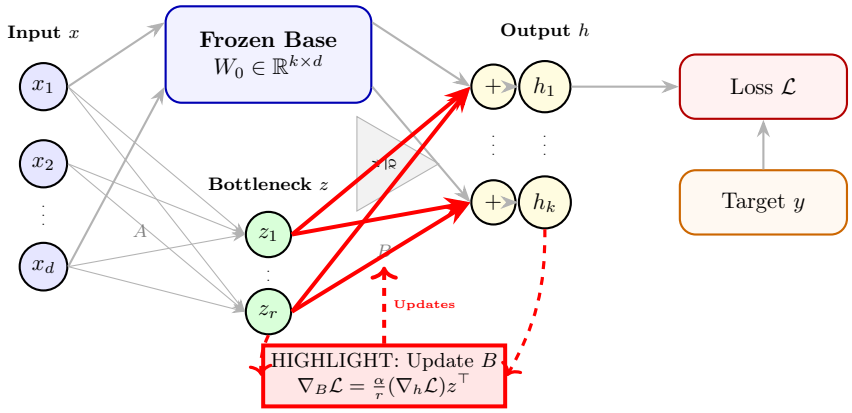
- **Mental Model**: This is a set of correction instructions for the final output dials. It says: "dial 1 needs to turn clockwise by 0.2, and dial 2 needs to turn counter-clockwise by 1.5" to lower the error score.
- **Neuron-Level Physical Explanation**: For a single output neuron j, the element $(\nabla_h \mathcal{L})_j = \frac{\partial \mathcal{L}}{\partial h_j}$ represents the error feedback signal. It tells the neuron whether it fired too strongly (positive gradient) or too weakly (negative gradient) and by how much it needs to adjust its total output to minimize the error.
- **Mathematical Representation**: $\nabla_h \mathcal{L} \in \mathbb{R}^{k \times 1}$
- **Code Representation**: automatically tracked by PyTorch's `loss.backward()`, represented as the grad tensor associated with output `h.grad`
- **Relationships**: Directly drives the gradient computation for B and z.

11. **Bottleneck Layer Gradient ($\nabla_z \mathcal{L}$)**:
    - **Formula**: $\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L}$
    - **Neuron Highlight Diagram**:



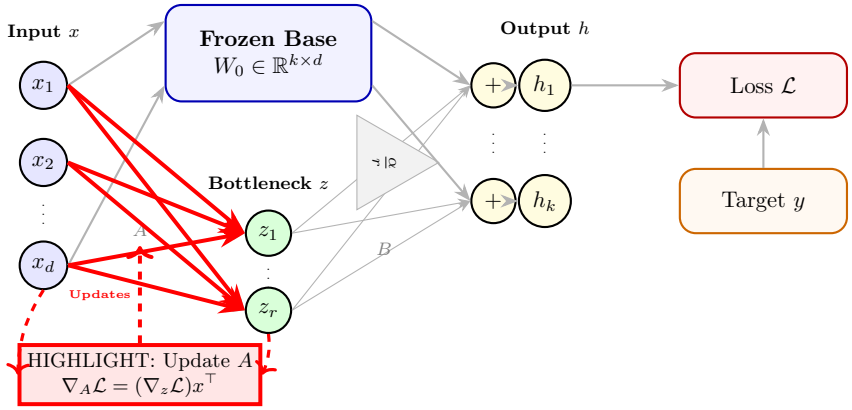
- **Mental Model**: This translates the correction instructions from the output room back down into the bottleneck. It tells the summary dials inside the funnel how they need to adjust to fix the output dials.
- **Neuron-Level Physical Explanation**: For a bottleneck neuron m, this is the backpropagated error signal. It is computed by summing the error feedback signals from all output neurons, weighted by their connections B_{jm} and scaled: $(\nabla_z \mathcal{L})_m = \frac{\alpha}{r} \sum_j B_{jm} (\nabla_h \mathcal{L})_j$. It tells the bottleneck neuron how its activation should have changed.
- **Mathematical Representation**: $\nabla_z \mathcal{L} \in \mathbb{R}^{r \times 1}$
- **Code Representation**: `grad_z = (alpha / r) * torch.matmul(grad_output, B.weight)`
- **Relationships**: Drives the gradient computation for A.

12. **Update Gradient for Matrix B ($\nabla_B \mathcal{L}$)**:
    - **Formula**: $\nabla_B \mathcal{L} = \frac{\alpha}{r} (\nabla_h \mathcal{L}) z^\top$
    - **Neuron Highlight Diagram**:



- **Mental Model**: This is the instruction sheet showing how to rewire the dials of the expansion nozzle (B) so that they do a better job of translating the summary back to the output dials.
- **Neuron-Level Physical Explanation**: For the synapse connecting bottleneck neuron m to output neuron j, the gradient is calculated as $(\nabla_B \mathcal{L})_{jm} = \frac{\alpha}{r} (\nabla_h \mathcal{L})_j z_m$. This follows Hebbian learning: the strength of the update is proportional to the product of the pre-synaptic activation (z_m) and the post-synaptic error signal ($(\nabla_h \mathcal{L})_j$).
- **Mathematical Representation**: $\nabla_B \mathcal{L} \in \mathbb{R}^{k \times r}$
- **Code Representation**: `B.weight.grad = (alpha / r) * torch.matmul(grad_output.t(), z)`
- **Relationships**: Used by the optimizer to update the values of B.

13. **Update Gradient for Matrix A ($\nabla_A \mathcal{L}$)**:
    - **Formula**: $\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top$
    - **Neuron Highlight Diagram**:



- **Mental Model**: This is the instruction sheet showing how to rewire the dials of the compression funnel (A) so that they do a better job of compressing the input dials into the summary.
- **Neuron-Level Physical Explanation**: For the synapse connecting input neuron i to bottleneck neuron m, the gradient is $(\nabla_A \mathcal{L})_{mi} = (\nabla_z \mathcal{L})_m x_i$. This also follows Hebbian learning: it multiplies the pre-synaptic input signal (x_i) by the post-synaptic bottleneck error signal ($(\nabla_z \mathcal{L})_m$) to determine how much the synapse should strengthen or weaken.
- **Mathematical Representation**: $\nabla_A \mathcal{L} \in \mathbb{R}^{r \times d}$
- **Code Representation**: `A.weight.grad = torch.matmul(grad_z.t(), x)`
- **Relationships**: Used by the optimizer to update the values of A.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of d = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

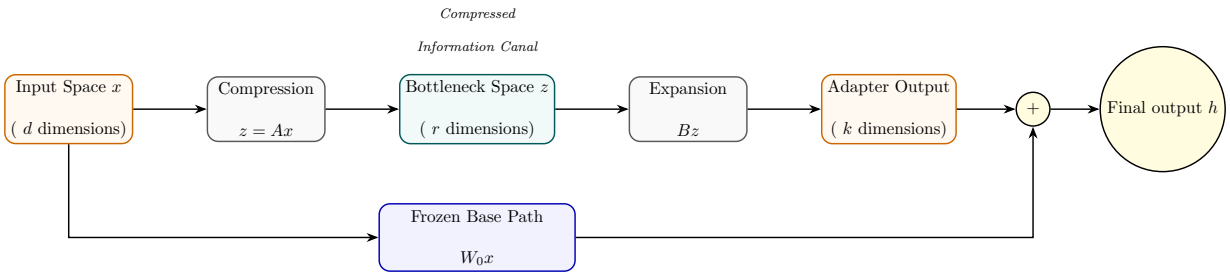


Figure 12: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of d = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k\times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k\times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

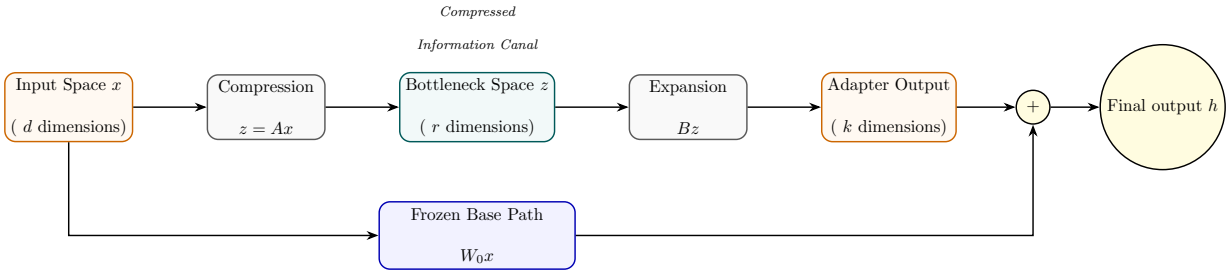


Figure 13: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of d = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to

underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($mathbbR^{rtimes(d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

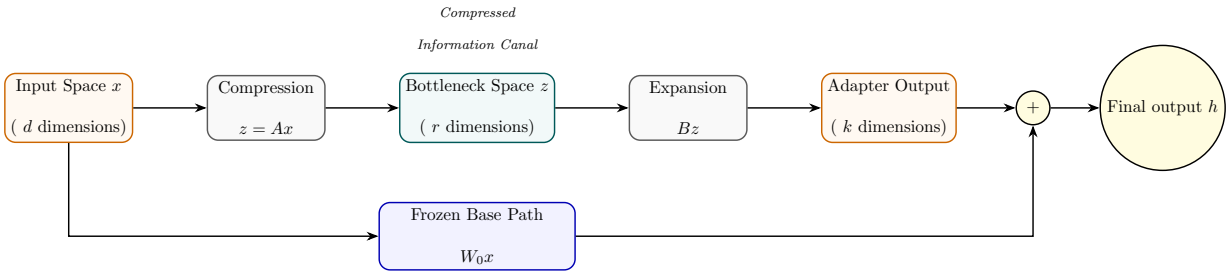


Figure 14: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of d = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k\times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($mathbbR^{rtimes(d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k\times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

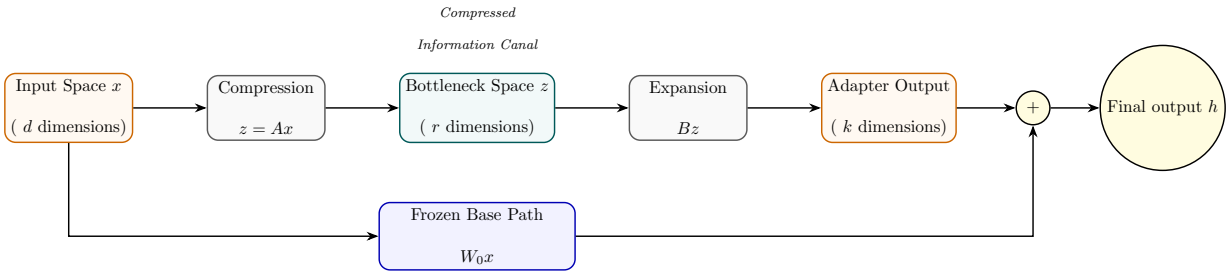


Figure 15: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of d = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

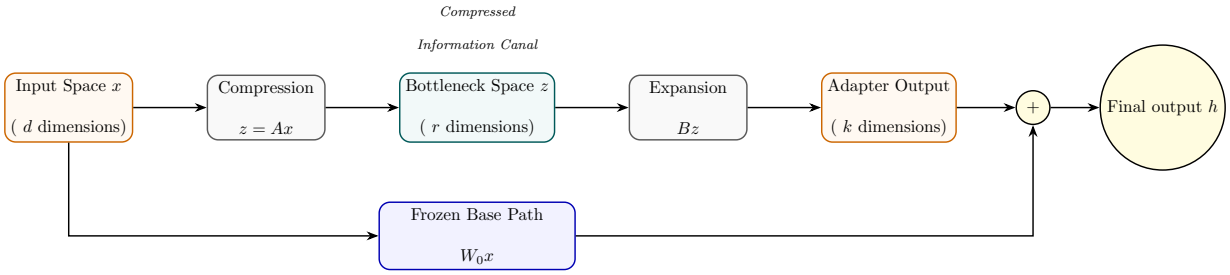


Figure 16: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill

this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k\times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k\times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

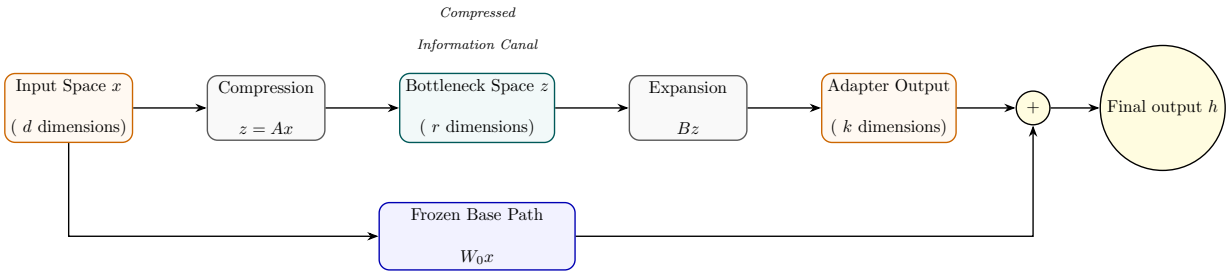


Figure 17: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to

capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k\times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k\times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{d\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

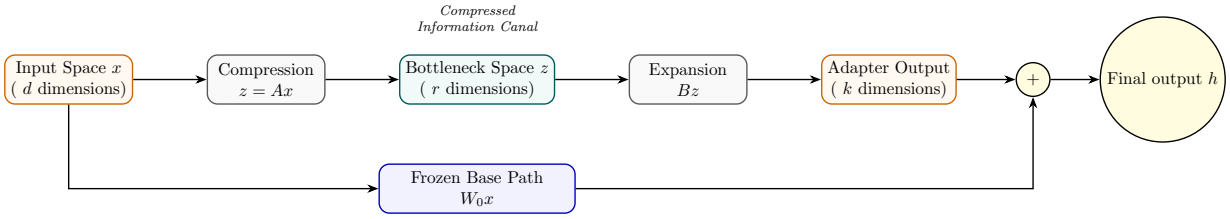


Figure 18: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

LoRA CLRS-Style Algorithms

This section contains the CLRS-style mathematical algorithms for the forward pass, manual backward pass, parameter merging, and AdamW optimizer update step.

LoRA-Forward$(x, W_0, A, B, \alpha, r)$
  $h_{\text{base}} = W_0 x$
  $z = Ax$
  $z_{\text{out}} = Bz$
  $h = h_{\text{base}} + \frac{\alpha}{r} z_{\text{out}}$
  return h

Figure 19: LoRA-Forward algorithm: computes base output and scaled low-rank adapter output.

Analysis of Lora-Forward

- **What is achieved**: Computes the layer output h by parallelizing the computation of the frozen base weight path (W_0x) and the trainable low-rank adapter path ($B(Ax)$), and scaling the adapter's contribution.
- **What internally changed**: Instantiates intermediate activation tensors $z = Ax \in \mathbb{R}^{r\times 1}$ and $z_{\text{out}} = Bz \in \mathbb{R}^{k\times 1}$ in memory. During training, these tensors are kept in GPU memory (VRAM) because their values are required to compute the gradients in the backward pass.
- **User-modifiable parameters & impact**:
  - **Rank** r (hyperparameter): Modifies the bottleneck dimension. A larger r (e.g., 32) captures more task-specific information but increases memory requirements and latency. A smaller r (e.g., 8) is highly memory-efficient but might underfit complex tasks.
  - **Scaling constant** α (hyperparameter): Adjusts the strength of the adapter's adjustments. Increasing α scales up the adapter's updates. Keep this constant when tuning r to stabilize learning.

LoRA-Backward$(x, A, B, \nabla_h \mathcal{L}, \alpha, r)$
  $z = Ax$
  $\nabla_B \mathcal{L} = \frac{\alpha}{r} (\nabla_h \mathcal{L}) z^\top$
  $\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L}$
  $\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top$
  $\nabla_{x,\text{adapters}} \mathcal{L} = A^\top \nabla_z \mathcal{L}$
  return $\nabla_A \mathcal{L}, \nabla_B \mathcal{L}, \nabla_{x,\text{adapters}} \mathcal{L}$

Figure 20: LoRA-Backward algorithm: derives gradients for adapters and backpropagated input gradient.

Analysis of Lora-Backward

- **What is achieved**: Calculates the exact gradients $\nabla_A \mathcal{L}$ and $\nabla_B \mathcal{L}$ for the adapter weights, and computes the backpropagated input gradient $\nabla_{x,\text{adapters}} \mathcal{L}$ to pass back to previous layers.
- **What internally changed**: Populates the gradient buffers `A.grad` and `B.grad` in VRAM. The frozen weights W_0 do not have gradient buffers, saving $d \times k \times 2$ or 4 bytes of memory.
- **User-modifiable parameters & impact**:
  - **Rank** r **and Scaling** α: Directly scale the gradients via the factor $\frac{\alpha}{r}$. Modifying them scales the gradient step size, affecting convergence stability.

LORA-MERGE($W_0, A, B, \alpha, r, mode$)
  $\Delta W = \frac{\alpha}{r} BA$
  if $mode ==$ "merge"
    $W = W_0 + \Delta W$
  else if $mode ==$ "demerge"
    $W = W_0 - \Delta W$
  return W

Figure 21: LORA-MERGE algorithm: handles parameter folding and unfolding for zero-latency inference.

Analysis of Lora-Merge

- **What is achieved**: Merges the low-rank adapters directly into the base weights ($W = W_0 \pm \Delta W$) for zero-latency inference, or subtracts them to restore the base weights.
- **What internally changed**: In-place updates are applied to the base weight tensor W_0 in memory. Once merged, the adapter weights A and B can be discarded, and the model behaves as a single unified feedforward network, eliminating the need to compute separate low-rank paths during inference.
- **User-modifiable parameters & impact**:
  - **Mode**: User can choose `"merge"` to prepare the model for high-throughput deployment, or `"demerge"` to unpack the adapters and swap in a different set of adapter weights.

LORA-ADAMW-UPDATE($\theta, g, m, v, \eta, \lambda, \beta_1, \beta_2, \epsilon, t$)
  for each parameter matrix $P \in \theta$ **do**
    $P = P - \eta\lambda P$
    $m_P = \beta_1 m_P + (1 - \beta_1) g_P$
    $v_P = \beta_2 v_P + (1 - \beta_2) g_P^2$
    $\hat{m}_P = m_P / (1 - \beta_1^t)$
    $\hat{v}_P = v_P / (1 - \beta_2^t)$
    $P = P - \eta \frac{\hat{m}_P}{\sqrt{\hat{v}_P} + \epsilon}$
  return θ, m, v

Figure 22: LORA-ADAMW-UPDATE algorithm: updates parameters with decoupled weight decay and momentum/-variance tracking.

Analysis of Lora-AdamW-Update

- **What is achieved**: Applies decoupled weight decay to the adapter parameters and updates their values using momentum and variance of gradients.
- **What internally changed**: Trainable parameters A and B are updated. Additionally, the first moment vectors m_A, m_B and second moment vectors v_A, v_B are updated in GPU memory. Because W_0 is frozen, no optimizer states are created or updated for W_0, resulting in huge VRAM savings.
- **User-modifiable parameters & impact**:
  - **Learning Rate** η: Controls the step size. High values speed up training but risk divergence; low values ensure stability but slow down convergence.
  - **Weight Decay** λ: Regularizes the weights to prevent overfitting.
  - **Momentum coefficients** β_1, β_2: Control the memory of past gradients.

Quantized LoRA (QLoRA)

Mathematical Foundation

QLoRA quantizes the base weight W_0 to 4-bit NormalFloat (NF4) and uses Double Quantization (DQ) to minimize memory requirements. NF4 divides the standard normal distribution $\mathcal{N}(0,1)$ into $2^k = 16$ equal-area intervals. The quantization levels q_i are calculated using the standard normal quantile function Q_X:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k}\right) + Q_X\left(\frac{i+1}{2^k}\right)\right) \tag{42}$$

The base weights are normalized to $[-1,1]$ using block-wise scaling. Let the block size be $B = 64$. For a block W_b, the scale is $c_b = \max(|W_b|)$. The quantized values are:

$$q_t = \arg\min_{i\in\{0,\ldots,15\}} \left|\frac{W_{b,t}}{c_b} - q_i\right| \tag{43}$$

Double Quantization (DQ) quantizes the scaling factors c_b themselves to 8-bit floats (c_b^{FP8}) with a second scale block size of 256, reducing scale memory overhead from $32/64 = 0.5$ bits/weight to $8/64 + 32/(64 \times 256) \approx 0.127$ bits/weight. During backpropagation, the NF4 weights are dequantized to FP16/BF16 to compute outputs, and gradients are computed only for the low-rank parameters:

$$h = \text{dequant}(c_b, W^{\text{NF4}})x + \frac{\alpha}{r}BAx \tag{44}$$

Architecture Flow Diagram

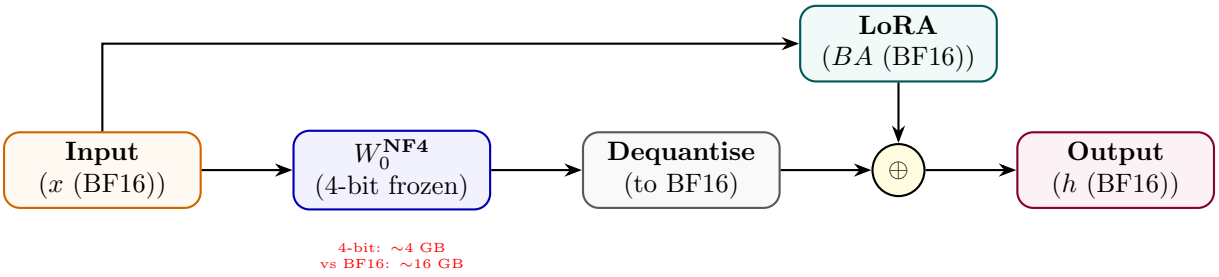


Figure 23: QLoRA: 4-bit frozen base dequantised at runtime; LoRA adapters trained in BF16.

State Transition Table

Property	Before	After
W_0 precision	BF16 / FP16	NF4 (4-bit)
GPU memory for 70B model	~140 GB	~35 GB
LoRA adapter precision	N/A	BF16
Training fidelity	Full	Near-full (straight-through)
Inference latency	Baseline	+dequant overhead

Table 12: State properties before and after QLoRA.

Adaptive LoRA (AdaLoRA)

Mathematical Foundation

AdaLoRA parameterizes incremental weight updates using a singular value decomposition (SVD) representation to dynamically adjust the adapter rank during training:

$$\Delta W = P\Lambda Q \tag{45}$$

where $P \in \mathbb{R}^{d\times r}$ and $Q \in \mathbb{R}^{r\times k}$ are left and right singular vectors, and $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_r) \in \mathbb{R}^{r\times r}$ contains singular values. To prevent SVD divergence and maintain parameter stability, we enforce orthogonality constraints:

$$\mathcal{L}_{\text{orth}}(P, Q) = \gamma \left(\|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2 \right) \tag{46}$$

We estimate the training importance of each singular triplet $(P_{*,i}, \lambda_i, Q_{i,*})$ using gradient-weight product magnitude:

$$S_i = s(\lambda_i) + \frac{1}{d}\sum_{j=1}^{d} s(P_{j,i}) + \frac{1}{k}\sum_{j=1}^{k} s(Q_{i,j}) \tag{47}$$

where $s(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$. At designated intervals, triplets with the lowest importance scores S_i are pruned by zeroing out their singular values λ_i, dynamically reducing the adapter rank budget.

Architecture Flow Diagram

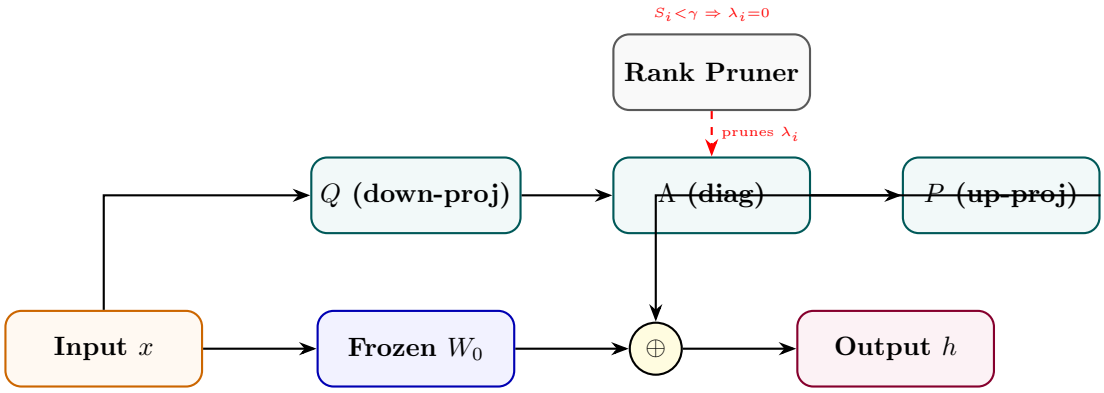


Figure 24: AdaLoRA: SVD-decomposed update with importance-based rank pruning per layer.

State Transition Table

Property	Before	After
Rank per layer	Fixed r	Adaptive (varies by importance)
Update structure	BA product	$P\Lambda Q$ SVD triplet
Parameter budget	Fixed	Redistributed across layers
Low-importance ranks	Included	Pruned ($\lambda_i = 0$)
Training stability	Standard	Constrained by SVD orthogonality

Table 13: State properties before and after AdaLoRA.

Weight-Decomposed LoRA (DoRA)

Mathematical Foundation

DoRA decomposes the weight matrix $W \in \mathbb{R}^{k \times d}$ into its magnitude vector $m \in \mathbb{R}^k$ and direction matrix $V \in \mathbb{R}^{k \times d}$:

$$W = m \cdot \frac{V}{\|V\|_r} = m \cdot \frac{W_0 + \Delta W}{\|W_0 + \Delta W\|_r} \tag{48}$$

where $\| \cdot \|_r$ is the L2 norm computed along each row (corresponding to output channels). In parameter-efficient training, we freeze the direction matrix to the base weight $V = W_0$ and introduce low-rank directional updates using adapters $\Delta V = \frac{\alpha}{r} BA$ (where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$):

$$W = m \cdot \frac{W_0 + \frac{\alpha}{r} BA}{\|W_0 + \frac{\alpha}{r} BA\|_r} \tag{49}$$

Only the magnitude vector m and low-rank directional matrices B and A are trainable. The gradient flow for the directional components is:

$$\nabla_V \mathcal{L} = \frac{m}{\|V\|_c} \left(\nabla_W \mathcal{L} - \frac{V}{\|V\|_c^2} \odot \left(V^\top \nabla_W \mathcal{L} \right) \right) \tag{50}$$

This mathematically separates the updates to magnitude and direction, mimicking full parameter fine-tuning behavior while retaining low parameter overhead.

Architecture Flow Diagram

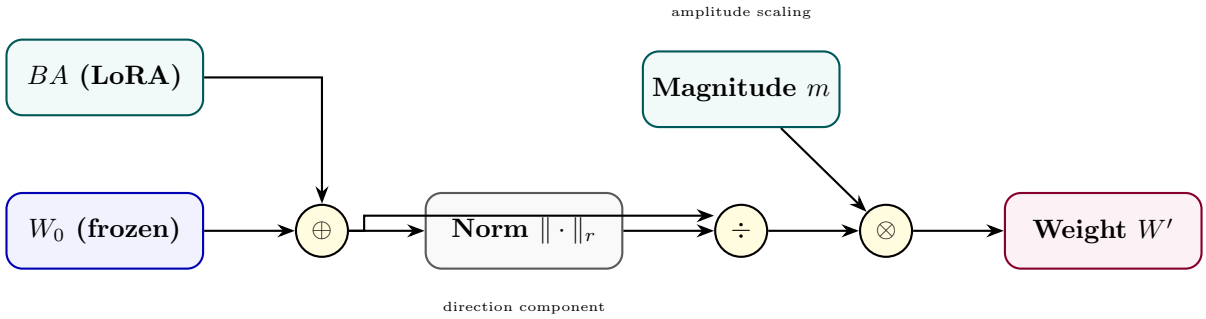


Figure 25: DoRA: weight decomposed into direction (LoRA-adapted) and magnitude (trainable scalar).

State Transition Table

Property	Before (LoRA)	After (DoRA)
Magnitude component	Implicit in W_0	Explicit trainable m
Direction component	$W_0 + BA$ unnorm	Column-normalised
Full FT similarity	Lower	Higher
Trainable params	$r(d+k)$	$r(d+k)+k$
Catastrophic forgetting	Low	Lower

Table 14: State properties comparing LoRA and DoRA.

IA^3 (Infused Adapter by Inhibiting and Amplifying Inner Activations)

Mathematical Foundation

IA^3 introduces element-wise scaling vectors to directly modify inner activations in attention keys, values, and intermediate feed-forward layers. Let the scaling vectors be $l_K \in \mathbb{R}^d$, $l_V \in \mathbb{R}^d$, and $l_{FFN} \in \mathbb{R}^{d_{FFN}}$. The attention layer activations are computed as:

$$\tilde{K} = K \odot l_K = K \cdot \text{diag}(l_K) \tag{51}$$

$$\tilde{V} = V \odot l_V = V \cdot \text{diag}(l_V) \tag{52}$$

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right)\tilde{V} \tag{53}$$

In the feed-forward network (FFN), let W_1 and W_2 be the weight matrices. The intermediate activations are scaled before the output projection:

$$\text{FFN}(x) = \sigma\big(xW_1 \odot l_{FFN}\big)W_2 = \sigma\big(xW_1 \cdot \text{diag}(l_{FFN})\big)W_2 \tag{54}$$

The base model weights W_Q, W_K, W_V, W_1, W_2 are frozen. This diagonal matrix transformation scales the columns of the projection matrices with minimal parameter updates.

Architecture Flow Diagram

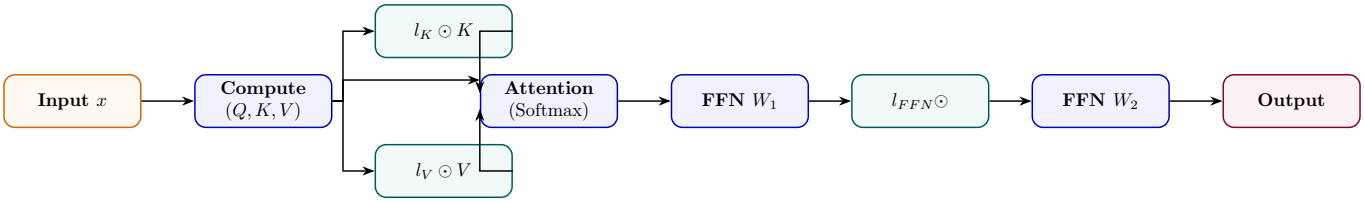


Figure 26: IA^3: three tiny scaling vectors rescale keys, values, and FFN intermediate activations.

State Transition Table

Property	Before	After
Trainable params	0 (frozen)	$3dL$ scalars
Init value of l vectors	N/A	**1** (identity behaviour)
K, V, FFN activations	Unchanged	Element-wise scaled
Param overhead vs LoRA	N/A	$\sim$10$\times$ fewer params
Task adaptation strength	None	Moderate

Table 15: State properties before and after IA^3 injection.

Adapter Tuning

Mathematical Foundation

Adapter tuning inserts small bottleneck layers into the transformer block, typically after attention and feed-forward modules. Let $h \in \mathbb{R}^d$ be the input hidden state. In the Pfeiffer architecture, a single adapter is inserted per block. In the Houlsby architecture, two adapters are inserted per block. The mathematical transformation for an adapter block is:

$$h' = h + \sigma(hW_{down})W_{up} \tag{55}$$

where $W_{down} \in \mathbb{R}^{d \times r}$ projects to a bottleneck dimension $r \ll d$, σ is an activation function (typically GELU), and $W_{up} \in \mathbb{R}^{r \times d}$ projects back to the original dimension. To ensure the adapter behaves as an identity mapping at the start of training, W_{up} is initialized to zero:

$$h'\big|_{t=0} = h + \sigma(hW_{down}) \cdot 0 = h \tag{56}$$

Architecture Flow Diagram

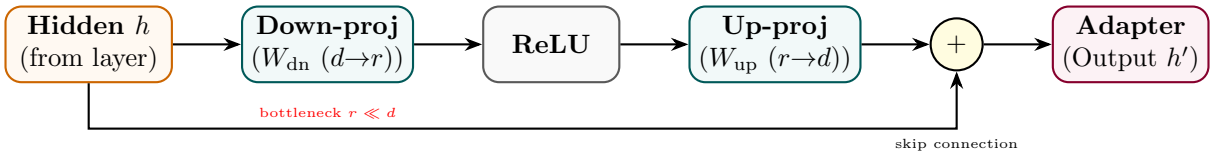


Figure 27: Adapter module: down-project to rank r, apply nonlinearity, up-project back, plus skip connection.

State Transition Table

Property	Before	After
Sub-layer output path	Direct pass-through	+adapter bottleneck
Transformer weights	Trainable	Frozen
New params per layer	0	$2dr + 2d$
Adapter init	N/A	Near-identity ($W_{\text{dn}} \sim \mathcal{N}$)
Inference latency	Baseline	+two extra matmuls per layer

Table 16: State properties before and after adapter insertion.

Prefix Tuning

Mathematical Foundation

Prefix Tuning prepends continuous task-specific virtual prefix vectors $P_K, P_V \in \mathbb{R}^{l \times d}$ directly to the Key and Value matrices at every transformer layer. Let the original Key and Value matrices for sequence length T be $K, V \in \mathbb{R}^{T \times d}$. The modified Key and Value matrices become:

$$\tilde{K} = [P_K; K] \in \mathbb{R}^{(l+T) \times d} \tag{57}$$

$$\tilde{V} = [P_V; V] \in \mathbb{R}^{(l+T) \times d} \tag{58}$$

To stabilize optimization, during training the prefix is reparameterized using a Multilayer Perceptron (MLP) over a small source embedding E_k:

$$P_K = \text{MLP}_K(E_k), \quad P_V = \text{MLP}_V(E_k) \tag{59}$$

where $E_k \in \mathbb{R}^{l \times d_{\text{mid}}}$. The MLP is discarded at inference, and only the generated static prefix matrices P_K, P_V are saved and used. The attention output is computed as:

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q[P_K; K]^\top}{\sqrt{d_k}}\right)[P_V; V] \tag{60}$$

Architecture Flow Diagram

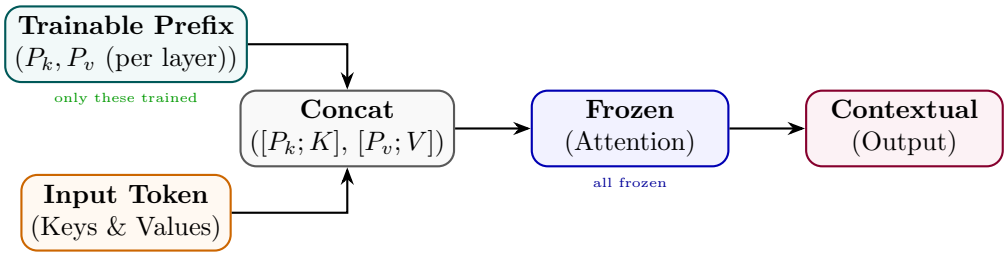


Figure 28: Prefix Tuning: trainable prefix vectors extend the key/value context at every attention layer.

State Transition Table

Property	Before	After
Attention context length	T tokens	$T + l$ tokens (prefix prepended)
Transformer params	Trainable	Fully frozen
Trainable params	0	$2 \times l \times d_h \times L$
Prefix init	N/A	Via MLP reparameterisation
Task switch at inference	Reload model	Swap prefix only

Table 17: State properties before and after prefix tuning.

Prompt Tuning

Mathematical Foundation

Prompt tuning prepends p learnable embedding vectors $P \in \mathbb{R}^{p \times d_{emb}}$ to the input word representations:

$$\tilde{X} = [P; X_e] \in \mathbb{R}^{(p+T) \times d_{emb}} \tag{61}$$

where $X_e \in \mathbb{R}^{T \times d_{emb}}$ is the original sequence word embedding matrix. The entire transformer backbone parameters θ_{base} remain frozen, and gradients are backpropagated only to the prompt parameters P:

$$\nabla_P \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \tilde{X}_{1..p}} \tag{62}$$

During backpropagation, the gradients are passed through all layers of the frozen transformer and update only the prompt embeddings.

Architecture Flow Diagram

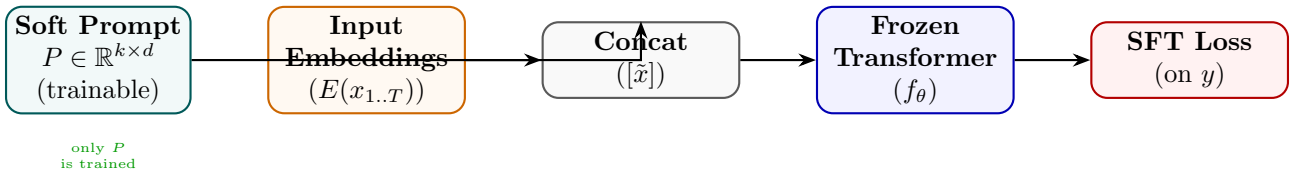


Figure 29: Prompt Tuning: soft tokens prepended at the embedding layer; the full transformer is frozen.

State Transition Table

Property	Before	After
Trainable params	Full model	$k \times d_{\text{emb}}$ only
Prompt format	Discrete hand-crafted	Continuous soft vectors
Transformer weights	All trainable	All frozen
Task switching	Reload fine-tuned model	Swap P only
Sensitivity to init	N/A	Strong (init from class labels helps)

Table 18: State properties before and after prompt tuning.

P-Tuning v2

Mathematical Foundation

P-Tuning v2 extends prompt tuning by prepending learnable virtual tokens $P^{(l)} \in \mathbb{R}^{p \times 2d_{attn}}$ directly to the attention key and value states at every layer $l \in [1, L]$ of the network. Unlike prefix tuning, P-Tuning v2 removes the MLP reparameterization network, directly optimizing the prefix parameters $P^{(l)}$:

$$\tilde{K}^{(l)} = [P_K^{(l)}; K^{(l)}] \tag{63}$$

$$\tilde{V}^{(l)} = [P_V^{(l)}; V^{(l)}] \tag{64}$$

This deep prefix injection resolves the capacity limitation of shallow prompt tuning, allowing small models (< 10B parameters) to achieve performance comparable to full fine-tuning.

Architecture Flow Diagram

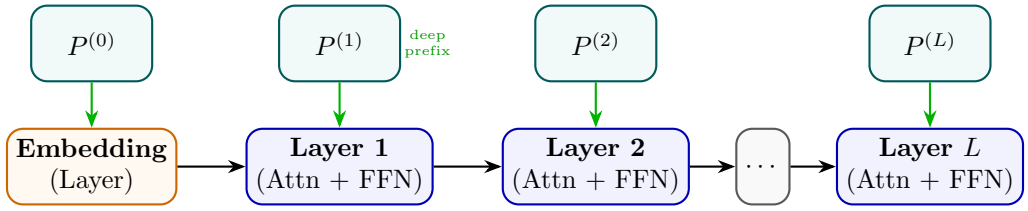


Figure 30: P-Tuning v2: independent trainable prefix tokens injected at every transformer layer.

State Transition Table

Property	Before (Prompt Tuning)	After (P-Tuning v2)
Prefix layers	Embedding only	All L layers
Trainable params	$k \times d$	$L \times l \times d$
NLU performance	Degrades at small scale	Competitive at all scales
Conditioning depth	Shallow (first layer)	Deep (every layer)
Task-specific storage	Single P matrix	L prefix matrices

Table 19: State properties comparing Prompt Tuning and P-Tuning v2.

Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods — Part 2: Preference Alignment, Reinforcement Learning, and Reward Modeling Chief Strategist J June 13, 2026

Contents

Part IV
Preference Alignment Methods

Reinforcement Learning from Human Feedback (RLHF)

Mathematical Foundation

RLHF aligns language models with human preferences in a two-stage process.

First, a Reward Model (RM) $r_\phi(x, y)$ is trained on a dataset of human comparisons $\mathcal{D} = \{(x, y_w, y_l)\}$, where y_w is preferred to y_l given prompt x. The preference is modeled using the Bradley-Terry preference model:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big) = \frac{1}{1 + \exp\big(-(r_\phi(x, y_w) - r_\phi(x, y_l))\big)} \tag{65}$$

The RM is trained by minimizing the negative log-likelihood of this pairwise choice:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}}\left[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\right] \tag{66}$$

Second, the SFT policy parameters θ are fine-tuned to maximize the expected reward under the learned RM, while regularized by a Kullback-Leibler (KL) divergence constraint to the reference policy π_{ref} (preventing policy collapse and reward hacking):

$$\mathcal{J}_{\text{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}_x, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big)\right] \tag{67}$$

where β is a regularizing coefficient, and the KL term at each token generation step is factorized as:

$$D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big) = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} \tag{68}$$

Architecture Flow Diagram

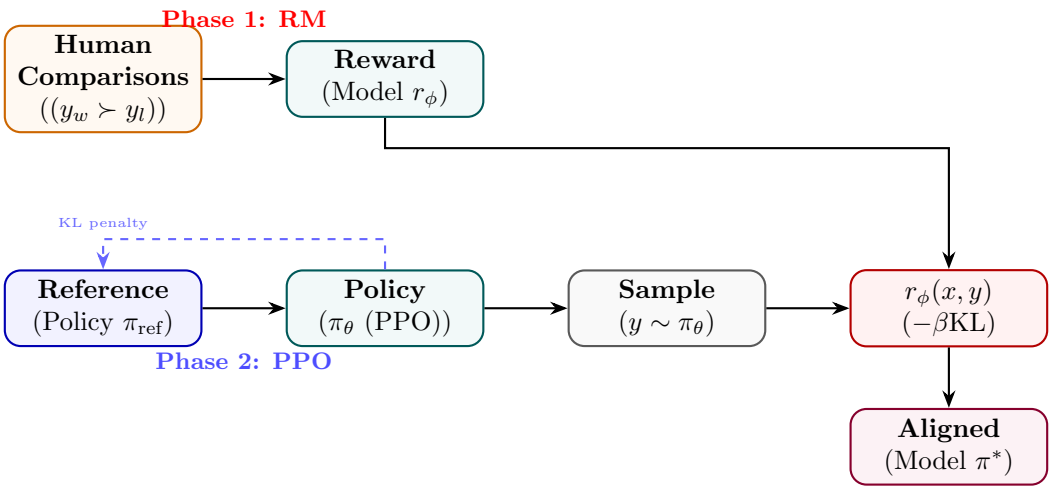


Figure 31: RLHF: reward model trained on human comparisons then used to guide PPO policy optimisation.

State Transition Table

Property	Before	After
Policy type	SFT checkpoint	RLHF-aligned policy
Reward signal	None	Human preference score
KL from reference	0	Bounded by β
Human preference win-rate	$\sim$50%	$> 70\%$
Training complexity	Single loop	Two-phase (RM + PPO)

Table 20: State properties before and after RLHF.

Direct Preference Optimization (DPO)

Mathematical Foundation

DPO eliminates the need for training an explicit reward model or running reinforcement learning. Starting from the KL-constrained RL objective:

$$\max_{\pi} \mathbb{E}_{x,y\sim\pi}\left[r(x,y)\right] - \beta D_{\mathrm{KL}}(\pi(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)) \tag{69}$$

we formulate the unconstrained objective using a partition function $Z(x)$:

$$\max_{\pi} \mathbb{E}_{x}\left[\mathbb{E}_{y\sim\pi}\left[r(x,y) - \beta \log \frac{\pi(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}\right]\right] \tag{70}$$

The closed-form optimal policy solution $\pi^*(y \mid x)$ under this objective is:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)\exp\left(\frac{r(x,y)}{\beta}\right), \quad Z(x) = \sum_{y} \pi_{\mathrm{ref}}(y \mid x)\exp\left(\frac{r(x,y)}{\beta}\right) \tag{71}$$

By rearranging this relation, we express the implicit reward function $r(x,y)$ in terms of the policy likelihoods:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x) \tag{72}$$

Substituting this expression into the Bradley-Terry preference probability $P(y_w \succ y_l \mid x) = \sigma(r(x,y_w) - r(x,y_l))$, the partition function $Z(x)$ cancels out, resulting in the DPO loss function:

$$\mathcal{L}_{\mathrm{DPO}}(\theta;\pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right] \tag{73}$$

Architecture Flow Diagram

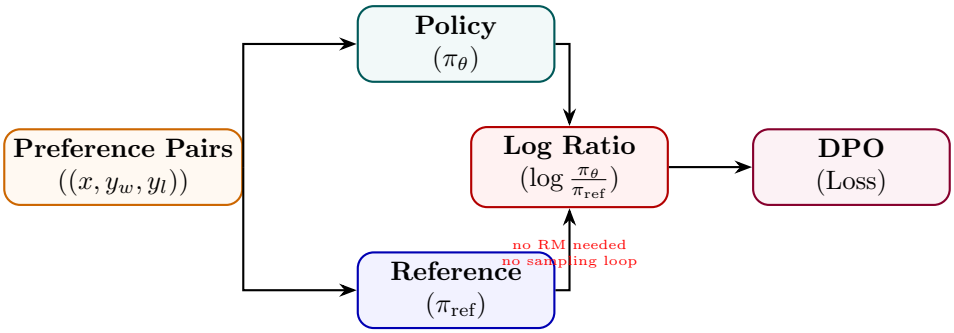


Figure 32: DPO: both policy and frozen reference score chosen/rejected responses; loss uses their log-ratio difference.

State Transition Table

Property	Before (RLHF)	After (DPO)
Reward model	Explicit r_ϕ	Implicit in log-ratio
Sampling loop	Yes (PPO rollouts)	No
Reference policy	Frozen copy	Frozen copy
Training stability	Sensitive to reward hacking	More stable
Compute cost	High (2-phase)	Lower (single SFT-like pass)

Table 21: State properties comparing RLHF and DPO.

Identity Preference Optimization (IPO)

Mathematical Foundation

IPO relaxes the Bradley-Terry preference assumption of DPO to prevent overfitting and improve generalization on preference pairs. By applying identity mapping directly to the pairwise probability, the IPO objective minimizes the mean squared error between the policy log-ratio difference and a target preference margin:

$$\mathcal{L}_{\text{IPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] + \text{regularization} \tag{74}$$

Formally, IPO minimizes the quadratic loss of the difference of log-ratios relative to a regularizing parameter τ:

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\left(\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \frac{\tau}{2\beta}\right)^2\right] \tag{75}$$

This bounds the likelihood shift of both preferred and dispreferred sequences, preventing the model from outputting degenerate probabilities for preferred responses.

Architecture Flow Diagram

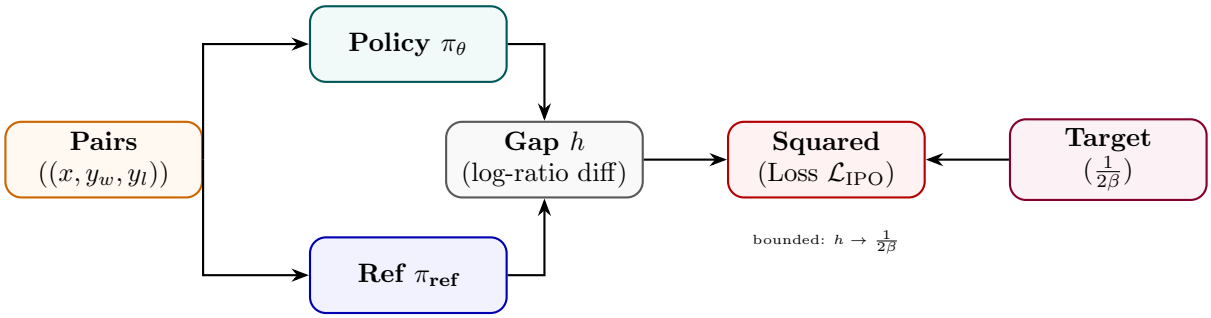


Figure 33: IPO: like DPO but the log-ratio gap is penalised toward a finite target $1/(2\beta)$.

State Transition Table

Property	Before (DPO)	After (IPO)
Loss type	Log-sigmoid	Squared error
Divergence at optimum	Unbounded	Bounded by $\frac{1}{2\beta}$
Target preference gap	Maximise	Converge to $\frac{1}{2\beta}$
Regularisation	KL implicit	Explicit squared bound
Training saturation	Can saturate	More stable

Table 22: State properties comparing DPO and IPO.

Odds Ratio Preference Optimization (ORPO)

Mathematical Foundation

ORPO combines SFT loss and an odds-ratio contrast in a single objective no reference model needed. The odds of generating y under policy π_θ is $\text{odds}(y|x) = \pi_\theta(y|x)/(1-\pi_\theta(y|x))$. The combined loss is:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} - \lambda\,\mathbb{E}\left[\log\sigma\left(\log\frac{\text{odds}(y_w|x)}{\text{odds}(y_l|x)}\right)\right] \tag{76}$$

The SFT term increases $\pi_\theta(y_w|x)$ while the odds-ratio term simultaneously penalises y_l without requiring a frozen reference copy.

Architecture Flow Diagram

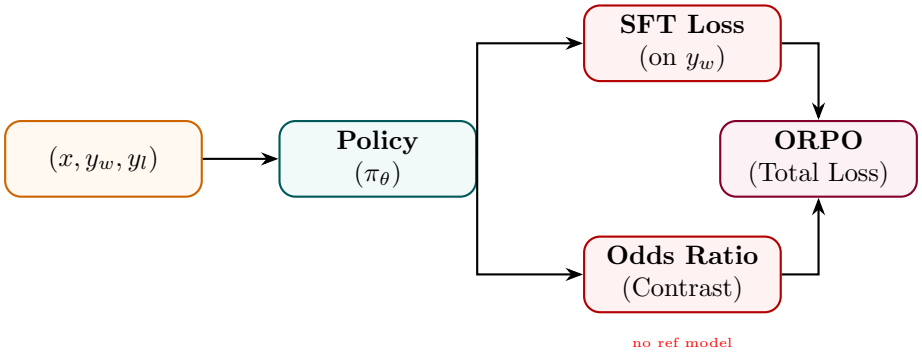


Figure 34: ORPO: SFT loss on the chosen response combined with an odds-ratio preference contrast.

State Transition Table

Property	Before	After
Reference model	Required (DPO/RLHF)	Not required
Training phases	2 (SFT then align)	1 (joint)
Loss terms	Single	SFT + odds-ratio
Memory overhead	2× model	1× model only
Data format	Prompt + completion	Triplet (x, y_w, y_l)

Table 23: State properties before and after ORPO.

Kahneman-Tversky Optimization (KTO)

Mathematical Foundation

KTO aligns language models using binary preference labels (e.g., correct/incorrect, helpful/harmful) instead of comparative pairs. This objective mathematically models human utility based on Kahneman and Tversky's prospect theory, which describes how humans weigh losses and gains asymmetricaly. Let y be a response to prompt x, and $r \in \{0, 1\}$ be its correctness label. The utility of the policy is defined as:

$$\mathcal{L}_{\text{KTO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{x,y}\left[w(y) \cdot v\left(\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - z(x)\right)\right] \tag{77}$$

where $v(u)$ is the value function mapping gains and losses:

$$v(u) = \begin{cases} u & \text{if } u > 0 \\ \lambda u & \text{if } u \leq 0 \end{cases} \tag{78}$$

where $\lambda > 1$ represents loss aversion. The term $z(x)$ is a running reference point representing the expected log-ratio of the policy relative to the reference:

$$z(x) = \mathbb{E}_{y' \sim \pi_{\text{ref}}(\cdot|x)}\left[\log \frac{\pi_\theta(y' \mid x)}{\pi_{\text{ref}}(y' \mid x)}\right] \tag{79}$$

and $w(y)$ is a weighting factor balancing positive and negative feedback frequencies.

Architecture Flow Diagram

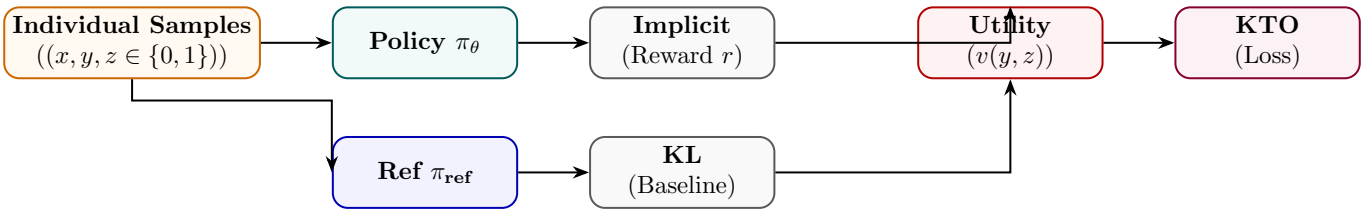


Figure 35: KTO: each sample is individually labelled good/bad; utility function uses prospect theory asymmetry.

State Transition Table

Property	Before	After
Data format	Pairwise (y_w, y_l)	Individual (y, z)
Data collection effort	High (pairwise)	Lower (binary label)
Loss-aversion modelling	None	Asymmetric w_+, w_-
Reference model	Optional	Used for KL baseline
Applicable at scale	Full pairs required	Works with any ratio

Table 24: State properties before and after KTO.

Simple Preference Optimization (SimPO)

Mathematical Foundation

SimPO improves upon DPO by removing the reference policy π_{ref} during training, reducing memory usage, and introducing a length-normalized reward with a target margin γ. The SimPO implicit reward for a sequence y given x is defined as the average log-likelihood:

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_\theta(y \mid x) \tag{80}$$

where $|y|$ is the sequence length in tokens. The SimPO pairwise loss function is formulated as:

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma \right) \right] \tag{81}$$

where $\gamma > 0$ is a target margin that forces the average likelihood of the winning response to exceed the losing response by a strict buffer, preventing over-optimization of short sequences.

Architecture Flow Diagram

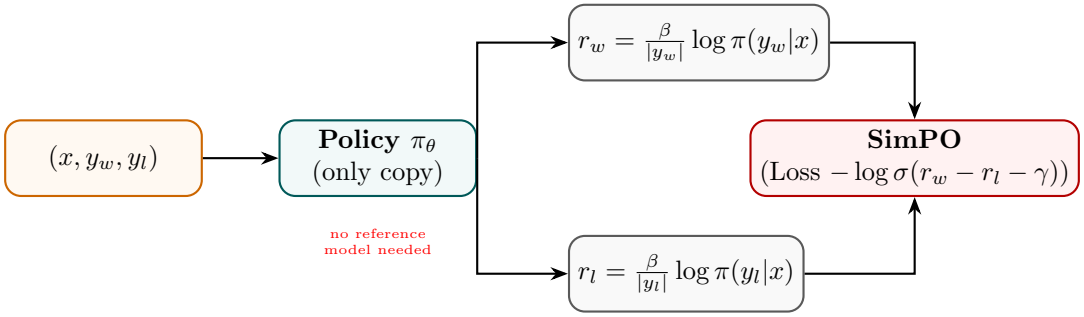


Figure 36: SimPO: length-normalised log-probs replace the reference log-ratio; margin γ enforces quality gap.

State Transition Table

Property	Before (DPO)	After (SimPO)
Reference model	Required	Not needed
Reward definition	Log-ratio π/π_{ref}	Length-norm avg log-prob
Length bias	Possible	Mitigated by $1/\|y\|$
Margin hyperparameter	None	$\gamma > 0$
Memory saving	None	No second model forward pass

Table 25: State properties comparing DPO and SimPO.

Constrained Preference Optimization (CPO)

Mathematical Foundation

CPO formulates alignment as a constrained optimisation: maximise reward while staying within a trust region ϵ of the reference policy measured in reverse KL divergence:

$$\max_{\pi_\theta} \mathbb{E}[r(x,y)] \quad \text{s.t.} \quad \mathbb{KL}[\pi_{\text{ref}} \| \pi_\theta] \leq \epsilon \tag{82}$$

The Lagrangian relaxation yields a combined training objective:

$$\mathcal{L}_{\text{CPO}} = -\mathbb{E}[r(x,y)] + \mu\big(\mathbb{KL}[\pi_{\text{ref}} \| \pi_\theta] - \epsilon\big) \tag{83}$$

where $\mu \geq 0$ is a dual variable updated online to enforce the constraint.

Architecture Flow Diagram

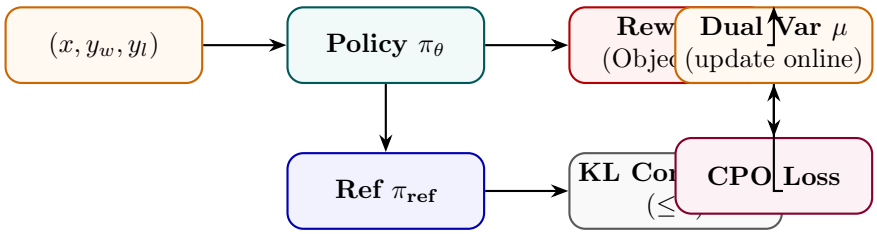


Figure 37: CPO: reward maximisation with an explicit KL constraint; dual variable μ adapts online.

State Transition Table

Property	Before	After
Constraint form	Soft KL penalty	Hard KL constraint $\leq \epsilon$
Dual variable μ	Fixed	Dynamically updated
Policy deviation	Unbounded	Bounded within trust region
Reward maximisation	Soft	Subject to constraint
Training dynamics	Standard	Saddle-point optimisation

Table 26: State properties before and after CPO.

Anchored Preference Optimization (APO)

Mathematical Foundation

APO introduces an anchor term that explicitly regularises the policy toward the reference model while simultaneously pushing chosen responses up and rejected responses down:

$$\mathcal{L}_{\text{APO}} = \mathbb{E}\left[\underbrace{-\log \sigma\big(\beta(r_w - r_l)\big)}_{\text{preference}} + \lambda \underbrace{\Big(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big)^2}_{\text{anchor}}\right] \tag{84}$$

The anchor term is minimised when both the chosen and rejected log-ratios equal a fixed reference, preventing the model from drifting.

Architecture Flow Diagram

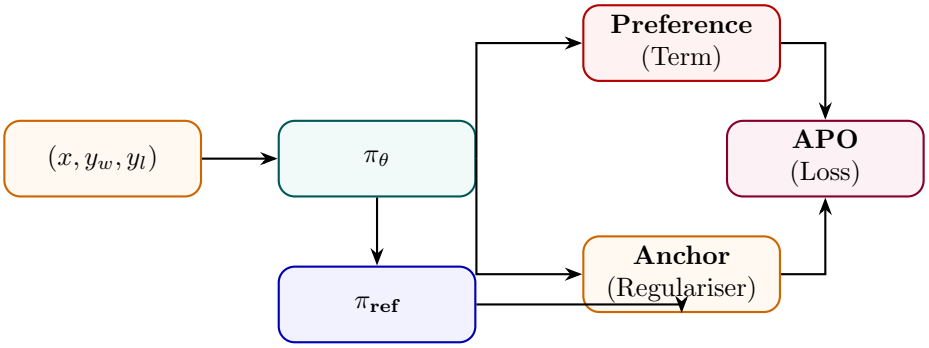


Figure 38: APO: combines standard preference loss with an anchor regulariser against the reference model.

State Transition Table

Property	Before (DPO)	After (APO)
Anchor regularisation	Implicit KL	Explicit squared anchor
Reference role	Log-ratio denominator	Explicit target
Policy drift	Can drift	Penalised
Loss terms	1	2 (preference + anchor)
Stability	Moderate	Higher (dual grounding)

Table 27: State properties comparing DPO and APO.

Rank Responses from Human Feedback (RRHF)

Mathematical Foundation

RRHF generates K candidate responses $\{y^{(1)}, \ldots, y^{(K)}\}$ from the model, ranks them by human or AI annotators, and trains the model to assign higher log-probability to higher-ranked responses via a pairwise ranking loss:

$$\mathcal{L}_{\text{RRHF}} = \sum_{i<j} \max\bigl(0,\, \text{score}(y^{(j)}) - \text{score}(y^{(i)}) + \delta\bigr) + \lambda \mathcal{L}_{\text{SFT}}(y^{(1)}) \tag{85}$$

where $\text{score}(y^{(k)}) = \log P_\theta(y^{(k)}|x)/|y^{(k)}|$ is the normalised log-prob. The SFT term anchors the top-ranked response.

Architecture Flow Diagram

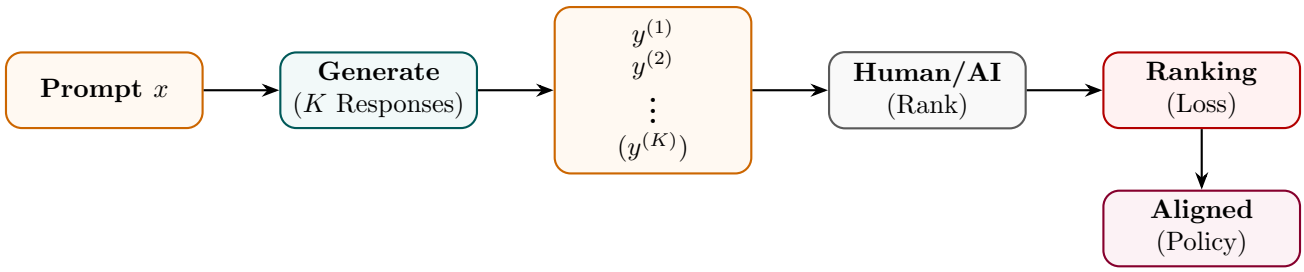


Figure 39: RRHF: K responses are generated and ranked; policy trained to match rank order via pairwise loss.

State Transition Table

Property	Before	After
Responses per prompt	1	K (ranked)
Supervision signal	Binary (win/lose)	Full rank ordering
Loss type	Log-sigmoid	Pairwise margin
Top-response training	Separate SFT	Joint SFT + ranking
Annotation granularity	Single comparison	Full K-way ranking

Table 28: State properties before and after RRHF.

Reinforcement Learning from AI Feedback (RLAIF)

Mathematical Foundation

RLAIF replaces expensive human annotators with a capable AI model ("AI annotator" $\mathcal{A}$) to generate preference labels at scale. The AI annotator scores pairs via a prompt p:

$$\hat{r}(x, y_i, y_j) = P_{\mathcal{A}}(y_i \succ y_j \mid p, x, y_i, y_j) \tag{86}$$

These AI-generated preferences are then used identically to human preferences to train the RM and run PPO (or DPO):

$$\mathcal{L}_{\text{RLAIF}} = -\mathbb{E}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))], \quad (y_w \succ y_l) \text{ from } \mathcal{A} \tag{87}$$

Architecture Flow Diagram

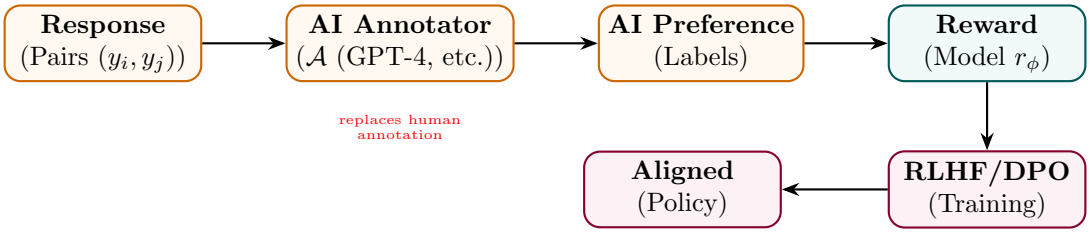


Figure 40: RLAIF: an AI model generates preference labels at scale, replacing costly human annotators.

State Transition Table

Property	Before (RLHF)	After (RLAIF)
Annotation source	Humans	AI annotator $\mathcal{A}$
Annotation cost	High ($$$)	Low (API calls)
Annotation scale	Limited (thousands)	Massive (millions)
Annotation consistency	Variable	Consistent per model
Annotation bias	Human bias	AI model bias

Table 29: State properties comparing RLHF and RLAIF.

Constitutional AI

Mathematical Foundation

Constitutional AI uses a set of principles $\mathcal{C} = \{c_1, \ldots, c_M\}$ (the "constitution") to iteratively critique and revise model outputs before training. In the RL phase, an AI Feedback model scores responses against the constitution:

$$r_{\text{CAI}}(x, y) = \frac{1}{M} \sum_{m=1}^{M} P_{\mathcal{A}}(\text{harmless} \mid c_m, x, y) \tag{88}$$

Two training phases: (1) supervised revision via critique-revise loop, (2) RLHF with AI-generated preference labels from the constitution.

Architecture Flow Diagram

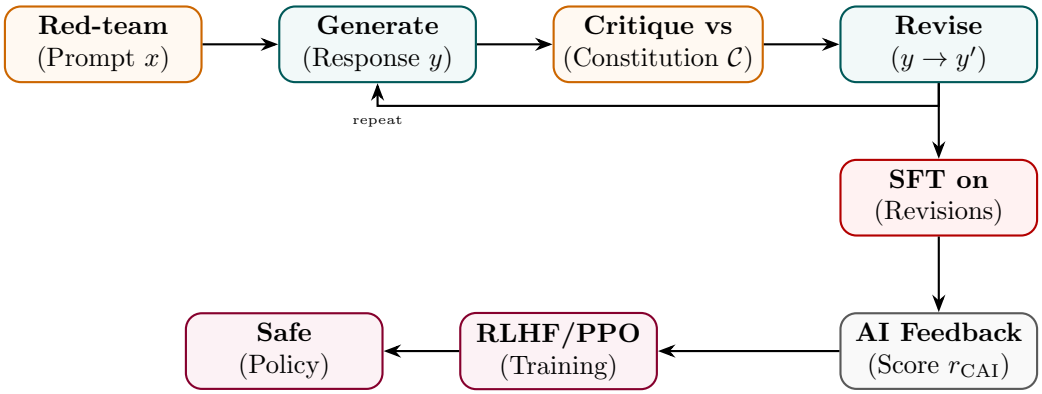


Figure 41: Constitutional AI: critique-revise loop creates SFT data; AI feedback from constitution guides RLHF.

State Transition Table

Property	Before	After
Safety signal	None / Human labels	AI + constitutionl principles
Training phases	SFT only	Critique-revise SFT + RLAIF
Red-team coverage	Manual	Systematic via constitution
Harmlessness	Moderate	Substantially improved
Helpfulness trade-off	N/A	Explicitly balanced

Table 30: State properties before and after Constitutional AI training.

Part V
Reinforcement Learning Training Methods

Proximal Policy Optimization (PPO)

Mathematical Foundation

PPO optimizes policy parameters θ using a clipped objective function to prevent destabilizing parameter updates. Let $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ be the action probability ratio. The surrogate objective maximizes:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{89}$$

where ϵ is a clipping hyperparameter (typically 0.1 or 0.2), and $\hat{A}_t$ is the advantage estimated using Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V, \quad \delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{90}$$

where V_ϕ is the value network parameters, γ is the discount factor, and λ controls bias-variance trade-offs. The value function loss is clipped to match policy updates:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\max \left((V_\phi(s_t) - V_t^{\text{target}})^2,\ (V_{\phi_{\text{old}}}(s_t) + \text{clip}(V_\phi(s_t) - V_{\phi_{\text{old}}}(s_t), -c, c) - V_t^{\text{target}})^2 \right) \right] \tag{91}$$

Architecture Flow Diagram

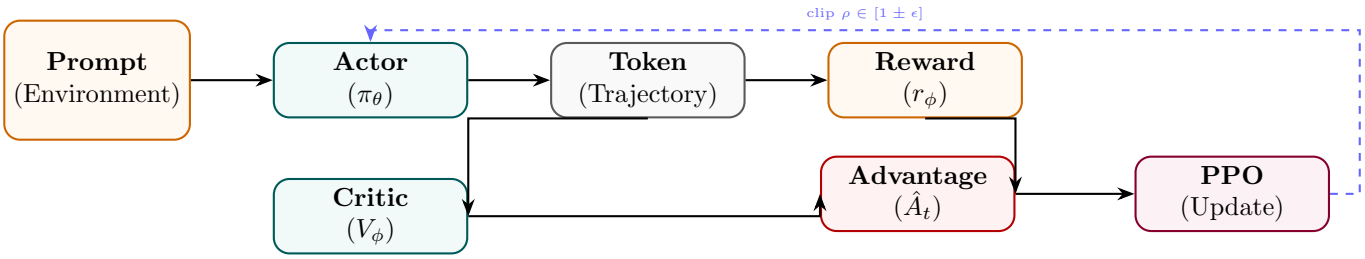


Figure 42: PPO: actor generates trajectory, critic estimates value, advantage drives clipped surrogate update.

State Transition Table

Property	Before	After
Policy update size	Unconstrained	Clipped by ϵ
Value network	None	Trained V_ϕ
Advantage estimation	N/A	GAE $\hat{A}_t$
Training stability	Unstable (vanilla PG)	Stable (clip constraint)
Reward signal	Sparse (end of seq)	Discounted returns

Table 31: State properties before and after PPO setup.

Group Relative Policy Optimization (GRPO)

Mathematical Foundation

GRPO optimizes policies by calculating rewards relative to a group of sampled outputs, eliminating the need for a separate critic model. For a given prompt x, the policy generates a group of G candidate completions $\{y_1, y_2, \ldots, y_G\}$. Each candidate y_i is evaluated by a reward function (or judge) yielding score r_i. The relative advantage A_i of each candidate is computed as:

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)} = \frac{r_i - \frac{1}{G}\sum_j r_j}{\sqrt{\frac{1}{G}\sum_j (r_j - \bar{r})^2 + \epsilon}} \tag{92}$$

The policy objective maximizes the clipped advantage with a KL penalty directly evaluated on the generated tokens:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G}\left[\min\left(\rho_i(\theta)A_i,\ \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon)A_i\right) - \beta D_{\text{KL}}\left(\pi_\theta(y_i \mid x) \,\|\, \pi_{\text{ref}}(y_i \mid x)\right)\right] \tag{93}$$

where the probability ratio $\rho_i(\theta)$ is defined as:

$$\rho_i(\theta) = \frac{\pi_\theta(y_i \mid x)}{\pi_{\theta_{\text{old}}}(y_i \mid x)} \tag{94}$$

Architecture Flow Diagram

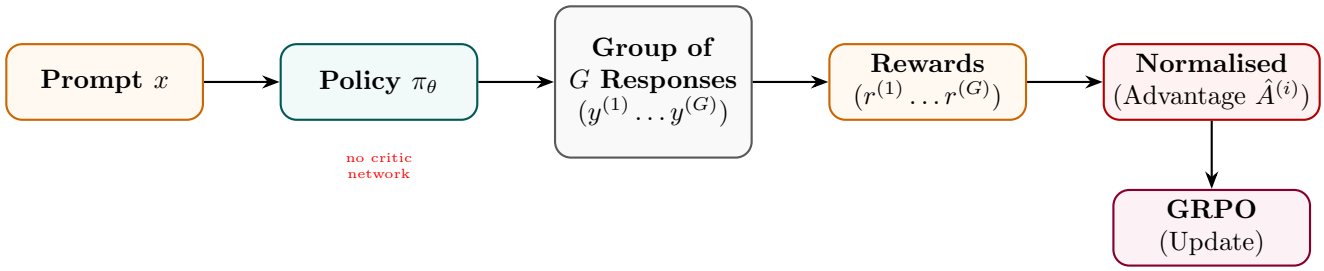


Figure 43: GRPO: group of responses scored together; within-group normalised reward replaces value network.

State Transition Table

Property	Before (PPO)	After (GRPO)
Critic / value network	Required	Not needed
Advantage estimation	GAE (per token)	Group normalisation
Memory overhead	2× model (actor + critic)	≈ 1× model
Samples per prompt	1	G (group sampling)
Variance reduction	Value baseline	Mean group reward

Table 32: State properties comparing PPO and GRPO.

Rejection Fine-Tuning (RFT)

Mathematical Foundation

RFT generates K candidate solutions per problem using the current policy, filters by a correctness verifier $\mathcal{V}$, and fine-tunes on the correct subset essentially a self-improvement loop without RL:

$$\mathcal{D}_{\text{correct}} = \{(x, y^{(k)}) : \mathcal{V}(x, y^{(k)}) = 1,\ y^{(k)} \sim \pi_\theta,\ k = 1..K\} \tag{95}$$

$$\mathcal{L}_{\text{RFT}} = -\sum_{(x,y)\in\mathcal{D}_{\text{correct}}} \sum_t \log P_\theta(y_t \mid x, y_{<t}) \tag{96}$$

The process repeats: train on correct outputs, generate new candidates, filter again.

Architecture Flow Diagram

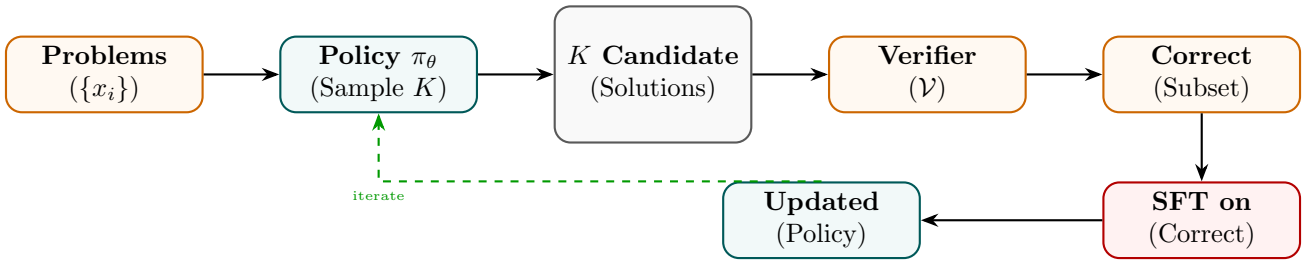


Figure 44: RFT: policy samples K solutions, verifier filters correct ones, SFT improves the policy iteratively.

State Transition Table

Property	Before	After
Data source	Static human labels	Self-generated correct solutions
RL component	Required (PPO)	None (SFT only)
Verifier	None	Required (exact-match, unit tests)
Pass@K improvement	Baseline	Improves with iterations
Training complexity	High (RL)	Standard SFT loop

Table 33: State properties before and after RFT.

REINFORCE

Mathematical Foundation

REINFORCE is the foundational policy gradient algorithm. For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$, the gradient of expected return is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau) \nabla_\theta \log \pi_\theta(\tau)], \quad G(\tau) = \sum_{t=0}^{T} \gamma^t r_t \tag{97}$$

For LLMs, the trajectory is the token sequence, each token is an action, and r_T (end reward) is discounted back. A baseline b (e.g., mean reward) reduces variance:

$$\nabla_\theta J \approx \frac{1}{N} \sum_n (G_n - b) \nabla_\theta \log \pi_\theta(y_n | x_n) \tag{98}$$

Architecture Flow Diagram

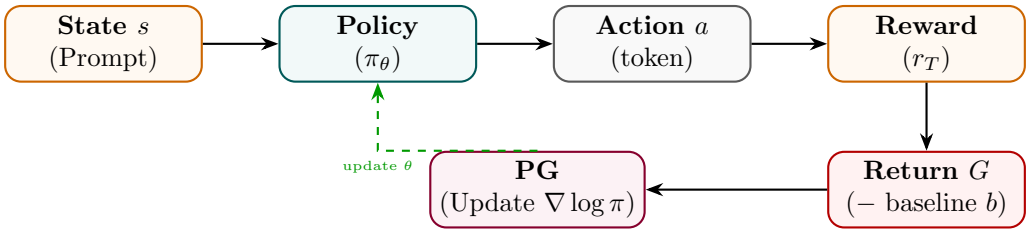


Figure 45: REINFORCE: policy samples action sequence, receives return G, updates via policy gradient.

State Transition Table

Property	Before	After
Gradient type	Supervised CE	Policy gradient $G\nabla \log \pi$
Variance	N/A	High (needs baseline)
Critic network	Not needed	Not needed
Reward type	N/A	Any differentiable or sparse
Update frequency	Per batch	Per episode/trajectory

Table 34: State properties before and after REINFORCE setup.

Advantage Actor-Critic (A2C / A3C)

Mathematical Foundation

A2C uses a shared backbone with two heads: a policy head (actor) and a value head (critic). The advantage $A(s,a) = Q(s,a) - V(s)$ reduces gradient variance:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_t\big[A(s_t,a_t)\log \pi_\theta(a_t|s_t)\big] \tag{99}$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t\big[(r_t + \gamma V(s_{t+1}) - V(s_t))^2\big] \tag{100}$$

A3C extends A2C with asynchronous parallel workers each running independent environments and gradients are asynchronously applied to a central parameter server.

Architecture Flow Diagram

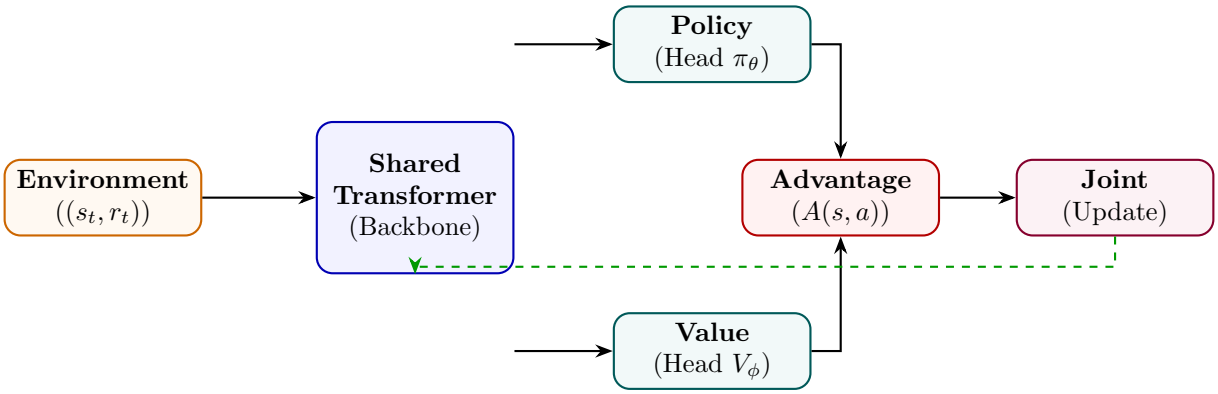


Figure 46: A2C: shared backbone feeds actor (policy) and critic (value) heads; advantage combines their outputs.

State Transition Table

Property	Before (REINFORCE)	After (A2C)
Variance reduction	Baseline only	Full advantage estimation
Value network	Separate	Shared backbone, separate head
Gradient stability	Low	Higher (advantage)
Parallelism	None	A3C: asynchronous workers
Sample efficiency	Low	Higher (on-policy)

Table 35: State properties comparing REINFORCE and A2C.

Self-Play Reinforcement Learning

Mathematical Foundation

In Self-Play RL, the model competes against previous versions of itself. At iteration n, the opponent is a snapshot $\pi_{\theta_{n-1}}$ (or mixture of past policies). The reward is determined by competitive outcome:

$$r(y_\theta, y_{\theta_{\text{old}}}) = \begin{cases} +1 & \text{if } y_\theta \text{ is judged better} \\ -1 & \text{otherwise} \end{cases} \tag{101}$$

$$\mathcal{J} = \mathbb{E}_{\pi_\theta, \pi_{\theta_{\text{old}}}} \left[r(y_\theta, y_{\theta_{\text{old}}}) \right] \tag{102}$$

This creates an ever-escalating difficulty curriculum since the opponent improves alongside the model.

Architecture Flow Diagram

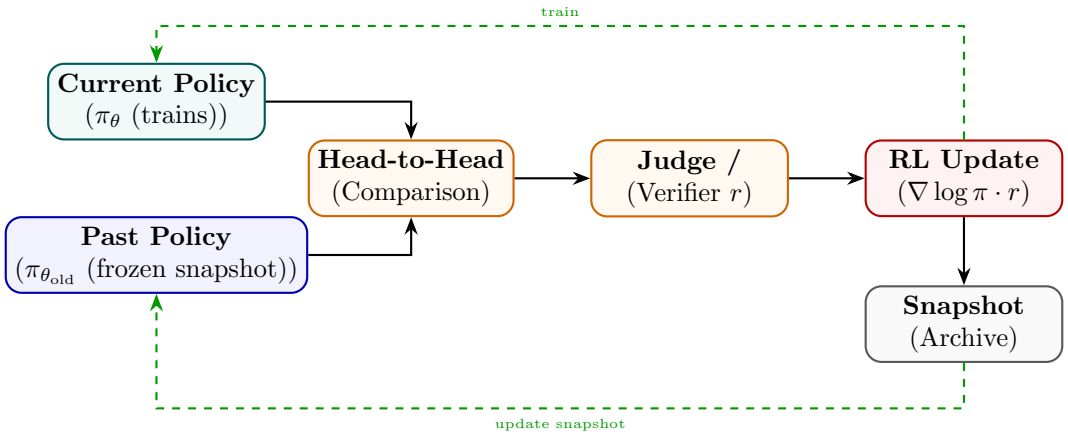


Figure 47: Self-Play RL: current policy competes against a frozen past snapshot; win/loss drives RL updates.

State Transition Table

Property	Before	After
Opponent	None / static	Evolving past self
Difficulty curriculum	Fixed	Auto-scaling
Reward source	External judge	Comparative win/loss
Policy snapshot archive	N/A	Maintained
Capability ceiling	Fixed by data	Self-determined

Table 36: State properties before and after Self-Play RL setup.

Monte Carlo Tree Search RL (MCTS-RL)

Mathematical Foundation

MCTS-RL builds a reasoning tree by alternating between *selection* (UCT formula), *expansion*, *rollout*, and *backpropagation*. The UCT score for node n:

$$\mathrm{UCT}(n) = \frac{W(n)}{N(n)} + c\sqrt{\frac{\ln N(\mathrm{parent}(n))}{N(n)}} \tag{103}$$

where $W(n)$ is total rollout reward and $N(n)$ is visit count. The policy is improved by training on trajectories weighted by MCTS visit counts:

$$\pi_\theta \leftarrow \pi_\theta + \eta\,\mathbb{E}\big[\text{MCTS visits}(a|s)\cdot\nabla_\theta \log \pi_\theta(a|s)\big] \tag{104}$$

Architecture Flow Diagram

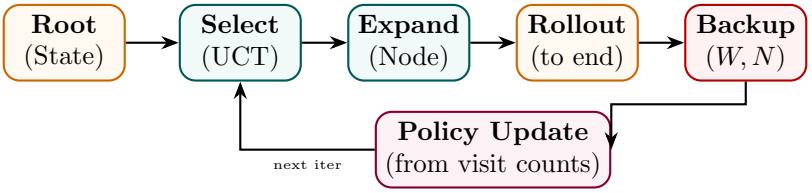


Figure 48: MCTS-RL: select (UCT), expand, rollout, backpropagate; visit counts guide policy improvement.

State Transition Table

Property	Before	After
Search strategy	Greedy / beam	Tree search (UCT)
State value estimate	None	$W(n)/N(n)$ from rollouts
Planning horizon	Local	Multi-step look-ahead
Compute per query	Inference only	Inference + tree expansion
Policy quality	Single-pass	Iteratively refined

Table 37: State properties before and after MCTS-RL integration.

Search-Augmented RL (Search-RL)

Mathematical Foundation

Search-RL augments the reasoning process with an explicit search over candidate next steps. At each step t, the policy generates B candidates $\{s_t^{(1)}, \ldots, s_t^{(B)}\}$, a verifier scores them, and the best is selected:

$$s_t^* = \arg\max_b V(x, s_{1:t-1}, s_t^{(b)}) \tag{105}$$

Training uses the full selected trajectory as RL experience:

$$\mathcal{L}_{\text{SearchRL}} = -\sum_t \mathbb{1}[\mathcal{V}(x, y) = 1] \log \pi_\theta(s_t^* \mid x, s_{1:t-1}) \tag{106}$$

Architecture Flow Diagram

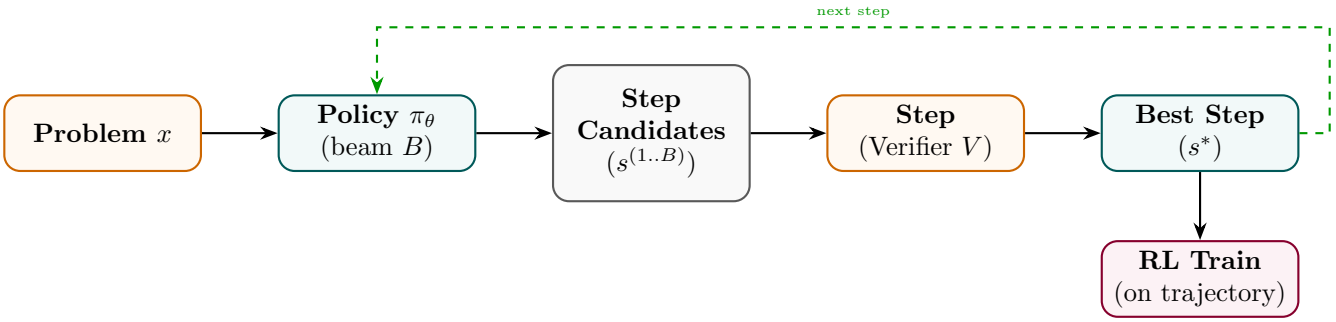


Figure 49: Search-RL: at each step, beam search generates candidates; verifier selects best; policy trained on trajectory.

State Transition Table

Property	Before	After
Step selection	Greedy / top-1	Verifier-guided beam search
Search breadth	1	B candidates per step
Verifier	None	Step-level verifier V
Training signal	Outcome reward	Step-selected trajectory
Error recovery	None	Search can backtrack

Table 38: State properties before and after Search-RL.

Tree-of-Thought Training

Mathematical Foundation

Tree-of-Thought (ToT) Training teaches the model to generate, evaluate, and prune multiple reasoning branches. The model is trained on annotated trees where each branch b_i has a quality score $q_i \in [0, 1]$. The training objective rewards correct branches and penalises dead-ends:

$$\mathcal{L}_{\text{ToT}} = -\sum_i q_i \sum_t \log P_\theta(b_{i,t} \mid x, b_{i,<t}) + (1 - q_i) \sum_t \log(1 - P_\theta(b_{i,t} \mid x, b_{i,<t})) \tag{107}$$

A separate scoring model $s_\phi(x, b_i)$ is trained to evaluate intermediate thought nodes and guide beam search at inference.

Architecture Flow Diagram

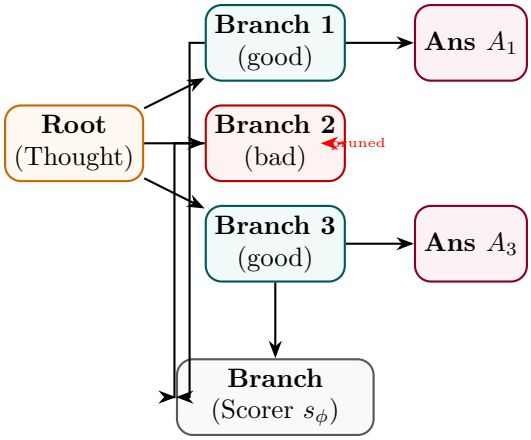


Figure 50: ToT Training: good branches lead to answers; bad branches are pruned; scorer trained to evaluate all.

State Transition Table

Property	Before	After
Reasoning structure	Linear chain	Branching tree
Dead-end handling	None	Explicit pruning
Branch evaluator	None	Trained s_ϕ
Search at inference	Linear	Guided tree beam
Complex planning	Weak	Strong

Table 39: State properties before and after Tree-of-Thought Training.

Self-Consistency Training

Mathematical Foundation

Self-Consistency Training generates K independent reasoning paths and uses majority voting to determine the correct answer. The model is then trained to increase probability of the majority-vote answer y^*:

$$y^* = \arg\max_y \sum_{k=1}^{K} \mathbf{1}[\text{answer}(y^{(k)}) = y], \quad y^{(k)} \sim \pi_\theta(\cdot|x) \tag{108}$$

$$\mathcal{L}_{\text{SC}} = -\sum_{\{k:\,\text{answer}(y^{(k)})=y^*\}} \sum_t \log P_\theta(y_t^{(k)} \mid x, y_{<t}^{(k)}) \tag{109}$$

Paths that agree with the majority vote are reinforced; minority paths are not trained on (or penalised).

Architecture Flow Diagram

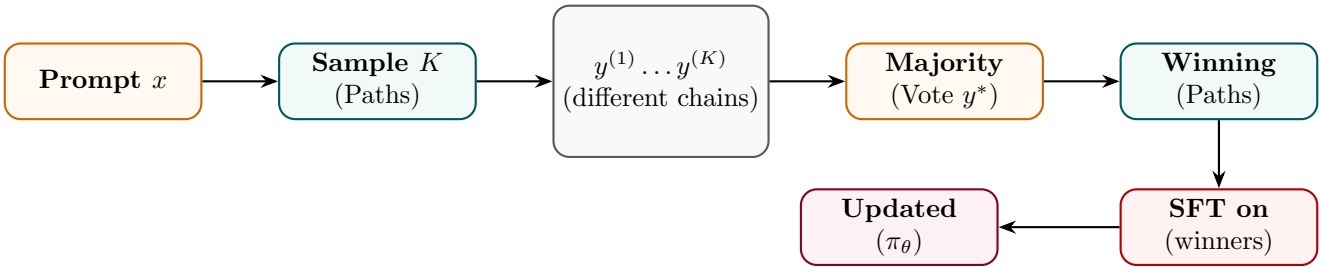


Figure 51: Self-Consistency Training: majority-vote over K paths identifies winning chains; model trained on those.

State Transition Table

Property	Before	After
Inference mode	Single greedy	K diverse samples
Answer selection	Argmax	Majority vote
Training target	All completions	Majority-consistent paths only
Accuracy ceiling	Limited by single pass	Higher (ensemble effect)
Compute cost	$1\times$	$K\times$ at sampling time

Table 40: State properties before and after Self-Consistency Training.

Part VI
Reward Modeling

Reward Model (RM)

Mathematical Foundation

A Reward Model takes a prompt x and a completion y and outputs a scalar preference score. It is trained on human-ranked pairs $(y_w \succ y_l)$ via the Bradley-Terry log-likelihood:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{110}$$

The RM is typically initialised from a fine-tuned LLM with the final token's hidden state projected to a scalar via a linear head $W_r \in \mathbb{R}^{d\times 1}$:

$$r_\phi(x, y) = W_r^\top h_{\text{last}}(x, y) \tag{111}$$

Architecture Flow Diagram

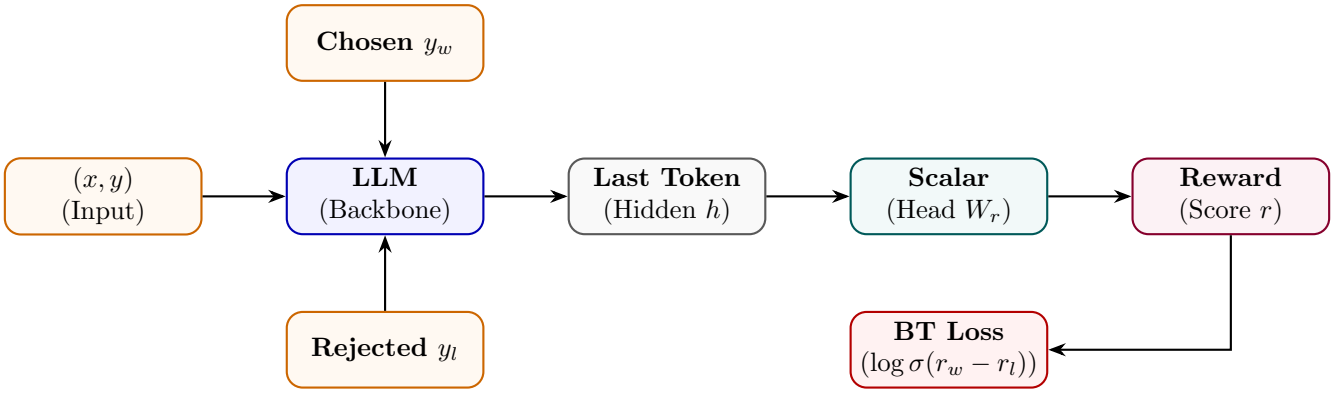


Figure 52: Reward Model: LLM backbone maps (x, y) pair to scalar via linear head; trained on chosen/rejected pairs.

State Transition Table

Property	Before	After
Output type	Token distribution	Scalar reward
Final layer	LM head	Scalar head W_r
Training data	Text completions	(x, y_w, y_l) preference pairs
Loss function	Cross-entropy	Bradley-Terry log-likelihood
Usage	Language generation	RLHF reward signal

Table 41: State properties before and after reward model training.

Process Reward Model (PRM)

Mathematical Foundation

A PRM assigns a reward to each intermediate reasoning step s_t rather than to the final answer alone. Given a solution prefix $(s_1, \ldots, s_t)$, the PRM outputs a binary step quality $q_t \in \{0, 1\}$:

$$r_{\text{PRM}}(x, s_{1:T}) = \prod_{t=1}^{T} r_t, \quad r_t = P_\phi(q_t = 1 \mid x, s_{1:t}) \tag{112}$$

The PRM is trained on step-level human labels with binary cross-entropy:

$$\mathcal{L}_{\text{PRM}} = -\sum_t \big[q_t \log r_t + (1 - q_t) \log(1 - r_t)\big] \tag{113}$$

Architecture Flow Diagram

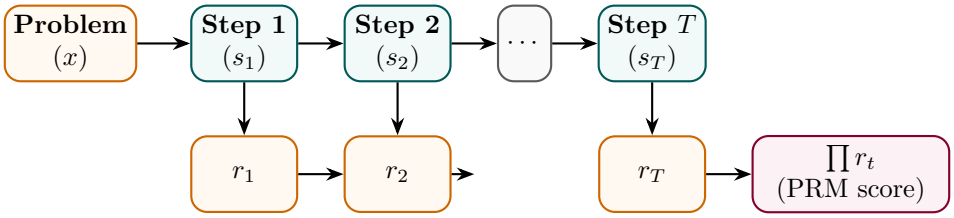


Figure 53: PRM: each reasoning step independently scored; product of step scores gives overall process reward.

State Transition Table

Property	Before (RM)	After (PRM)
Reward granularity	Final answer only	Per-step
Error localisation	None	Identifies failing step
Label annotation	Answer-level	Step-level (expensive)
Reward signal density	Sparse (1 per traj.)	Dense (T per traj.)
Training on correct solutions	Required	Not required

Table 42: State properties comparing RM and PRM.

Outcome Reward Model (ORM)

Mathematical Foundation

An ORM judges only the *correctness of the final answer* y_{ans}, ignoring intermediate steps. For verifiable tasks (math, code), correctness can be checked automatically:

$$r_{\text{ORM}}(x, y) = \mathbf{1}[\text{verify}(y_{\text{ans}}, y^*_{\text{gold}})] \tag{114}$$

For open-ended tasks, the ORM is a learned binary classifier trained on (correct, incorrect) outcome labels:

$$\mathcal{L}_{\text{ORM}} = -\big[c \log P_\phi(y) + (1 - c) \log(1 - P_\phi(y))\big], \quad c \in \{0, 1\} \tag{115}$$

Architecture Flow Diagram

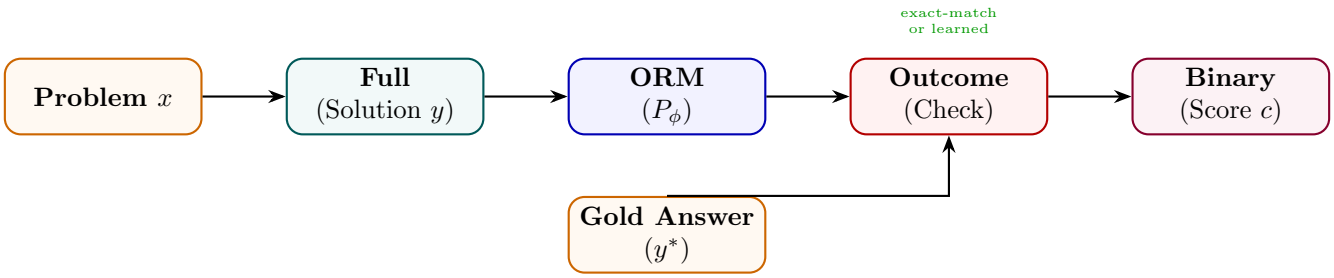


Figure 54: ORM: scores the final answer against gold; can be rule-based (exact-match) or learned classifier.

State Transition Table

Property	Before	After
Reward scope	N/A	Final answer only
Intermediate credit	None	None
Annotation cost	N/A	Low (binary label)
Verifiable tasks	Manual check	Automatic (rule-based)
Open-ended tasks	None	Learned classifier

Table 43: State properties before and after ORM setup.

Verifier Model

Mathematical Foundation

A Verifier Model is a binary classifier that determines whether a candidate solution is correct. Unlike an ORM that outputs a probability, verifiers typically use a hard decision rule learned from (correct, incorrect) solution pairs:

$$\hat{v}(x, y) = \mathbf{1}\big[P_\phi(\text{correct} \mid x, y) \geq \tau\big] \tag{116}$$

Verifiers are used in test-time search (Best-of-N) and training-time filtering (RFT). The decision threshold τ trades off precision and recall on the verification task:

$$\mathcal{L}_{\text{ver}} = -\big[v \log P_\phi + (1 - v) \log(1 - P_\phi)\big], \quad v \in \{0, 1\} \tag{117}$$

Architecture Flow Diagram

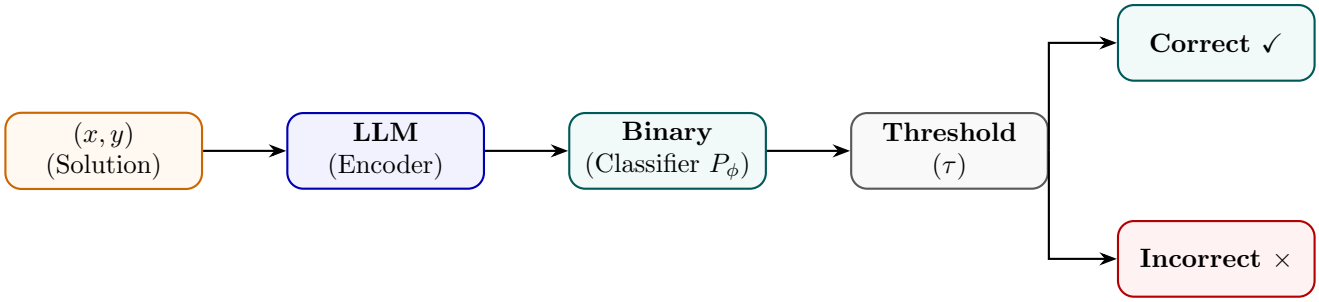


Figure 55: Verifier Model: encodes solution, classifies as correct/incorrect via learned threshold.

State Transition Table

Property	Before	After
Output	Continuous score	Binary decision
Threshold	N/A	Tunable τ
Primary use	Ranking	Filtering (accept/reject)
Training data	Ranked pairs	Correct/incorrect labels
Precision/recall trade-off	N/A	Controlled via τ

Table 44: State properties before and after Verifier Model training.

Critic Model

Mathematical Foundation

A Critic Model generates natural-language *critiques* of candidate outputs and optionally a numerical score. It is trained on (output, critique, revision) triplets. The critique generation loss:

$$\mathcal{L}_{\text{crit}} = -\sum_{t} \log P_\phi(c_t \mid x, y, c_{<t}) \tag{118}$$

where c is the gold critique. Critics enable self-refinement loops: the model generates y, the critic evaluates it producing c, and the model revises $y \to y'$ conditioned on (x, y, c):

$$y' \sim P_\theta(\cdot \mid x, y, c) \tag{119}$$

Architecture Flow Diagram

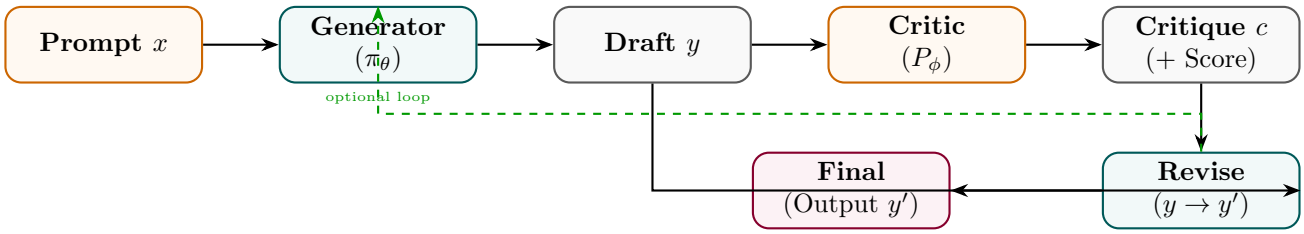


Figure 56: Critic Model: generates natural-language critique of a draft; generator revises using the critique.

State Transition Table

Property	Before	After
Feedback type	Scalar reward	Natural-language critique
Interpretability	Low (scalar)	High (textual feedback)
Revision loop	Not applicable	Generator refines on critique
Training data	Preference pairs	(output, critique, revision) triplets
Use case	RM scoring	Self-refinement / RLHF

Table 45: State properties before and after Critic Model training.

Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods — Part 3: Data Generation, Curation, Distillation, Compression, Agentic, Multimodal, Knowledge Injection, and Deployment Chief Strategist J June 13, 2026

Contents

Part VII
Synthetic Data Generation

Self-Instruct

Mathematical Foundation

Self-Instruct bootstraps an instruction dataset from a small seed pool $\mathcal{S}$ by using the LLM itself as a generator. At each iteration, k seed instructions are sampled and the model generates a new instruction $\hat{x}$, its input, and output:

$$(\hat{x}, \hat{i}, \hat{y}) \sim P_\theta(\cdot \mid \text{few-shot}(\mathcal{S})) \tag{120}$$

Generated examples pass a quality filter (ROUGE similarity $< \delta$ to existing pool, length check):

$$\mathcal{S} \leftarrow \mathcal{S} \cup \{(\hat{x}, \hat{i}, \hat{y})\} \quad \text{iff} \quad \max_{s \in \mathcal{S}} \text{ROUGE}(\hat{x}, s) < \delta \tag{121}$$

The growing pool is used to fine-tune θ, improving the quality of subsequent generations.

Architecture Flow Diagram

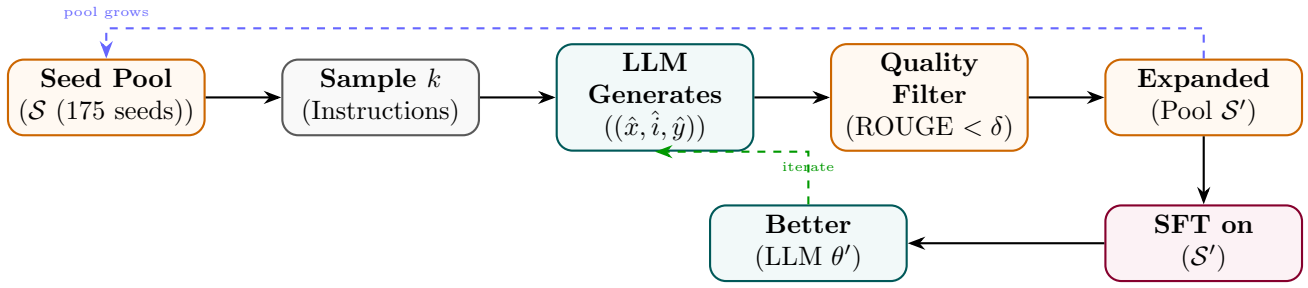


Figure 57: Self-Instruct: LLM generates new instructions from seed pool; quality filter expands the pool for SFT.

State Transition Table

Property	Before	After
Pool size	175 seed instructions	52,000+ generated
Annotation source	Human-written seeds	LLM-generated
Diversity filter	N/A	ROUGE deduplication
Training data cost	Low (seeds only)	Essentially free to scale
Instruction following	Weak	Strong

Table 46: State properties before and after Self-Instruct data generation.

Evol-Instruct

Mathematical Foundation

Evol-Instruct evolves existing instructions into harder variants via two mutation strategies. *In-depth* evolution adds constraints, increases steps, or requires deeper reasoning. *In-breadth* evolution creates new topic-adjacent instructions:

$$\hat{x} = M_{\text{depth}}(x) \quad \text{or} \quad \hat{x} = M_{\text{breadth}}(x), \qquad \hat{x} \sim P_{\text{teacher}}(\cdot \mid \text{evolve-prompt}, x) \tag{122}$$

Evolved instructions are validated (non-empty, no refusal, dissimilar from parent) and collected into $\mathcal{D}_{\text{evol}}$. Difficulty increases monotonically with evolution rounds E:

$$\mathbb{E}[\text{difficulty}(\hat{x}_E)] > \mathbb{E}[\text{difficulty}(\hat{x}_{E-1})] \tag{123}$$

Architecture Flow Diagram

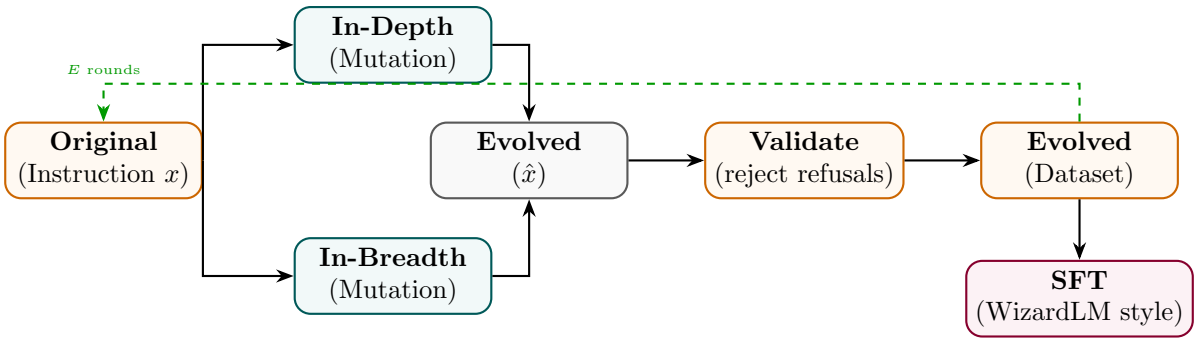


Figure 58: Evol-Instruct: in-depth and in-breadth mutations generate harder variants; validated pool used for SFT.

State Transition Table

Property	Before	After
Instruction difficulty	Low (seed)	Progressively higher (per round E)
Instruction diversity	Limited	Broad via breadth mutations
Data source	Human-written	LLM-evolved
Evolution rounds	0	E (typically 4–8)
Model performance	Baseline	WizardLM-style improvements

Table 47: State properties before and after Evol-Instruct.

Evol-Code

Mathematical Foundation

Evol-Code applies the same evolutionary principle to programming tasks, using code-specific mutation operators: adding input/output constraints, combining two functions, increasing time/space complexity, or switching programming language:

$$\hat{p} = M_{\text{code}}(p), \quad M \in \{\text{constrain}, \text{combine}, \text{optimize}, \text{translate}\} \tag{124}$$

Each evolved problem $\hat{p}$ is solved by a teacher model, and the (problem, solution) pair is validated by unit-test execution:

$$(\hat{p}, \hat{s}) \in \mathcal{D}_{\text{code}} \iff \text{unit\_tests}(\hat{s}, \hat{p}) = \text{PASS} \tag{125}$$

Architecture Flow Diagram

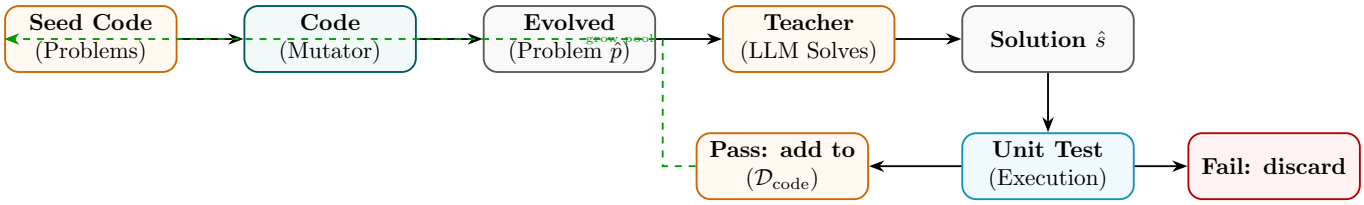


Figure 59: Evol-Code: code problems are mutated, solved by teacher, validated by unit tests before dataset inclusion.

State Transition Table

Property	Before	After
Problem complexity	Simple seed tasks	Multi-constraint, optimised
Validation method	Human review	Automated unit tests
Data quality gate	None	Unit-test pass rate
Programming languages	Fixed	Multi-language via translate mutation
Code reasoning ability	Baseline	Significantly improved

Table 48: State properties before and after Evol-Code data generation.

Synthetic Data Generation

Mathematical Foundation

A powerful teacher model P_T generates synthetic (input, output) pairs from topic/schema prompts, which are then used to train a smaller student P_S. The synthetic data distribution approximates the real task distribution P_{real}:

$$\mathcal{D}_{\text{synth}} = \{(x_i, y_i) : x_i \sim P_{\text{topic}},\ y_i \sim P_T(\cdot \mid x_i)\} \tag{126}$$

Quality is improved by filtering on a scoring model or perplexity threshold τ_p:

$$(x, y) \in \mathcal{D}_{\text{final}} \iff \text{score}(x, y) \geq \tau_s \ \text{ and } \ \text{PPL}(y) \leq \tau_p \tag{127}$$

Architecture Flow Diagram

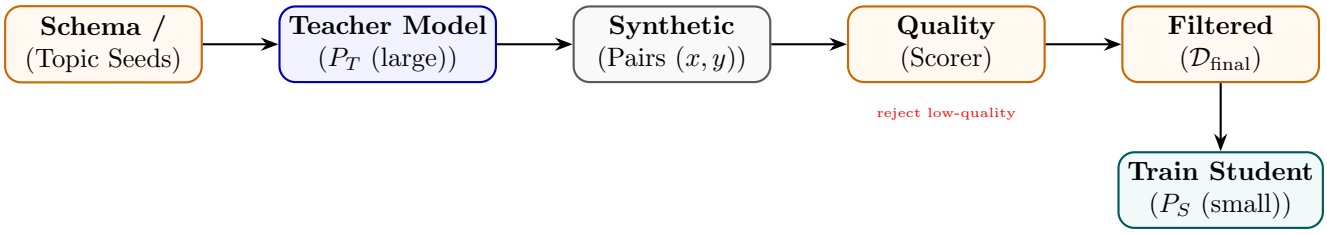


Figure 60: Synthetic Data Generation: teacher generates pairs from schema seeds; quality filter builds training set for student.

State Transition Table

Property	Before	After
Data source	Human annotation	LLM-generated at scale
Annotation cost	High	Very low (API cost)
Data volume	Thousands	Millions scalable
Quality control	Human review	Automated scoring
Domain coverage	Limited	Any topic via schema

Table 49: State properties before and after synthetic data generation.

Rejection Sampling

Mathematical Foundation

Rejection Sampling generates N candidate completions from the current policy and keeps only those that pass a quality threshold, building a filtered fine-tuning corpus:

$$\mathcal{D}_{\text{RS}} = \{(x, y^{(k)}) : y^{(k)} \sim \pi_\theta(x),\ r(x, y^{(k)}) \geq \tau,\ k = 1..N\} \tag{128}$$

Expected acceptance rate at threshold τ: $\alpha = P_{y\sim\pi_\theta}[r(x,y) \geq \tau]$. After SFT on $\mathcal{D}_{\text{RS}}$, the model's probability mass shifts toward high-reward completions:

$$\mathbb{E}_{y\sim\pi_{\theta'}}[r(x,y)] > \mathbb{E}_{y\sim\pi_\theta}[r(x,y)] \tag{129}$$

Architecture Flow Diagram

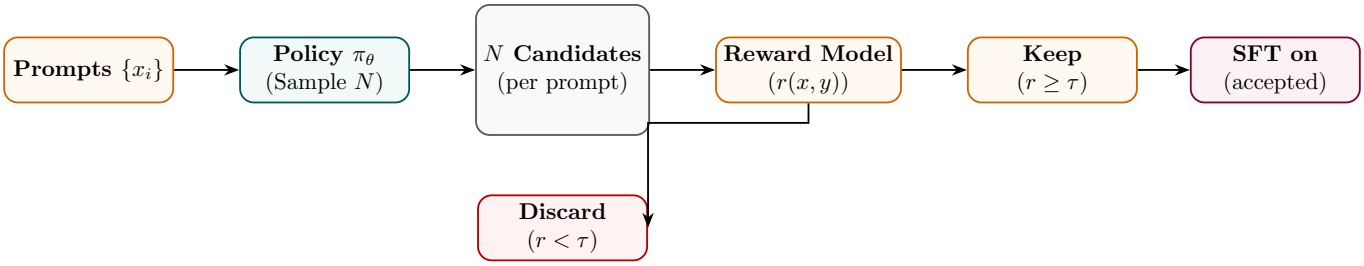


Figure 61: Rejection Sampling: N completions generated; only those above reward threshold kept for SFT.

State Transition Table

Property	Before	After
Training data quality	Mixed	High (above threshold τ)
Samples per prompt	1	N (with αN accepted)
Reward model	Optional	Required as filter
Iteration benefit	None	Compound with model updates
RL complexity	None	None (SFT only)

Table 50: State properties before and after Rejection Sampling.

Best-of-N Sampling

Mathematical Foundation

Best-of-N (BoN) sampling selects the highest-reward completion from N independent samples at inference time:

$$y^* = \arg\max_{k \in 1..N} r(x, y^{(k)}), \quad y^{(k)} \sim \pi_\theta(\cdot \mid x) \tag{130}$$

The expected reward improvement over single-sample baseline scales as:

$$\mathbb{E}[\max_k r(x, y^{(k)})] \approx \mu_r + \sigma_r \cdot \Phi^{-1}\left(\frac{N}{N+1}\right) \tag{131}$$

where μ_r, σ_r are the reward mean and std under π_θ, and Φ^{-1} is the inverse normal CDF. BoN can also be used to build SFT datasets by collecting (x, y^*) pairs.

Architecture Flow Diagram

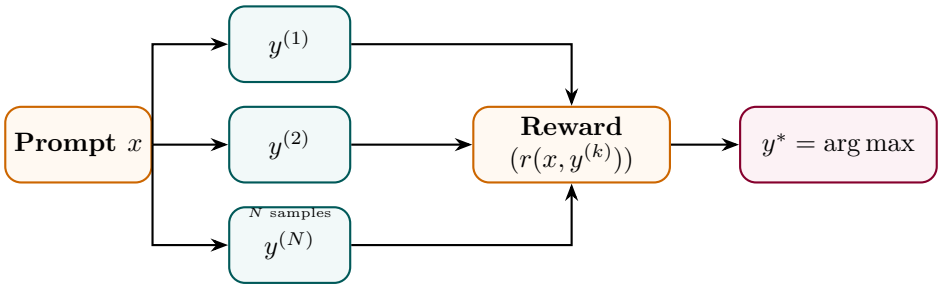


Figure 62: Best-of-N: N samples scored by reward model; highest reward selected as final output.

State Transition Table

Property	Before (single)	After (BoN)
Samples per query	1	N
Compute at inference	$1\times$	$N\times$
Expected reward	μ_r	$\mu_r + O(\sigma_r \log N)$
Model weights	Unchanged	Unchanged
Best usage	Cheap inference	Quality-critical generation

Table 51: State properties comparing single-sample and Best-of-N sampling.

Part VIII
Data Curation

Data Filtering

Mathematical Foundation

Data filtering removes low-quality examples from a raw corpus using a set of quality signals $\{q_1, \ldots, q_M\}$. A sample (x, y) is accepted iff it passes all signal thresholds:

$$(x, y) \in \mathcal{D}_{\text{clean}} \iff \bigwedge_{m=1}^{M} q_m(x, y) \geq \tau_m \tag{132}$$

Common signals include perplexity under a small LM (detects gibberish), toxic content classifiers, length ratios, and language identification. A learned quality classifier c_ϕ can consolidate multiple signals:

$$c_\phi(x, y) = \sigma(w^\top [q_1(x, y), \ldots, q_M(x, y)]) \tag{133}$$

Architecture Flow Diagram

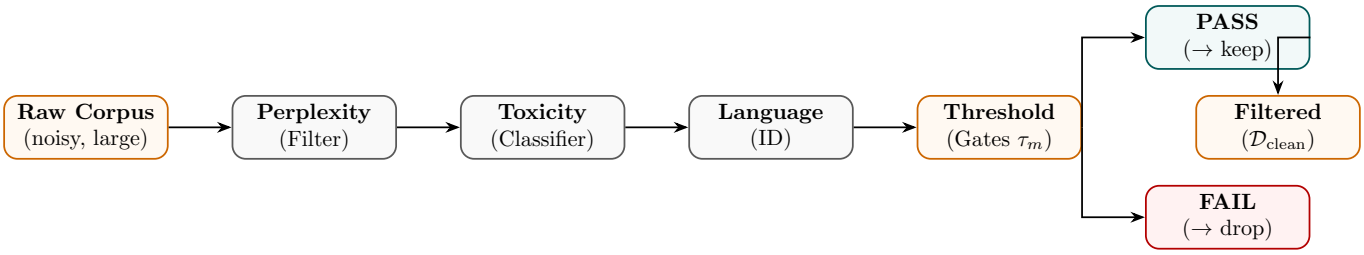


Figure 63: Data filtering pipeline: multiple quality signals applied in sequence; only passing samples retained.

State Transition Table

Property	Before	After
Corpus size	Very large (noisy)	Smaller (high quality)
Noise level	High	Low
Domain coverage	Broad (noisy)	Focused (clean)
Filter signals	None	M quality signals
Training efficiency	Low	Higher (less wasted compute)

Table 52: State properties before and after data filtering.

Deduplication

Mathematical Foundation

Deduplication removes near-duplicate documents to prevent memorisation and improve sample diversity. MinHash LSH estimates Jaccard similarity between documents:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}[h_k(A) = h_k(B)] \tag{134}$$

where $\{h_k\}$ are K independent hash functions and $h_k(A) = \min_{w \in \text{ngrams}(A)} h_k(w)$. Documents with $J(A, B) > \tau_J$ are considered duplicates and one is removed:

$$\mathcal{D}_{\text{dedup}} = \{d_i : \forall j < i,\ J(d_i, d_j) \leq \tau_J\} \tag{135}$$

Architecture Flow Diagram

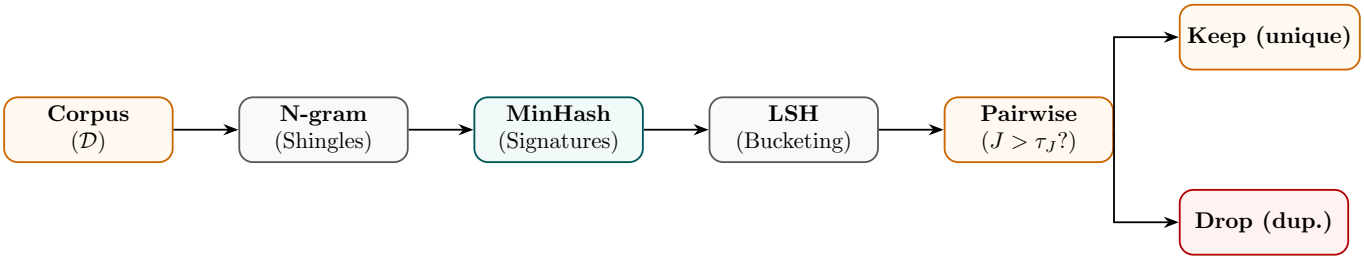


Figure 64: Deduplication: MinHash signatures group near-duplicates via LSH; high-similarity pairs discarded.

State Transition Table

Property	Before	After
Near-duplicate rate	15–30% (web data)	Near zero
Memorisation risk	High	Reduced
Effective token diversity	Low	High
Corpus size	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}_{\text{dedup}}\rvert < \lvert\mathcal{D}\rvert$
Compute wasted on duplicates	High	Eliminated

Table 53: State properties before and after deduplication.

Active Learning

Mathematical Foundation

Active Learning selects the most *informative* unlabelled examples for annotation, maximising model improvement per annotation dollar. Uncertainty sampling selects x^* with highest predictive entropy:

$$x^* = \arg\max_{x \in \mathcal{U}} \mathbb{H}[P_\theta(y \mid x)] = \arg\max_x -\sum_y P_\theta(y|x) \log P_\theta(y|x) \tag{136}$$

Query-by-committee uses disagreement between an ensemble $\{\theta_1, \ldots, \theta_C\}$:

$$x^* = \arg\max_{x \in \mathcal{U}} \frac{1}{C} \sum_c \mathbb{KL}[P_{\theta_c}(\cdot|x) \| \bar{P}(\cdot|x)] \tag{137}$$

Architecture Flow Diagram

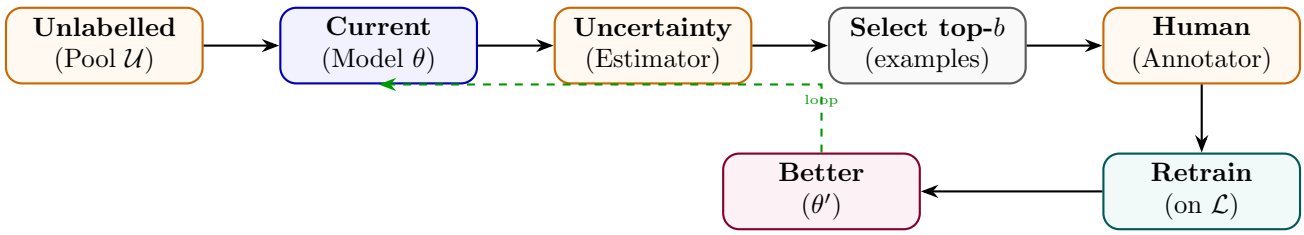


Figure 65: Active Learning: uncertainty estimator selects most informative examples; human annotates; model retrains.

State Transition Table

Property	Before	After
Annotation strategy	Random sampling	Uncertainty-driven
Labels per accuracy gain	Many	Few (targeted)
Unlabelled pool	Fully unused	Iteratively queried
Annotation budget	Fixed	Allocated by informativeness
Model improvement per label	Low	High

Table 54: State properties before and after Active Learning.

Curriculum Learning

Mathematical Foundation

Curriculum Learning trains on examples ordered by difficulty $d(x, y)$, starting with easy examples and progressively introducing harder ones. A competence function $c(t) \in [0, 1]$ controls what fraction of the difficulty range is accessible at step t:

$$c(t) = \min\left(1,\ c_0 + (1 - c_0)\frac{t}{T_{\text{ramp}}}\right) \tag{138}$$

The training distribution at step t only samples from examples with $d(x, y) \leq c(t) \cdot d_{\max}$:

$$\mathcal{D}_t = \{(x, y) : d(x, y) \leq c(t) \cdot d_{\max}\}, \quad (x, y) \sim \text{Uniform}(\mathcal{D}_t) \tag{139}$$

Architecture Flow Diagram

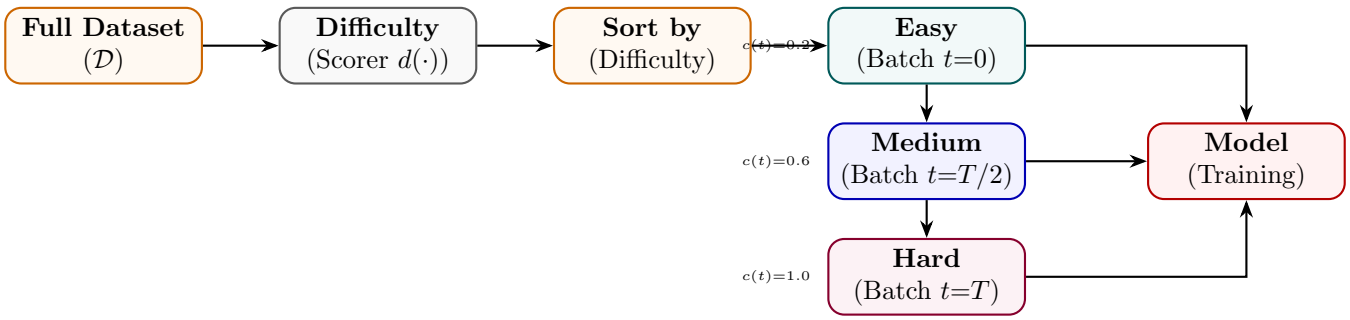


Figure 66: Curriculum Learning: examples sorted by difficulty; competence function progressively unlocks harder batches.

State Transition Table

Property	Before	After
Training order	Random	Difficulty-ordered
Early training batches	Mixed difficulty	Easy only
Convergence speed	Slower	Faster (easier start)
Final performance	Baseline	Improved
Difficulty metric	N/A	Perplexity, length, or task-specific

Table 55: State properties before and after Curriculum Learning.

Part IX
Knowledge Distillation

Knowledge Distillation

Mathematical Foundation

Knowledge distillation transfers dark knowledge from a large teacher model f_{teacher} to a compact student model f_{student}. Let z_s and z_t be the logits of the student and teacher models respectively. The soft probability distribution at temperature T is computed as:

$$p_i(z, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{140}$$

The total distillation loss is a weighted combination of hard target cross-entropy and soft target Kullback-Leibler (KL) divergence:

$$\mathcal{L}_{\text{KD}} = (1-\alpha)\mathcal{L}_{\text{CE}}(y, p(z_s, 1)) + \alpha T^2 D_{\text{KL}}\big(p(z_t, T) \parallel p(z_s, T)\big) \tag{141}$$

where $\alpha \in [0, 1]$ balances the loss terms, and the temperature scaling factor T^2 scales the soft gradient updates to remain invariant of temperature changes:

$$\frac{\partial \mathcal{L}_{\text{KL}}}{\partial z_{s,i}} = \frac{1}{T}\big(p_i(z_s, T) - p_i(z_t, T)\big) \tag{142}$$

Architecture Flow Diagram

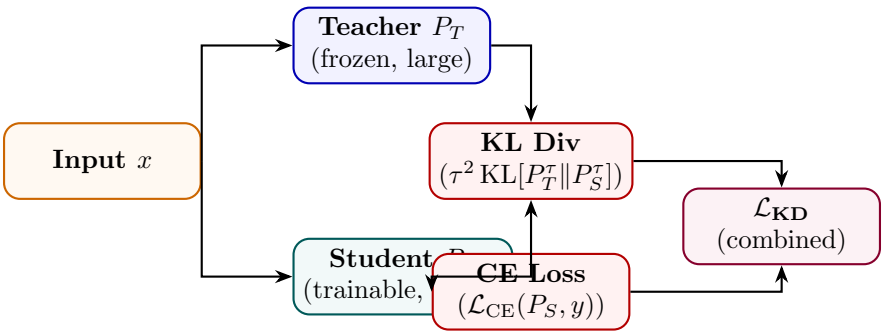


Figure 67: Knowledge Distillation: student learns from both hard labels (CE) and teacher soft logits (KL) simultaneously.

State Transition Table

Property	Before	After
Student accuracy	Low (random)	Near-teacher accuracy
Training signal	Hard labels only	Hard labels + soft teacher logits
Temperature τ	N/A	> 1 (typically 2–5)
Dark knowledge transfer	None	Via inter-class similarities
Student size	Large (teacher)	Small ($\sim$10–100$\times$ fewer params)

Table 56: State properties before and after Knowledge Distillation.

Response Distillation

Mathematical Foundation

Response Distillation trains the student by directly imitating the teacher's *full output sequence* rather than soft logits. The student minimises cross-entropy against teacher-generated responses:

$$\mathcal{L}_{\text{RD}} = -\sum_{(x,y_T)\in\mathcal{D}_T}\sum_{t}\log P_S(y_{T,t} \mid x, y_{T,<t}) \tag{143}$$

where $y_T \sim P_T(\cdot \mid x)$ are teacher completions. This is simpler than logit matching (no access to teacher logits required) and enables distillation from black-box APIs.

Architecture Flow Diagram

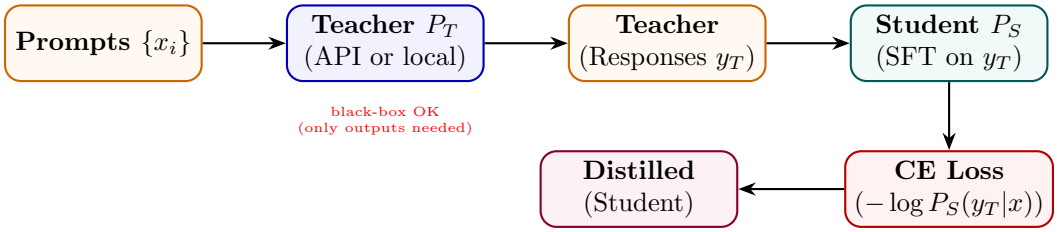


Figure 68: Response Distillation: teacher generates responses; student trained via SFT to imitate those outputs.

State Transition Table

Property	Before (KD)	After (Response KD)
Teacher access required	Logits (white-box)	Outputs only (black-box OK)
Loss type	KL on logits	CE on token sequence
Sequence-level knowledge	Partial	Full (complete generation)
Implementation complexity	High	Same as SFT
Applicable to proprietary APIs	No	Yes

Table 57: State properties comparing logit-level KD and Response Distillation.

Chain-of-Thought Distillation

Mathematical Foundation

CoT Distillation transfers the teacher's *reasoning traces* to the student, so the student learns not just the answer but how to think step by step. The student is trained on (x, c_T, y_T) triplets where c_T is the teacher-generated chain:

$$\mathcal{L}_{\text{CoTD}} = -\sum_{(x,c_T,y_T)} \left[\sum_m \log P_S(c_{T,m} \mid x, c_{T,<m}) + \sum_t \log P_S(y_{T,t} \mid x, c_T, y_{T,<t}) \right] \tag{144}$$

This allows a small student to match the accuracy of a large teacher that reasons step by step.

Architecture Flow Diagram

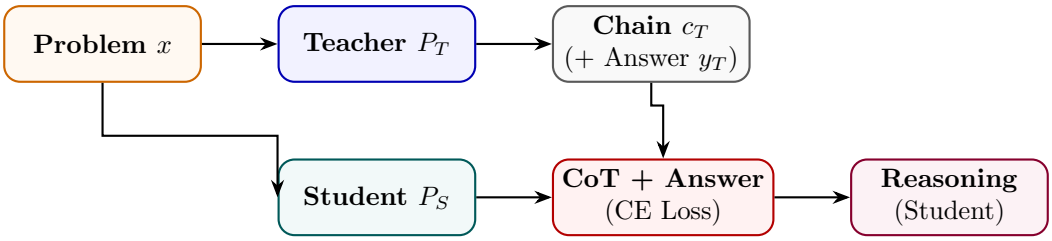


Figure 69: CoT Distillation: teacher's reasoning chains and answers jointly train student to produce similar traces.

State Transition Table

Property	Before	After
Student reasoning	Direct answer	Step-by-step chain
Training target	Answer only	Chain + answer
Reasoning data source	None	Teacher-generated
Multi-step problem accuracy	Low	Significantly higher
Output verbosity	Terse	Explanatory

Table 58: State properties before and after CoT Distillation.

Policy Distillation

Mathematical Foundation

Policy Distillation minimises the KL divergence between a teacher policy π_T and student policy π_S over the full output distribution at every decoding step, not just the final token:

$$\mathcal{L}_{\text{PD}} = \mathbb{E}_{x\sim\mathcal{D}} \sum_t \mathbb{KL}[\pi_T(\cdot \mid x, y_{<t}) \| \pi_S(\cdot \mid x, y_{<t})] \tag{145}$$

Unlike response distillation (which uses sampled strings), policy distillation uses teacher token-level distributions and requires teacher logit access. This aligns the student's generation *process* with the teacher's, not just its outputs.

Architecture Flow Diagram

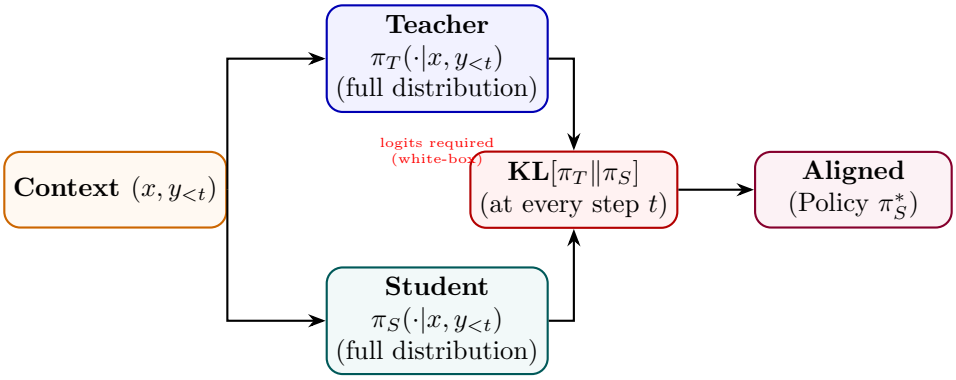


Figure 70: Policy Distillation: KL divergence between teacher and student distributions minimised at every token step.

State Transition Table

Property	Before (Response KD)	After (Policy KD)
Teacher signal	Sampled string	Full token distribution
KL divergence	N/A	Minimised per step
Teacher access	Black-box	White-box (logits)
Process alignment	Output only	Step-by-step
Overconfidence risk	Higher	Lower (distribution match)

Table 59: State properties comparing Response and Policy Distillation.

Reward Distillation

Mathematical Foundation

Reward Distillation compresses a large teacher reward model r_T into a smaller student reward model r_S by regression on teacher-generated scores:

$$\mathcal{L}_{\text{RewardD}} = \mathbb{E}_{(x,y)}\left[(r_T(x,y) - r_S(x,y))^2\right] \tag{146}$$

The student can also be trained with a ranking-consistency loss to preserve relative orderings even if absolute score values differ:

$$\mathcal{L}_{\text{rank}} = -\log \sigma(r_S(x,y_w) - r_S(x,y_l)) \text{ whenever } r_T(x,y_w) > r_T(x,y_l) \tag{147}$$

Architecture Flow Diagram

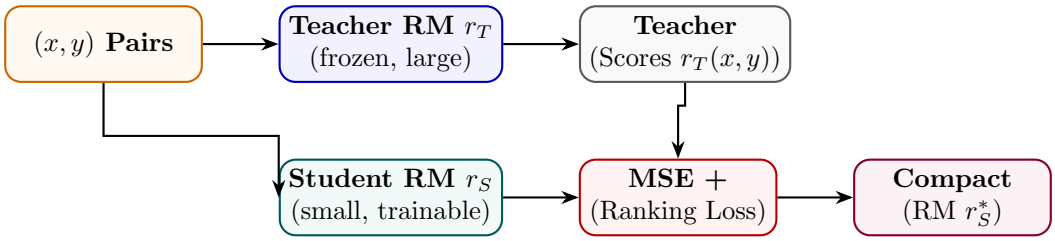


Figure 71: Reward Distillation: small student reward model trained via regression on large teacher reward scores.

State Transition Table

Property	**Before**	**After**
Reward model size	Large teacher	Small student
Inference latency	Slow	Fast
Score accuracy	High (teacher)	Near-teacher (distilled)
Training objective	Human preferences	Teacher score regression
Deployment cost	High	Low

Table 60: State properties before and after Reward Distillation.

Retrieval Distillation

Mathematical Foundation

Retrieval Distillation trains a student retriever q_S to match the relevance judgments of a larger teacher retriever q_T (or cross-encoder) without requiring access to teacher gradients. The student minimises the KL divergence between teacher and student retrieval distributions over a document set $\mathcal{C}$:

$$\mathcal{L}_{\text{RD}} = \mathbb{KL}[P_T(\cdot \mid q) \| P_S(\cdot \mid q)], \quad P(d \mid q) \propto \exp(q_\phi(q)^\top e(d)) \tag{148}$$

where q_ϕ is the query encoder and $e(d)$ is the document embedding.

Architecture Flow Diagram

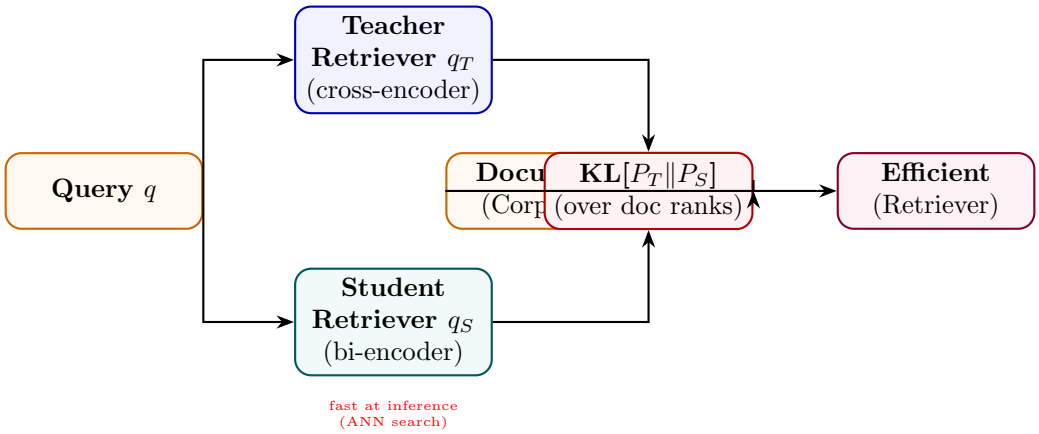


Figure 72: Retrieval Distillation: student bi-encoder trained to match teacher cross-encoder relevance distributions.

State Transition Table

Property	Before (Teacher)	After (Student)
Retriever type	Cross-encoder (slow)	Bi-encoder (fast ANN)
Query latency	$O(\lvert\mathcal{C}\rvert)$	$O(\log \lvert\mathcal{C}\rvert)$
Ranking quality	Very high	Near-teacher
Index requirement	None	Pre-computed embeddings
Training signal	Human judgments	Teacher soft scores

Table 61: State properties comparing teacher cross-encoder and student bi-encoder after retrieval distillation.

Part X
Model Compression

Quantization-Aware Training (QAT)

Mathematical Foundation

QAT simulates low-bit precision during model training, allowing the optimizer to adjust parameters to compensate for quantization errors. Let w be the continuous weights. The quantized representation $\bar{w}$ is calculated as:

$$\bar{w} = S \cdot \text{clip}\left(\text{round}\left(\frac{w}{S}\right) - Z,\, q_{\min},\, q_{\max}\right) \tag{149}$$

where S is the quantization scale factor, Z is the zero-point, and $[q_{\min}, q_{\max}]$ defines the integer boundary (e.g. $[-128, 127]$ for 8-bit quantization). Because the rounding function has zero gradients almost everywhere ($\frac{\partial \text{round}(x)}{\partial x} = 0$), standard backpropagation fails. QAT resolves this by applying the Straight-Through Estimator (STE) during the backward pass, approximating the gradient of the rounding operator as identity:

$$\frac{\partial \bar{w}}{\partial w} \approx \begin{cases} 1 & \text{if } w \in [w_{\min}, w_{\max}] \\ 0 & \text{otherwise} \end{cases} \tag{150}$$

Architecture Flow Diagram

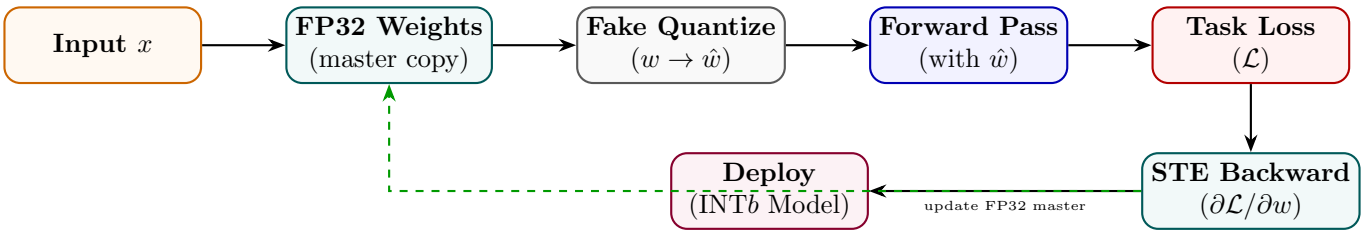


Figure 73: QAT: fake quantization in forward pass; STE allows gradients to flow to FP32 master weights.

State Transition Table

Property	Before (PTQ)	After (QAT)
Quantization awareness	None during training	Simulated in forward pass
Accuracy loss	1–5% degradation	<1% typical
Calibration data	Small set required	Full training data
Weight format	FP32	FP32 master, INTb deploy
Gradient flow	Normal	Via STE

Table 62: State properties comparing Post-Training Quantization and QAT.

Pruning

Mathematical Foundation

Pruning removes weights whose contribution to the model output is below a threshold, producing a sparse network. Magnitude-based unstructured pruning removes the fraction p of weights with the smallest absolute values:

$$m_i = \mathbf{1}[|w_i| \geq \tau_p], \quad W_{\text{sparse}} = W \odot m \tag{151}$$

where τ_p is chosen to achieve sparsity ratio $s = 1 - \|m\|_1 / \|m\|_0$. Structured pruning removes entire heads or neurons (rows/columns) and is hardware-friendly:

$$\text{importance}(h) = \sum_{i \in h} |w_i| \cdot |\nabla_{w_i} \mathcal{L}| \tag{152}$$

Architecture Flow Diagram

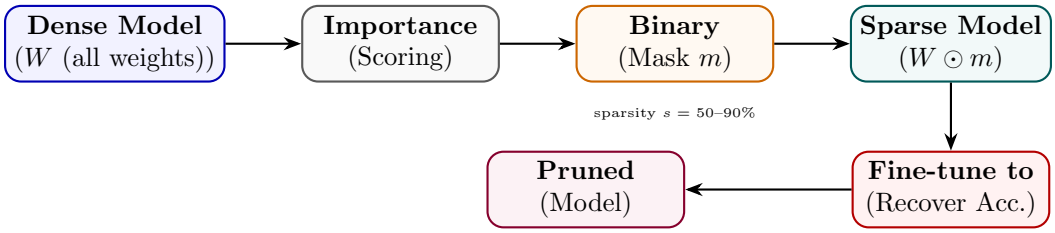


Figure 74: Pruning: importance scores determine binary mask; sparse model fine-tuned to recover accuracy.

State Transition Table

Property	Before	After
Model sparsity	0%	50–90%
Parameter count	$\|\theta\|$	$(1-s)\|\theta\|$
Inference FLOPs	Baseline	Reduced (structured)
Accuracy	Baseline	Slightly reduced
Memory footprint	Full	Reduced by s

Table 63: State properties before and after pruning.

Post-Training Quantization (PTQ)

Mathematical Foundation

PTQ converts a trained FP32 model to reduced precision (INT8, INT4) without additional training. Per-channel scale factors s_c are calibrated on a small unlabelled dataset $\mathcal{D}_{\text{cal}}$:

$$s_c = \frac{\max_{x \in \mathcal{D}_{\text{cal}}} |a_c(x)|}{2^{b-1} - 1}, \quad \hat{a}_c = \text{round}(a_c / s_c) \cdot s_c \tag{153}$$

GPTQ computes optimal quantization for each weight row by minimising the second-order reconstruction error using the Hessian:

$$\min_{\hat{W}} \|\hat{W}X - WX\|_F^2 \approx \sum_i (w_i - \hat{w}_i)^2 [H^{-1}]_{ii} \tag{154}$$

Architecture Flow Diagram

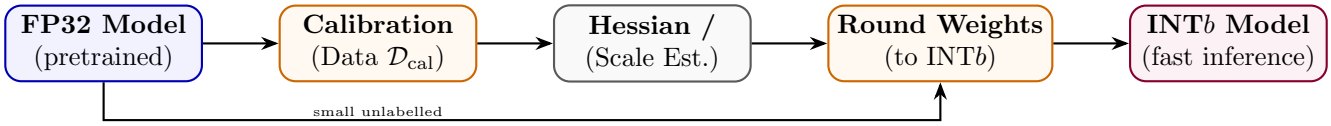


Figure 75: PTQ: calibration statistics guide per-channel scales; weights rounded to INTb without retraining.

State Transition Table

Property	Before	After
Weight precision	FP32 / BF16	INT8 / INT4
Memory usage	$4 \times n$ bytes	1–$0.5 \times n$ bytes
Retraining required	N/A	None (calibration only)
Accuracy loss	0%	0.5–3% (INT8), higher INT4
Calibration data	Not needed	Small sample (128–512 examples)

Table 64: State properties before and after Post-Training Quantization.

Part XI
Agentic Training

Tool-Use Training

Mathematical Foundation

Tool-Use Training teaches the model to generate structured tool invocations interleaved with reasoning (ReAct-style). The training data consists of trajectories $\tau = (t_1, a_1, o_1, \ldots, t_N, a_N, o_N, y)$ where t_i is a thought, a_i is a tool call, and o_i is the observation. The model is trained to predict thought and action tokens conditioned on prior context and observations:

$$\mathcal{L}_{\text{tool}} = -\sum_{i=1}^{N} \sum_{j} \log P_\theta(t_{i,j} \mid \text{context}_{<i}) + \log P_\theta(a_{i,j} \mid \text{context}_{<i}, t_i) \tag{155}$$

Architecture Flow Diagram

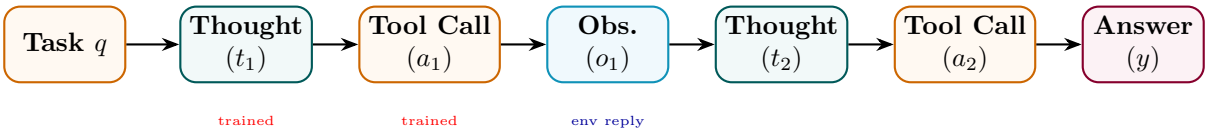


Figure 76: Tool-Use Training: thought-action-observation trajectory; model trained on thought and action tokens.

State Transition Table

Property	Before	After
Output format	Free text	Thought + tool call interleaved
Tool invocation	None	Structured API calls
Observation integration	N/A	Conditioned on tool results
Multi-step planning	Weak	Strong
Training data type	Static QA	Trajectory (thought, action, obs)

Table 65: State properties before and after Tool-Use Training.

Function Calling Training

Mathematical Foundation

Function Calling Training specifically trains the model to produce well-formed JSON function-call objects given a natural-language request and a tool schema. The model maps input x and schema $S = \{f_1, \ldots, f_K\}$ to a structured call $c = \{\texttt{name}\text{: } f_k, \texttt{args}\text{: } \{a_1{:}v_1, \ldots\}\}$:

$$\mathcal{L}_{\text{FC}} = -\log P_\theta(c \mid x, S) \tag{156}$$

where c is the ground-truth function call. Schema-aware token masking ensures the model only generates valid JSON that matches the function signature, reducing hallucination of non-existent arguments.

Architecture Flow Diagram

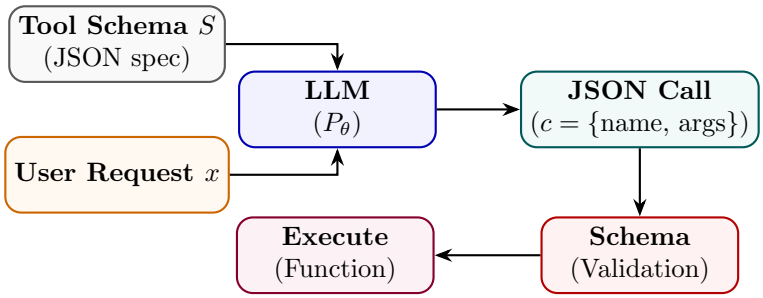


Figure 77: Function Calling Training: model receives request + schema, outputs structured JSON call validated against spec.

State Transition Table

Property	Before	After
Output format	Natural language	Structured JSON
Schema adherence	None	Enforced
Argument hallucination	High	Low
API integration	Manual parsing	Direct execution
Training data	Text completions	(request, schema, call) triplets

Table 66: State properties before and after Function Calling Training.

Multi-Agent Training

Mathematical Foundation

Multi-Agent Training trains a collection of specialised agents $\{\pi_1, \ldots, \pi_K\}$ and an orchestrator π_0 to collaboratively solve complex tasks. The orchestrator decomposes a task T into sub-tasks $\{t_1, \ldots, t_K\}$ and assigns them to specialists:

$$\pi_0 : T \to \{(t_k, k)\}_{k=1}^{K}, \quad \pi_k : t_k \to y_k, \quad y = \pi_0(\{y_k\}) \tag{157}$$

Training uses task-completion reward at the full-task level, distributed to agents via credit assignment:

$$r_k = r_{\text{task}} \cdot \mathbf{1}[\pi_k \text{ contributed to solution}] \tag{158}$$

Architecture Flow Diagram

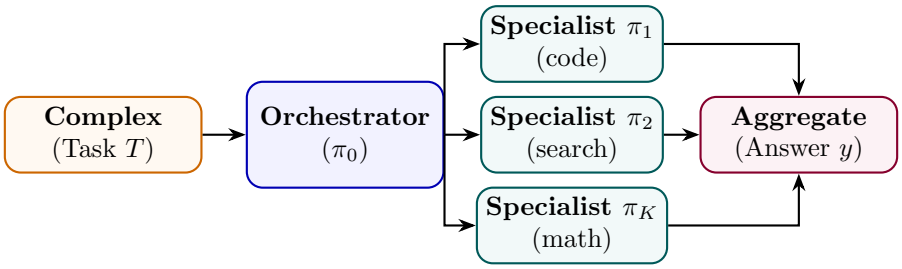


Figure 78: Multi-Agent Training: orchestrator decomposes task and assigns sub-tasks to specialised agents.

State Transition Table

Property	Before	After
Task handling	Single monolithic model	Orchestrator + specialists
Specialisation	None	Per-domain agents
Credit assignment	N/A	Per-agent contribution
Task complexity ceiling	Single-context limit	Decomposable to multiple contexts
Coordination	N/A	Learned delegation protocol

Table 67: State properties before and after Multi-Agent Training.

Part XII
Multimodal Training

Vision-Language Training

Mathematical Foundation

Vision-Language Training jointly trains a visual encoder f_v and language model f_l to process image-text pairs. A projection layer P aligns visual features to the LLM's token embedding space:

$$v_{\text{tokens}} = P(f_v(I)), \quad \tilde{x} = [v_{\text{tokens}}; E(x_{\text{text}})] \tag{159}$$

Training uses a masked language modelling or auto-regressive loss on the text tokens conditioned on visual tokens:

$$\mathcal{L}_{\text{VL}} = -\sum_t \log P_{f_l}(x_t \mid v_{\text{tokens}}, x_{<t}) \tag{160}$$

Architecture Flow Diagram

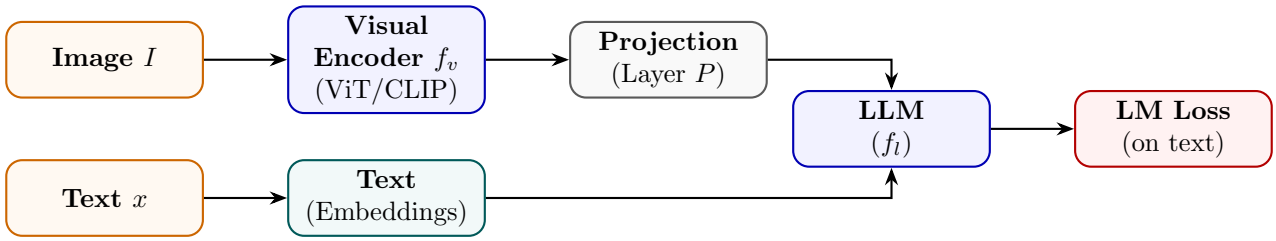


Figure 79: Vision-Language Training: visual encoder outputs projected to LLM token space; joint auto-regressive loss.

State Transition Table

Property	Before	After
Input modality	Text only	Text + images
Visual understanding	None	Strong (via encoder)
Projection layer	Not present	Trained to align modalities
Alignment quality	N/A	Measured by VQA, captioning
Training data	Text corpora	Image-caption / VQA pairs

Table 68: State properties before and after Vision-Language Training.

Grounding Training

Mathematical Foundation

Grounding Training teaches the model to link text spans to image regions (bounding boxes). Given a referring expression e and image I, the model predicts box coordinates (x_1, y_1, x_2, y_2) normalised to $[0, 1]^4$:

$$\hat{b} = f_\theta(I, e), \quad \mathcal{L}_{\text{GND}} = \text{L1}(\hat{b}, b^*) + \lambda(1 - \text{IoU}(\hat{b}, b^*)) \tag{161}$$

where b^* is the ground-truth box and IoU is intersection-over-union. The model generates boxes as text tokens (serialised coordinates), enabling unified seq2seq training.

Architecture Flow Diagram

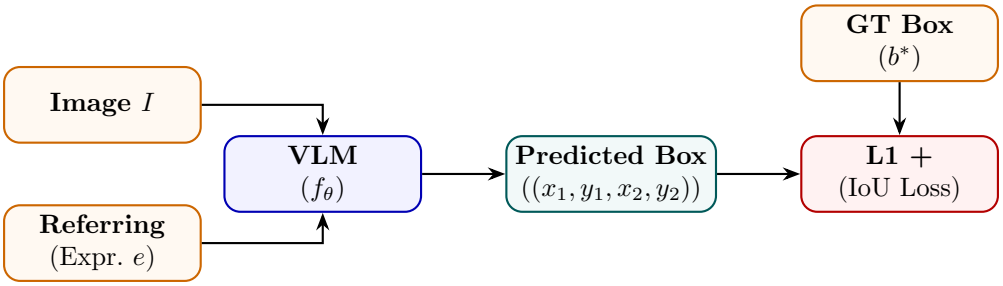


Figure 80: Grounding Training: VLM predicts bounding box coordinates from image and referring expression.

State Transition Table

Property	Before	After
Output type	Text description	Bounding box coordinates
Spatial understanding	Implicit	Explicit (pixel-level)
Loss type	Cross-entropy	L1 + IoU
Grounding ability	None	Text-to-region linking
Training data	Captions	(image, expression, bbox) triplets

Table 69: State properties before and after Grounding Training.

Part XIII
Knowledge Injection

Retrieval-Augmented Training

Mathematical Foundation

Retrieval-Augmented Training (RAT) trains the model to use retrieved documents $\mathcal{R}(x) = \{d_1, \ldots, d_K\}$ as context. The model generates y conditioned on the query and retrieved set:

$$P_\theta(y \mid x, \mathcal{R}(x)) = \prod_t P_\theta(y_t \mid x, d_1, \ldots, d_K, y_{<t}) \tag{162}$$

Training includes both retrieval-present and retrieval-absent examples to prevent over-reliance on context. A closed-book loss component ensures the model retains parametric knowledge:

$$\mathcal{L}_{\text{RAT}} = \lambda \, \mathcal{L}_{\text{open-book}} + (1 - \lambda) \, \mathcal{L}_{\text{closed-book}} \tag{163}$$

Architecture Flow Diagram

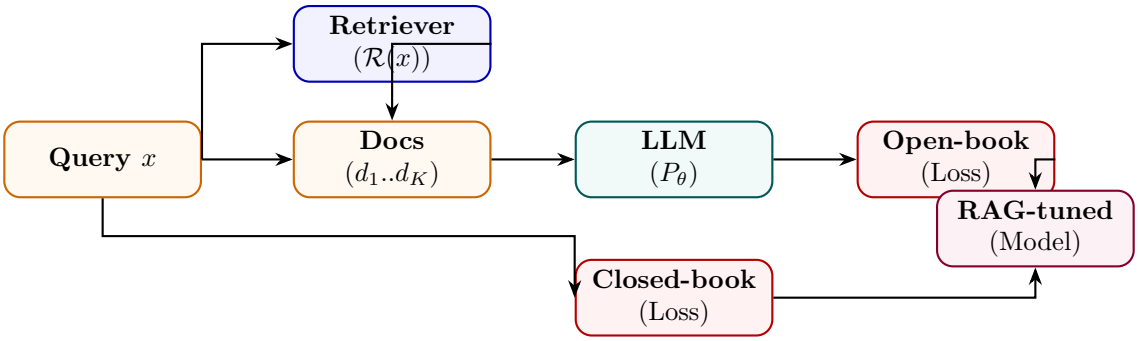


Figure 81: RAT: model trained on both retrieval-augmented and closed-book losses to balance open/parametric knowledge.

State Transition Table

Property	Before	After
Context awareness	Parametric only	Retrieved + parametric
Knowledge staleness	Fixed (training cutoff)	Dynamic (fresh retrieval)
Hallucination rate	Higher	Lower (grounded in docs)
Retriever dependency	None	Required at inference
Parametric retention	Full	Preserved (closed-book loss)

Table 70: State properties before and after Retrieval-Augmented Training.

Memory Tuning

Mathematical Foundation

Memory Tuning trains the model to store and retrieve episodic memories $\mathcal{M} = \{(k_i, v_i)\}$ in an external key-value store. The model learns to write memories via a write head w_θ and retrieve via dot-product attention:

$$\text{retrieve}(q) = \sum_i \text{softmax}(q^\top k_i / \sqrt{d}) \cdot v_i, \quad q = f_\theta^{\text{query}}(x) \tag{164}$$

Training uses a distractor task: memory-relevant queries should prefer correct memory entries, trained with a contrastive retrieval loss:

$$\mathcal{L}_{\text{mem}} = -\log \frac{\exp(q^\top k^+ / \tau)}{\sum_j \exp(q^\top k_j / \tau)} \tag{165}$$

Architecture Flow Diagram

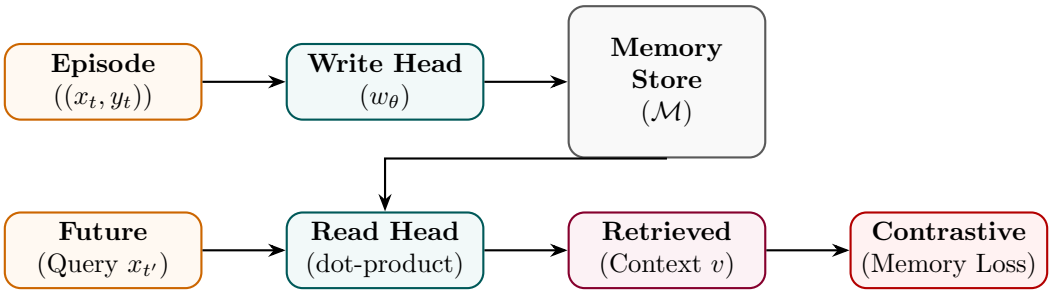


Figure 82: Memory Tuning: write head stores episodes; read head retrieves via attention; contrastive loss trains both.

State Transition Table

Property	Before	After
Memory type	Parametric only	Parametric + episodic KV store
Context window	Fixed T tokens	Unbounded (external memory)
Personalisation	None	Per-user episodic recall
Read/write mechanism	None	Learned attention-based
Long-term recall	Poor	Strong

Table 71: State properties before and after Memory Tuning.

Domain Knowledge Injection

Mathematical Foundation

Domain Knowledge Injection embeds structured domain knowledge (ontologies, knowledge graphs, fact databases) into the model. Entities e and relations r from a KG are embedded alongside token embeddings and injected via cross-attention:

$$h_t' = h_t + \text{CrossAttn}(h_t, \{e_i, r_{ij}\}_{(i,j)\in\mathcal{G}}) \tag{166}$$

Training uses a knowledge grounding loss that verifies generated text against retrieved facts:

$$\mathcal{L}_{\text{KI}} = \mathcal{L}_{\text{LM}} + \mu\,\mathcal{L}_{\text{fact-check}} \tag{167}$$

Architecture Flow Diagram

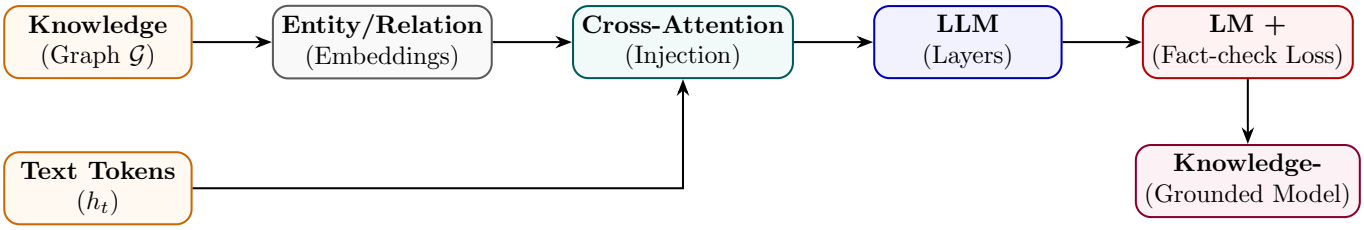


Figure 83: Domain Knowledge Injection: KG entity/relation embeddings injected into LLM via cross-attention.

State Transition Table

Property	Before	After
Knowledge source	Implicit (pretraining)	Explicit KG entities
Fact accuracy	Moderate	Significantly higher
Knowledge update	Retrain required	KG update sufficient
Cross-attention overhead	None	One extra module per layer
Domain coverage	General	Domain-specific + general

Table 72: State properties before and after Domain Knowledge Injection.

Part XIV
Deployment and Training Efficiency

Mixed Precision Training

Mathematical Foundation

Mixed Precision Training maintains FP32 master weights but computes forward and backward passes in FP16/BF16. A loss scaling factor S prevents underflow of small gradients:

$$\tilde{\mathcal{L}} = S \cdot \mathcal{L}, \quad g_{\text{FP32}} = \frac{1}{S} g_{\text{FP16}} \tag{168}$$

BF16 has the same exponent range as FP32 (8 bits) but fewer mantissa bits (7 vs 23), making it more numerically stable without loss scaling:

$$\text{memory(FP16)} = 2N \text{ bytes vs memory(FP32)} = 4N \text{ bytes} \tag{169}$$

Architecture Flow Diagram

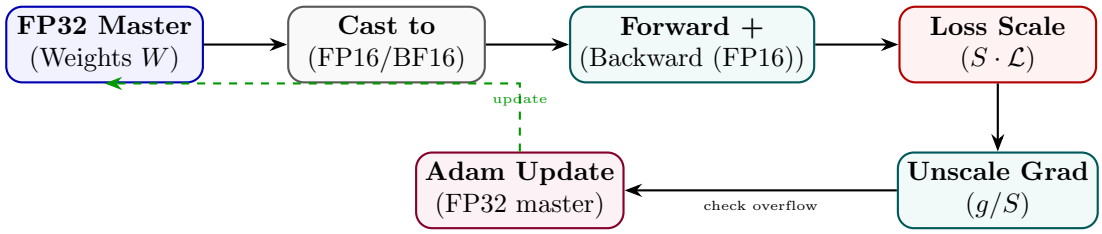


Figure 84: Mixed Precision Training: FP16 compute with FP32 master weights; loss scaling prevents gradient underflow.

State Transition Table

Property	Before (FP32)	After (Mixed FP16)
Compute precision	FP32	FP16/BF16
Memory footprint	$4N$ bytes	$2N + 4N$ (half + master)
Training speed	Baseline	2–3× faster
Numerical stability	High	Managed via loss scaling
GPU tensor cores	Unused	Utilised (FP16/BF16 native)

Table 73: State properties before and after Mixed Precision Training.

Gradient Checkpointing

Mathematical Foundation

Gradient Checkpointing trades compute for memory by not storing all intermediate activations during the forward pass. Instead, activations at checkpoint nodes $\mathcal{C} \subset \{1, \ldots, L\}$ are stored; all others are recomputed during the backward pass:

$$\text{Memory}_{\text{GC}} = O(\sqrt{L} \cdot d), \quad \text{vs} \quad O(L \cdot d) \text{ without GC} \tag{170}$$

For L layers, optimal checkpointing every $\sqrt{L}$ layers minimises memory while adding one extra forward pass overhead:

$$\text{Compute}_{\text{GC}} \approx 2 \cdot \text{Compute}_{\text{standard}} \tag{171}$$

Architecture Flow Diagram

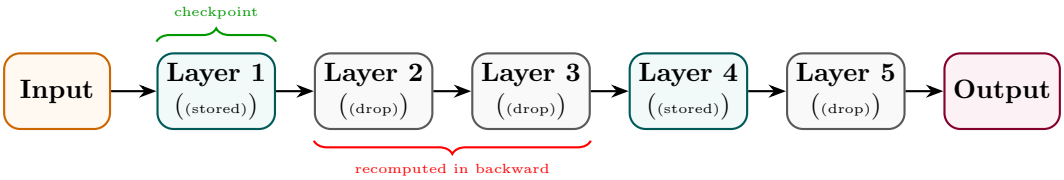


Figure 85: Gradient Checkpointing: only checkpoint activations stored; intermediate activations recomputed in backward.

State Transition Table

Property	Before	After
Activation memory	$O(L \cdot d)$	$O(\sqrt{L} \cdot d)$
Recomputation	None	Extra forward for non-checkpoints
Max batch size	Limited by activations	Larger (mem freed)
Compute overhead	Baseline	$\sim$30% more FLOPs
Large model feasibility	Memory OOM	Enabled

Table 74: State properties before and after Gradient Checkpointing.

Flash Attention

Mathematical Foundation

FlashAttention accelerates attention computation by tiling input blocks in SRAM and utilizing online softmax reduction to avoid materializing the massive $N \times N$ attention matrix in HBM. Let Q, K, V be split into blocks Q_i, K_j, V_j of sizes B_c, B_r. The online softmax computes running scaling statistics. For a query block Q_i: At step j (processing key block K_j), we compute raw attention scores:

$$S^{(j)} = \frac{Q_i K_j^\top}{\sqrt{d}} \tag{172}$$

We update running row-wise maximums $m^{(j)}$ and denominators $d^{(j)}$:

$$\tilde{m}^{(j)} = \max(S^{(j)}) \tag{173}$$

$$m^{(j)} = \max(m^{(j-1)}, \tilde{m}^{(j)}) \tag{174}$$

$$\tilde{d}^{(j)} = \sum_k \exp(S_k^{(j)} - m^{(j)}) \tag{175}$$

$$d^{(j)} = d^{(j-1)} \exp(m^{(j-1)} - m^{(j)}) + \tilde{d}^{(j)} \tag{176}$$

The running attention output block O_i is updated dynamically using:

$$O_i^{(j)} = \frac{d^{(j-1)} \exp(m^{(j-1)} - m^{(j)}) \cdot O_i^{(j-1)} + \exp(\tilde{m}^{(j)} - m^{(j)}) \cdot \tilde{P}^{(j)} V_j}{d^{(j)}} \tag{177}$$

where $\tilde{P}^{(j)} = \exp(S^{(j)} - \tilde{m}^{(j)})$. This achieves exact attention outputs with $O(N)$ memory access instead of $O(N^2)$.

Architecture Flow Diagram

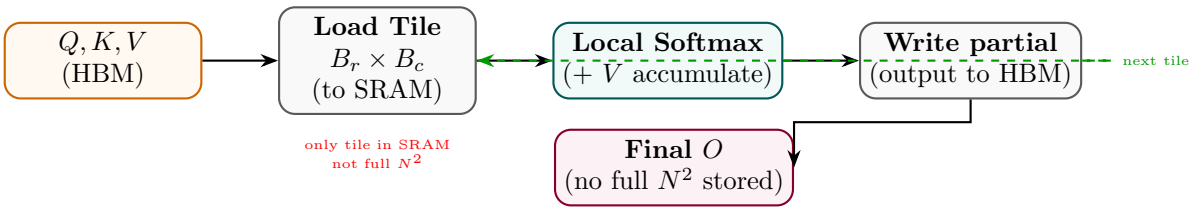


Figure 86: Flash Attention: tiled block-wise computation avoids materialising the full attention matrix in HBM.

State Transition Table

Property	Before (Standard)	After (Flash Attention)
Attention matrix memory	$O(N^2)$ HBM	$O(N)$ (tiles in SRAM)
HBM read/write passes	Multiple	Minimal (one pass)
Compute FLOPs	$O(N^2 d)$	$O(N^2 d)$ (same)
Speed (wall-clock)	Baseline	2–4× faster
Long-sequence feasibility	OOM at $N > 4096$	$N > 100{,}000$ feasible

Table 75: State properties comparing Standard and Flash Attention.

Fully Sharded Data Parallel (FSDP)

Mathematical Foundation

FSDP shards all three components of the model state (Parameters Ψ, Gradients G, and Optimizer States O) across all N available GPUs. Let the model parameters occupy $M_{\text{param}} = 2\Psi$ bytes in 16-bit precision, gradients occupy $M_{\text{grad}} = 2\Psi$ bytes, and Adam optimizer states occupy $M_{\text{opt}} = 12\Psi$ bytes (storing FP32 copies of weights, first moments, and second moments). Under standard Distributed Data Parallel (DDP), the parameters, gradients, and optimizer states are fully duplicated on each GPU:

$$M_{\text{baseline}} = 2\Psi + 2\Psi + 12\Psi = 16\Psi \text{ bytes} \tag{178}$$

FSDP shards these states across N GPUs. The peak memory occupancy per GPU is:

$$M_{\text{FSDP}} = \underbrace{\frac{2\Psi}{N}}_{\text{sharded params}} + \underbrace{\frac{2\Psi}{N}}_{\text{sharded grads}} + \underbrace{\frac{12\Psi}{N}}_{\text{sharded optimizer}} + \underbrace{2\Psi_{\text{block}}}_{\text{reconstructed block}} \tag{179}$$

During the forward pass, an 'All-Gather' operation reconstructs parameters block-by-block on the fly, which are discarded immediately after computing the block activation. During the backward pass, parameters are gathered again, gradients are computed, and a 'Reduce-Scatter' operation synchronizes and shards the gradients across GPUs.

Architecture Flow Diagram

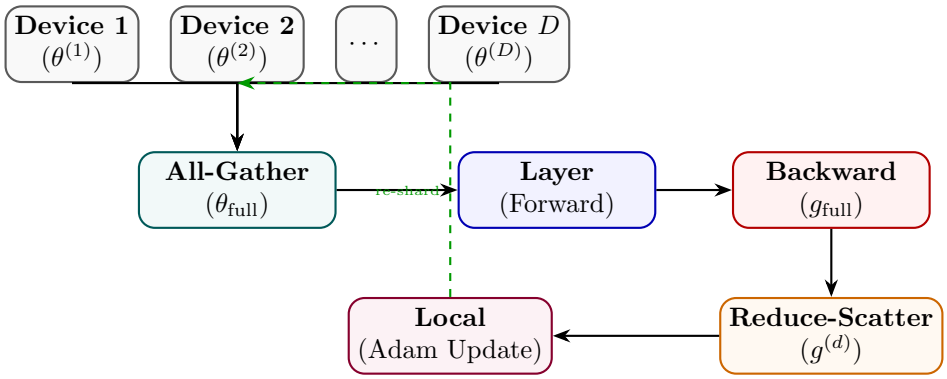


Figure 87: FSDP: parameters sharded across devices; All-Gather for compute, Reduce-Scatter for gradient aggregation.

State Transition Table

Property	Before (DDP)	After (FSDP)
Per-device param memory	$O(\|\theta\|)$	$O(\|\theta\|/D)$
Optimizer state memory	$O(\|\theta\|)$ per device	$O(\|\theta\|/D)$ per device
Communication pattern	AllReduce (gradients)	All-Gather + Reduce-Scatter
Max model size	GPU VRAM limited	$D\times$ larger
Training throughput	Baseline	Near-linear scaling

Table 76: State properties comparing DDP and FSDP.

DeepSpeed ZeRO

Mathematical Foundation

ZeRO (Zero Redundancy Optimizer) partitions the memory footprint of the model across data-parallel processes. It consists of three successive stages:

1. **ZeRO-1 (Optimizer State Partitioning)**: The optimizer states (e.g. Adam states) are sharded across N_{dp} data-parallel ranks. Parameters and gradients remain fully replicated. Peak memory per GPU is:

$$M_{\text{ZeRO-1}} = 2\Psi + 2\Psi + \frac{12\Psi}{N_{\text{dp}}} \tag{180}$$

2. **ZeRO-2 (Gradient Partitioning)**: Gradients are sharded immediately after their calculation in the backward pass. Peak memory per GPU is:

$$M_{\text{ZeRO-2}} = 2\Psi + \frac{2\Psi}{N_{\text{dp}}} + \frac{12\Psi}{N_{\text{dp}}} \tag{181}$$

3. **ZeRO-3 (Parameter Partitioning)**: Parameters are sharded across the ranks. The forward and backward passes reconstruct individual layer weights on the fly via 'All-Gather'. Peak memory per GPU is:

$$M_{\text{ZeRO-3}} = \frac{2\Psi}{N_{\text{dp}}} + \frac{2\Psi}{N_{\text{dp}}} + \frac{12\Psi}{N_{\text{dp}}} \tag{182}$$

Additionally, **ZeRO-Offload** offloads partitioned states to host CPU memory, utilizing PCIe bandwidth to scale training to trillion-parameter models on limited GPU nodes.

Architecture Flow Diagram

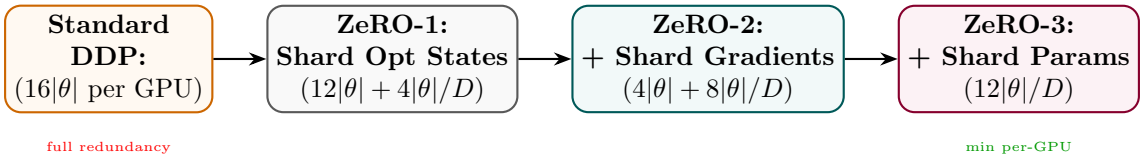


Figure 88: DeepSpeed ZeRO stages: each stage shards an additional component, progressively reducing per-GPU memory.

State Transition Table

Property	Standard DDP	ZeRO-3
Optimizer state memory	$8\|\theta\|$ per device	$8\|\theta\|/D$
Gradient memory	$4\|\theta\|$ per device	$4\|\theta\|/D$
Parameter memory	$4\|\theta\|$ per device	$4\|\theta\|/D$
Total per device	$16\|\theta\|$	$16\|\theta\|/D$
Max trainable size	GPU VRAM bound	$D\times$ larger

Table 77: Memory comparison between standard DDP and DeepSpeed ZeRO-3.